Cache Design

Extra Credit: Two-Way Set Associative Cache with LRU

VHDL Code and Simulation

CMPE 413 / CMPE 640 — Custom VLSI Design, UMBC

Student: Asif Ali

Reg No: RE16458

aali12@umbc.edu

Code and report: https://github.com/aali12-eng/CMPE413-Cache-Chip-2Way

October 29, 2026

Index

1 Description of the submission....4
1.1 Design strategy....4
1.2 Results at a glance....5
2 Specification....5
3 Design overview....7
3.1 Hierarchical block diagram....7
3.2 Design styles and why....9
3.3 Entity / architecture pairs....9
4 Building blocks....11
4.1 Course primitives: dff, Dlatch, tx....11
4.2 Shared package (cache_pkg)....12
4.3 Selector (Figure 2)....13
4.4 Cache cell (Figure 1)....16
4.5 Decoder, multiplexers and comparator....19
4.6 Registers....24
4.7 Write-signal flip-flop and clock cycle counter....29
5 Cache....35
5.1 Cache block....35
5.2 Cache way....41
5.3 Hit/Miss....46
5.4 Way select....49
5.5 LRU....53
5.6 Two-way cache (32 bytes)....59
6 State machine....67
6.1 States....68
6.2 Timing....70
6.3 Inputs, outputs and connections....70
6.4 Code....71
6.5 Testbench and simulation results....73
7 Chip (top level)....80
7.1 Code....81

7.2 Extended chip testbench....83
8 Course testbench (chip_test)....89
8.1 Sample input....89
8.2 Output and comparison with the reference....90
8.3 Waveforms....91
9 Notes for the TA: running the simulations....96
9.1 Assumptions....96
10 Breakup of work....97
Appendix A Course testbench files and output....98
Appendix B Testbench helper package....106

1 Description of the submission

This submission contains the VHDL design and simulation of the extra credit of the CMPE 413/640 project specification: "a two-way set associative cache i.e. instead of a 16 byte cache you have a 32 byte cache implementing 2 sets. For block replacement implement a least recently used (LRU) scheme." Everything else follows the main specification: byte addressable, 4-byte blocks, a 6-bit address (2-bit tag, 2-bit block offset, 2-bit byte offset), write through with no write allocate, the same chip interface, and the same timing (read hit 2 clocks, write hit or write miss 3 clocks, read miss 19 clocks).

The cache holds 2 ways (the specification's 2 sets) of 4 blocks of 4 bytes. The block offset of the address selects one block in each way, and the tag of the address is compared with the tags of both. A read miss fills an invalid way if there is one, otherwise the least recently used way; every hit and every fill makes its way the most recently used. The design reuses the direct mapped chip (the `chip design` folder) unchanged except for the cache and two control signals of the state machine.

1.1 Design strategy

- Hierarchical, structural design, built bottom-up: the course's primitive cells (`dff`, `Dlatch`, `tx`) and a small gate library form the cache cell, registers, decoders, multiplexers and counter; these form the cache, the hit/miss logic and the state machine; and these form the chip. Every level above the primitives is a netlist of instances, so each VHDL entity maps directly onto a schematic and a layout cell for Submission II.
- The cache storage uses exactly the cache cell of the specification (Figure 1): a positive level sensitive latch for Write Data, a transmission gate for Read Data, and the selector of Figure 2 that turns Chip enable and RD/WR into Write enable and Read enable. Read Data of all cells share bit lines, because a cell's transmission gate is off unless it is read.
- The chip works on the negative edge of the clock, as specified: the CPU and the memory change their signals on the positive edge, and every register of the chip samples them half a clock later. The only positive edge flip-flop makes the cache write signal, which the specification requires to be on for exactly one clock cycle from a positive edge to the next.
- Two ways built from the same cache blocks, sharing the block and byte decoders; one Hit/Miss unit per way; a way-select block that gives the hit, the way that hit and the victim way; and one LRU bit per block offset (exact LRU for two ways). The victim way is latched at the end of the tag compare, because the valid and LRU bits change while a read miss fills the block. While a read miss fills the block, each way holds its tag, valid bit and read byte, and the cache holds its hit/miss output from the end of the tag compare to the next request, so neither shows the cells being written.

- A one-hot state machine (one flip-flop per state) with a 5-bit counter sequences the four operations; its timing was taken from the specification, the desired-waveforms document and the reference output of the course testbench.
- Verification follows the hierarchy: each major block has a self-checking testbench; the chip has an extended self-checking testbench with a CPU model, a memory model and a reference model of the cache; finally the course testbench `chip_test.vhd` is run unchanged and its output is compared, line by line, with the course's reference output.

1.2 Results at a glance

- 12 of 12 self-checking testbenches pass.
- The course testbench `chip_test.vhd` runs unchanged (the component in it already matches the chip's ports) and its output matches the reference `ncsim.log` in all 70 printed clock edges (the only differences are before the first reset edge, where the reference simulator prints X and GHDL prints U for uninitialised flip-flops).
- Read hit 2 clocks, write hit and write miss 3 clocks, read miss 19 clocks, as specified.

2 Specification

Table 1: Specification summary

Item	Value
Organisation	Two-way set associative cache (extra credit)
Data	Byte addressable (8-bit data per location)
Size	2 ways ("2 sets") × 4 blocks × 4 bytes = 32 bytes
Replacement	LRU, one bit per block offset; an invalid way is filled first
Address	6 bits: 2-bit tag, 2-bit block offset, 2-bit byte offset
Write policy	Write through with no write allocate
Timing	Read hit 2 clocks; write hit / miss 3 clocks; read miss 19 clocks
Clocking	Inputs change on the positive edge; the chip works on the negative edge
Memory	Enable for one cycle with MA = tag, block offset, 00; first byte on the 8th negative edge after enable, then one byte every 2 cycles (8th, 10th, 12th, 14th)

Table 2: CPU address (CA)

Bits	5 – 4	3 – 2	1 – 0
Field	Tag	Block Offset	Byte Offset
Use	Compared with the tags stored in both ways	Selects 1 block in each way	Selects 1 of 4 bytes

Table 3: Chip interface (port names as in the course testbench)

Signal	Port	Dir	Width	Function
CPU Address (CA)	cpu_add	in	6	Address from the CPU, with start
CPU Data (CD)	cpu_data	inout	8	Write data in; read data out (high impedance otherwise)
Read_Write (RD/WR)	cpu_rd_wrn	in	1	1 = read, 0 = write
Start	start	in	1	Start of a CPU request
Busy	busy	out	1	Chip is processing a request
Reset	reset	in	1	Invalidates all blocks, resets the state machine
Clock	clk	in	1	Chip works on its negative edge
Memory Address (MA)	mem_add	out	6	Block address (last two bits 00) during enable
Memory Data (MD)	mem_data	in	8	Bytes from the memory on a read miss
Enable	mem_en	out	1	Memory read request, one clock cycle
Vdd, Gnd	Vdd, Gnd	in	1	Supply connections used for constant 1 and 0

3 Design overview

3.1 Hierarchical block diagram

The block diagram below shows the hierarchical breakup of this design (the diagram in the specification is generic and is only a reference). Colours: purple = chip, orange = cache, green = control, blue = building blocks, yellow = gates, grey = course primitives.

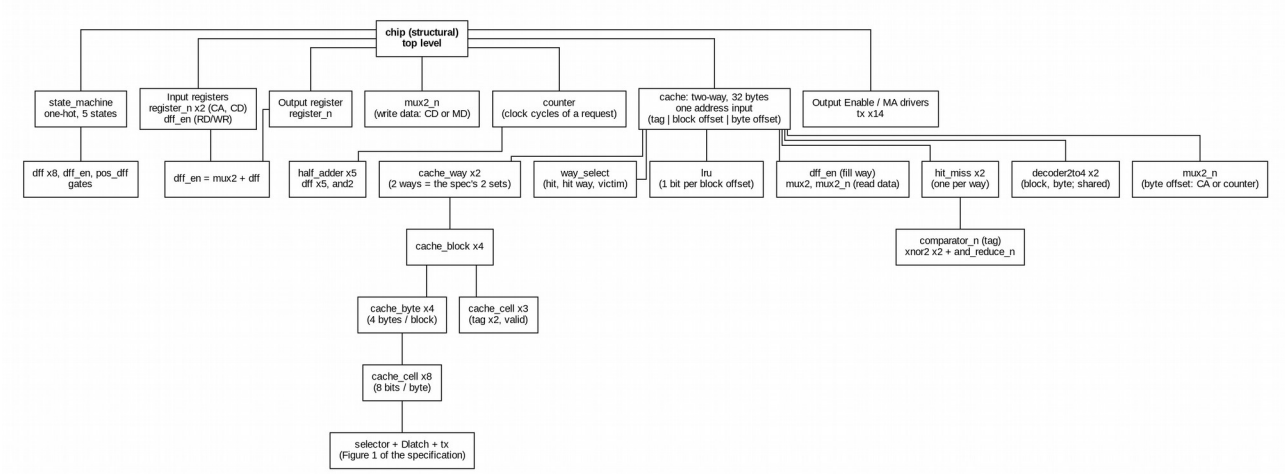


Figure 1: Hierarchical block diagram of the design

The chip's own block diagram, drawn from its VHDL netlist, shows how the blocks connect: the CPU registers feed the state machine and the cache; the cache and hit/miss logic tell the state machine whether the request hits; the state machine drives busy, enable, the output register and the cache write signal.

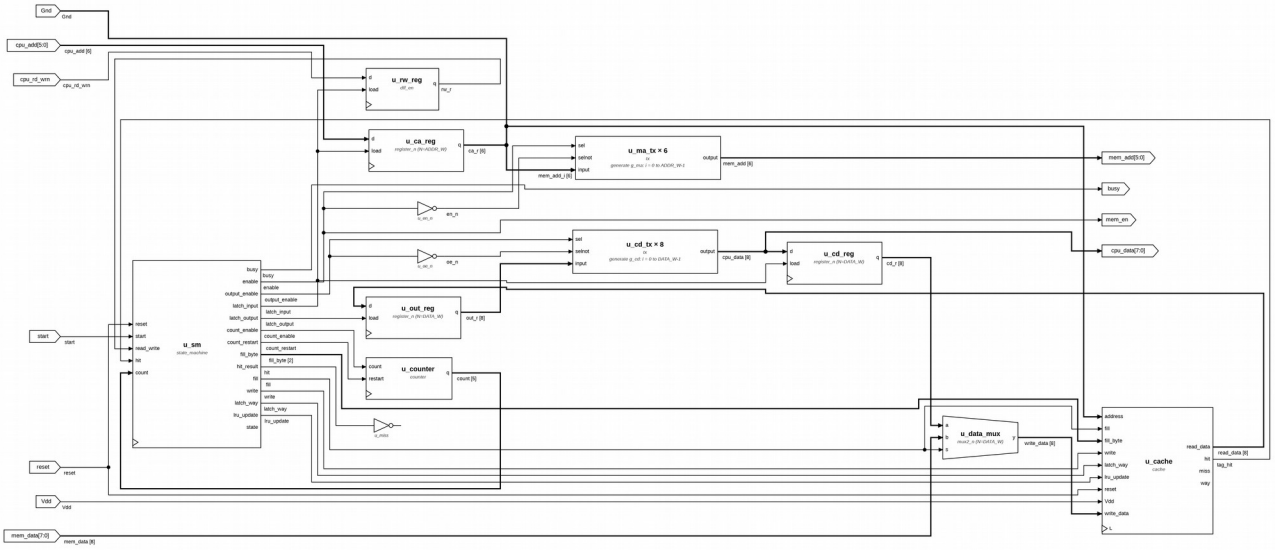


Figure 2: `chip`: top-level block diagram (all instances and nets)

The chip overview below shows every pin of chip.vhd grouped by interface (CPU on the left, memory on the right, clock and reset on top, supply at the bottom; buses with their width) and the data path between them, from left to right: the input registers, the address split into tag, block offset and byte offset, the cache with its decoders, storage and hit/miss logic, the output register and the tri-state drivers onto CD and MA. Control signals of the state machine are shown as green net tags, once at the state machine and again at every block they reach; sequential blocks carry the clock wedge.

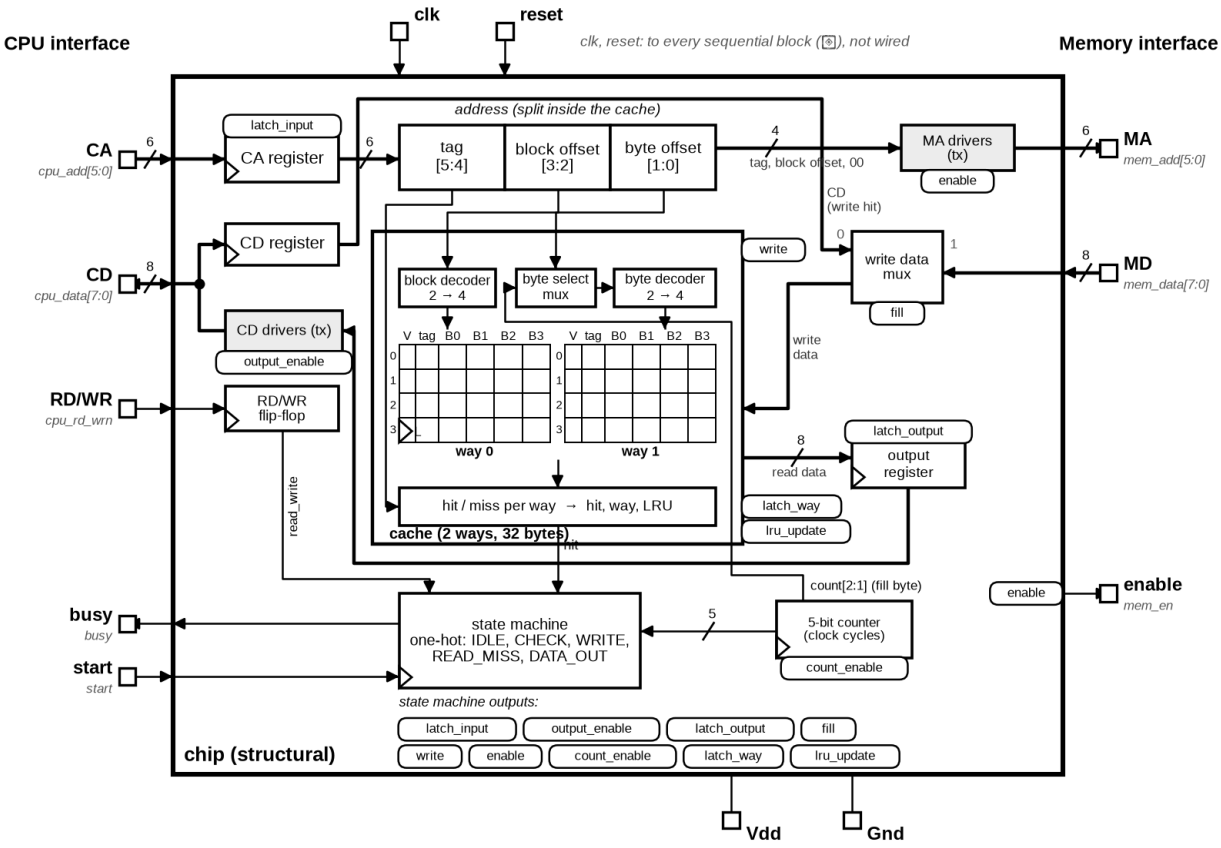


Figure 3: Chip overview: pins and data path

Each major module is shown in four ways: its inputs and outputs; its internal structure (every instance and net, drawn from the VHDL); a block diagram of its functional parts (the module as an outer box with its parts inside, inputs on the left, outputs on the right, arrows for the data flow); and an operation flowchart (how the module works: process steps and yes/no decisions from start to stop).

How to read the block diagrams in this report: grey arrow shapes are module ports; thick lines are buses, labelled with their width; a dot marks where a net branches. A triangle at the bottom-left of a block marks a sequential block, one that stores state (it contains negative edge flip-flops or, with an "L" next to the triangle, level sensitive latches: the cache cells). Blocks without the triangle are combinational. The clock is not drawn; it reaches every sequential block.

3.2 Design styles and why

- Structural VHDL (entity instantiation and generate loops) for every block above the primitives: it describes exactly the gates and cells that will be drawn in the schematic and layout in Submission II, so the schematics can be imported and LVS compared directly.
- Behavioural primitives only where the course provides them (`dff`, `Dlatch`, `tx`) and for the basic gates (`y <= a and b after GATE_DELAY`), so every gate has a propagation delay. `GATE_DELAY` = 100 ps, a conservative figure for the 45 nm (gpdk045) process; the longest path in one clock cycle is about 12 gates (1.2 ns) against a 10 ns clock.
- Negative edge registers everywhere: the CPU and memory signals are set up half a clock before they are sampled, which avoids setup time violations (as the specification intends).
- A one-hot state machine: one flip-flop per state, so every state decode is a single flip-flop output and the next-state logic is a few AND/OR gates per state; this is simple to lay out and has no decoding glitches on the outputs.
- Tri-state buses built from transmission gates: the cache cells share bit lines; CD is driven only while output enable is on; MA is driven only while enable is on (as in the reference output).
- Glitch-free cache writes: the cache address and data come from registers (or the memory) and are stable from half a cycle before the write signal until half a cycle after it, and the write signal itself comes straight from a flip-flop.

3.3 Entity / architecture pairs

The code and simulation results in Sections 4 to 7 are given in the same order as this list.

Table 4: Entity / architecture pairs

#	Entity	Architecture	File	Block	Testbench
1	dff	structural	two_way/ primitives/ dff.vhd	Course primitive: negative edge D flip-flop	— (primitive)
2	Dlatch	structural	two_way/ primitives/ Dlatch.vhd	Course primitive: positive level D latch	— (primitive)
3	tx	structural	two_way/ primitives/tx.vhd	Course primitive: transmission gate	— (primitive)
4	not_gate, and2_gate, or2_gate, nand2_gate, nor2_gate, xor2_gate, xnor2_gate	rtl	two_way/gates/ *.vhd	Basic gates (100 ps delay)	— (primitive)
5	and3_gate, or3_gate, or4_gate	structural	two_way/gates/ *.vhd	Basic gates (from 2-input gates)	— (primitive)
6	selector	structural	two_way/blocks/ selector.vhd	Cache cell: selector (Figure 2)	tb_selector
7	cache_cell	structural	two_way/blocks/ cache_cell.vhd	Cache cell (Figure 1)	tb_cache_cell
8	mux2, mux2_n	structural	two_way/blocks/	Mux (byte offset,	— (primitive)

			mux2*.vhd	write data, way, read data, register load)	
9	mux4_n	structural	two_way/blocks/ mux4_n.vhd	4:1 mux (LRU bit of the block offset)	— (primitive)
10	decoder2to4	structural	two_way/blocks/ decoder2to4.vhd	Decoder (block offset, byte offset)	— (primitive)
11	and_reduce_n, comparator_n	structural	two_way/blocks/ *.vhd	Hit/Miss: tag comparator	— (primitive)
12	dff_en	structural	two_way/blocks/ dff_en.vhd	Register bit with load	— (primitive)
13	register_n	structural	two_way/blocks/ register_n.vhd	Registers (CA, CD, output register)	tb_register_n
14	pos_dff	structural	two_way/blocks/ pos_dff.vhd	State machine: write signal flip-flop	— (primitive)
15	half_adder	structural	two_way/blocks/ half_adder.vhd	Counter bit	— (primitive)
16	counter	structural	two_way/blocks/ counter.vhd	Clock cycle counter of every request	tb_counter
17	cache_byte	structural	two_way/cache/ cache_byte.vhd	Cache: one byte (8 cache cells)	— (primitive)
18	cache_block	structural	two_way/cache/ cache_block.vhd	Cache block (valid, tag, 4 bytes)	tb_cache_block
19	cache_way	structural	two_way/cache/ cache_way.vhd	Cache way (4 blocks; 2 per cache)	tb_cache_way
20	hit_miss	structural	two_way/cache/ hit_miss.vhd	Cache: Hit/Miss of one way	tb_hit_miss
21	way_select	structural	two_way/cache/ way_select.vhd	Cache: hit, hit way, victim	tb_way_select
22	lru	structural	two_way/cache/ lru.vhd	Cache: LRU bits	tb_lru
23	cache	structural	two_way/cache/ cache.vhd	Two-way cache (address split, decoders, 2 ways, Hit/Miss, LRU)	tb_cache
24	state_machine	structural	two_way/control/ state_machine.vhd	State Machine	tb_state_machine
25	chip	structural	two_way/ chip.vhd	Top level (the chip)	tb_chip, chip_test

4 Building blocks

4.1 Course primitives: dff, Dlatch, tx

These three cells are the course's `useful_vhdl_files`, used unchanged: a negative edge triggered D flip-flop, a positive level sensitive D latch and a transmission gate (output high impedance unless `sel` = 1 and `selnot` = 0). Testbenches for low level primitive cells are not required.

Listing: two_way/primitives/dff.vhd

```
--
-- Entity: negative edge triggered D flip-flop (dff)
-- Architecture : structural
-- Author:
--

library STD;
library IEEE;
use IEEE.std_logic_1164.all;

entity dff is
  port ( d   : in  std_logic;
         clk : in  std_logic;
         q   : out std_logic;
         qbar: out std_logic);
end dff;

architecture structural of dff is


begin

  output: process

  begin
    wait until ( clk'EVENT and clk = '0' );
    q <= d;
    qbar <= not d ;
  end process output;


end structural;
```

Listing: two_way/primitives/Dlatch.vhd

```
-- Entity: positive level triggered D latch
-- Architecture : structural
-- Author:
--


library STD;
library IEEE;
use IEEE.std_logic_1164.all;

entity Dlatch is
  port ( d   : in  std_logic;
         clk : in  std_logic;
         q   : out std_logic;
         qbar: out std_logic);
end Dlatch;

architecture structural of Dlatch is



begin

  output: process (d,clk)
```

```
  begin
    if clk = '1' then
    q <= d;
    qbar <= not d ;
 end if;
 end process output;

end structural;
```

Listing: two_way/primitives/tx.vhd

```
-- Entity: tx -- transmission gate
-- Architecture : structural
-- Author:
--

library STD;
library IEEE;
use IEEE.std_logic_1164.all;

entity tx is
  port ( sel   : in std_logic;
         selnot: in std_logic;
         input : in std_logic;
         output:out std_logic);
end tx;

architecture structural of tx is

begin

  txprocess: process (sel, selnot, input)
  begin
    if (sel = '1' and selnot = '0') then
      output <= input;
    else
      output <= 'Z';
    end if;
  end process txprocess;

end structural;
```

4.2 Shared package (cache_pkg)

cache_pkg holds the design's constants (address, data, tag and counter widths) and GATE_DELAY, the 100 ps propagation delay of the basic gates. The basic gates (not, and2, or2, nand2, nor2, xor2, xnor2, and3, or3, or4) are one assignment each, or built from 2-input gates; as the lowest level of the design, their code, circuits and waveforms are not repeated here.

Listing: two_way/cache_pkg.vhd

```
-- Shared constants for the CMPE 413/640 cache chip - extra credit version
--   Two-way set associative with LRU replacement: 2 ways (the specification's "2 sets") of
--   4 blocks x 4 bytes = 32 bytes, byte addressable, write through / no write allocate
--   CPU address (CA) = | tag (2) | block offset (2) | byte offset (2) |
--                        5..4       3..2               1..0
--   The block offset selects one block in each way; the tag is compared with both.
library ieee;
use ieee.std_logic_1164.all;

package cache_pkg is
    -- propagation delay of every basic gate: 100 ps, a conservative figure for the 45 nm
    -- (gpdk045) process. The longest path in one clock cycle (CPU address register -> block
    -- decoder -> cache cells -> tag compare -> hit / victim -> next state) is about 15 gates,
    -- so it settles in ~1.5 ns, well inside the 10 ns clock period of the course testbench.
    constant GATE_DELAY : time := 100 ps;

    constant ADDR_W     : natural := 6;       -- CPU / memory address width
    constant DATA_W     : natural := 8;       -- one byte per location
    constant TAG_W      : natural := 2;
    constant NUM_WAYS   : natural := 2;
```

```
    constant NUM_BLOCKS : natural := 4;       -- blocks per way
    constant NUM_BYTES  : natural := 4;       -- bytes per block
    constant COUNT_W    : natural := 5;       -- read-miss cycle counter
end package cache_pkg;
```

4.3 Selector (Figure 2)

The selector of the cache cell implements the truth table of Figure 2 of the specification: Read enable = Chip enable · RD/WR, Write enable = Chip enable · not RD/WR.



Figure 4: `selector`: inputs and outputs

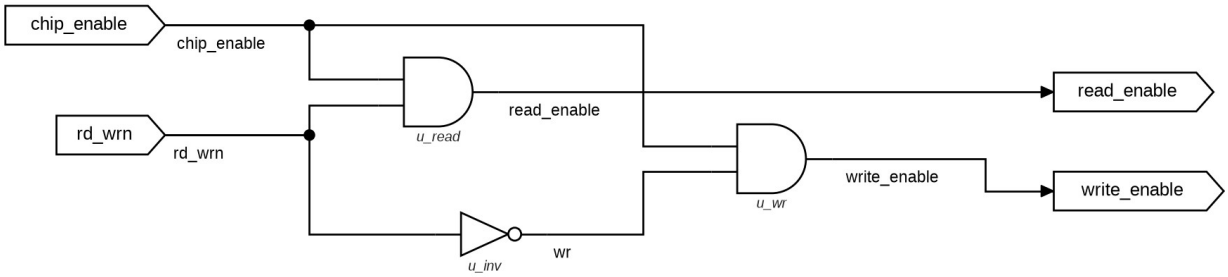


Figure 5: `selector`: internal structure

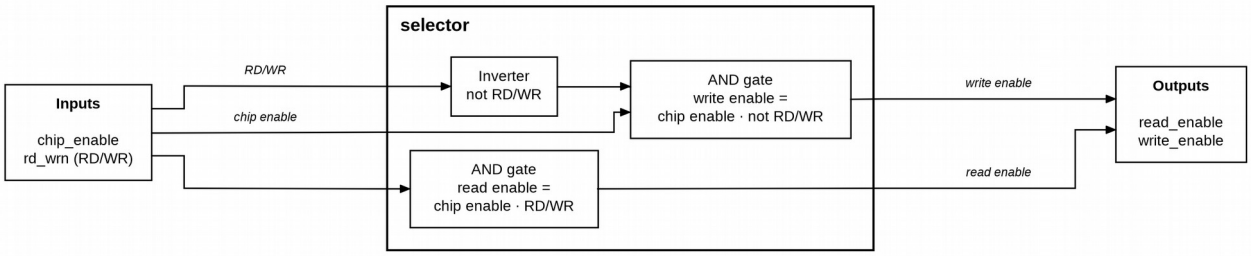


Figure 6: `selector`: block diagram

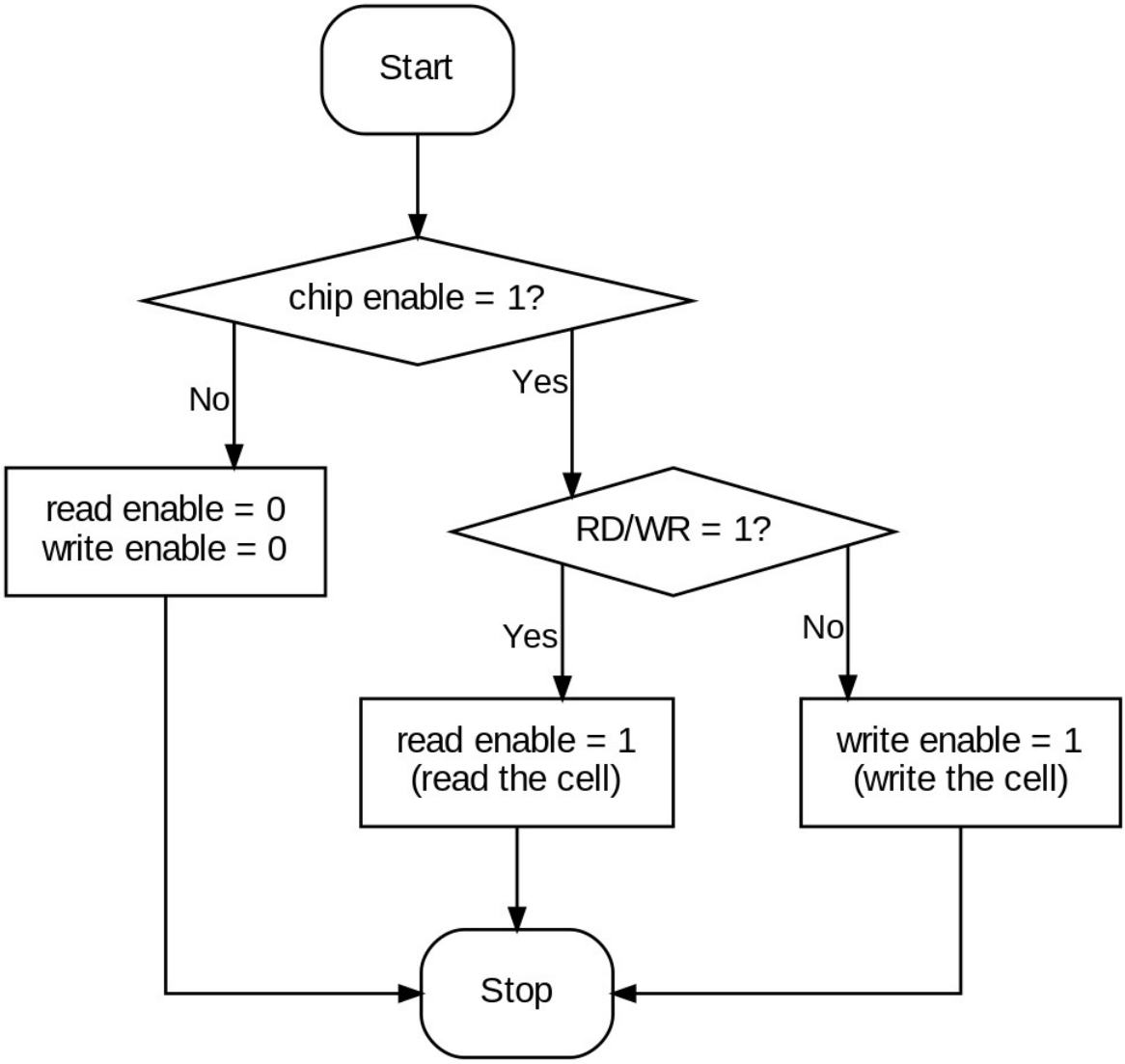


Figure 7: `selector`: operation flowchart

Code

Listing: two_way/blocks/selector.vhd

```
-- Selector (decoder) of the cache cell, Figure 2 of the project specification
--   Chip enable | RD/WR | Read enable | Write enable
--         0      |   x   |      0      |      0
--         1      |   1   |      1      |      0
--         1      |   0   |      0      |      1
library ieee;
use ieee.std_logic_1164.all;

entity selector is
    port (
        chip_enable  : in  std_logic;
        rd_wrn       : in  std_logic;   -- 1 = read, 0 = write
        read_enable  : out std_logic;
        write_enable : out std_logic
    );
end entity selector;
```

```
architecture structural of selector is
    signal wr : std_logic;
begin
    u_inv  : entity work.not_gate  port map (a => rd_wrn, y => wr);
    u_read : entity work.and2_gate port map (a => chip_enable, b => rd_wrn, y => read_enable);
    u_wr   : entity work.and2_gate port map (a => chip_enable, b => wr,     y => write_enable);
end architecture structural;
```

Testbench

- All rows of the truth table of Figure 2 (Chip enable 0 with RD/WR 0 and 1; read; write).

Listing: two_way/testbench/blocks/tb_selector.vhd

```
-- Testbench: selector, the truth table of Figure 2 of the specification
library ieee;
use ieee.std_logic_1164.all;
use work.tb_pkg.all;

entity tb_selector is
end entity tb_selector;

architecture sim of tb_selector is
    signal chip_enable, rd_wrn, read_enable, write_enable : std_logic := '0';
begin
    dut : entity work.selector
        port map (chip_enable => chip_enable, rd_wrn => rd_wrn,
                  read_enable => read_enable, write_enable => write_enable);

    stimulus : process
        type row_t is record
            ce, rw, re, we : std_logic;
        end record;
        type rows_t is array (natural range <>) of row_t;
        constant TABLE : rows_t := (
            ('0', '0', '0', '0'), ('0', '1', '0', '0'),   -- chip enable 0: neither (RD/WR = x)
            ('1', '1', '1', '0'),                         -- read
            ('1', '0', '0', '1'));                        -- write
        variable errors : natural := 0;
    begin
        for i in TABLE'range loop
            chip_enable <= TABLE(i).ce; rd_wrn <= TABLE(i).rw;
            wait for 10 ns;
            if read_enable /= TABLE(i).re or write_enable /= TABLE(i).we then
                report "FAIL: chip enable=" & str(TABLE(i).ce) & " RD/WR=" & str(TABLE(i).rw)
                     & " -> read enable=" & str(read_enable) & " write enable=" & str(write_enable)
                     severity error;
                errors := errors + 1;
            end if;
        end loop;
        if errors = 0 then
            report "tb_selector: ALL TESTS PASSED";
        else
            report "tb_selector: " & str(errors) & " TEST(S) FAILED" severity error;
        end if;
        wait;
    end process stimulus;
end architecture sim;
```

Simulation results

Simulation result: PASS — `tb_selector`: all checks passed, simulation ended at 40 ns.

selector: the truth table of Figure 2 of the specification

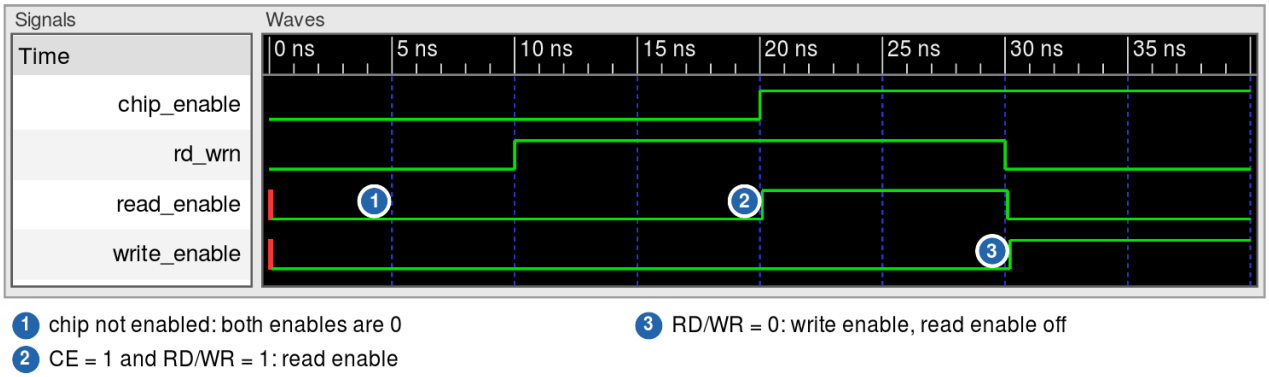


Figure 8: `tb_selector`: simulation waveform

Read enable is 1 only for Chip enable = 1, RD/WR = 1; Write enable only for Chip enable = 1, RD/WR = 0.

4.4 Cache cell (Figure 1)

One bit of the cache, exactly as Figure 1 of the specification: Write Data goes into a positive level sensitive latch whose clock is the selector's Write enable; the latch output goes through a transmission gate, opened by Read enable, to Read Data. When the cell is not read, Read Data is high impedance, so many cells can drive one bit line.

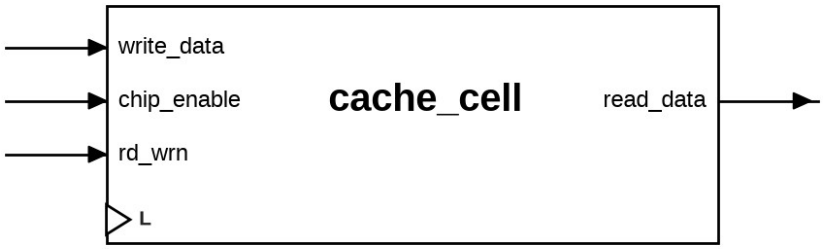


Figure 9: `cache_cell`: inputs and outputs

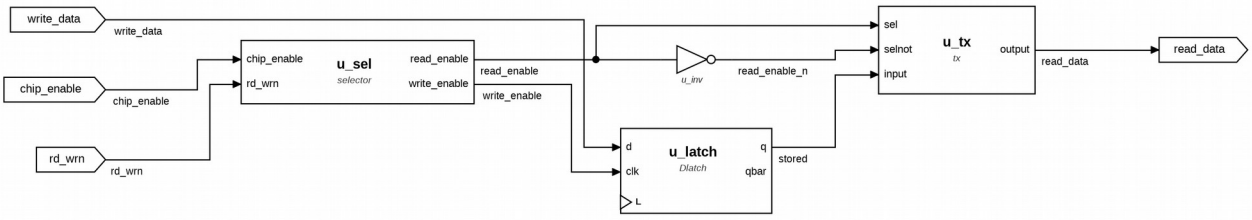


Figure 10: `cache_cell`: internal structure

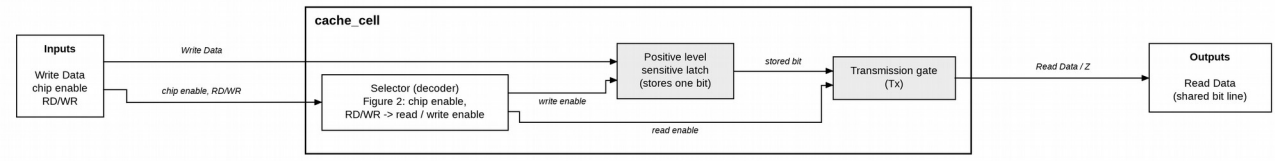


Figure 11: `cache_cell`: block diagram

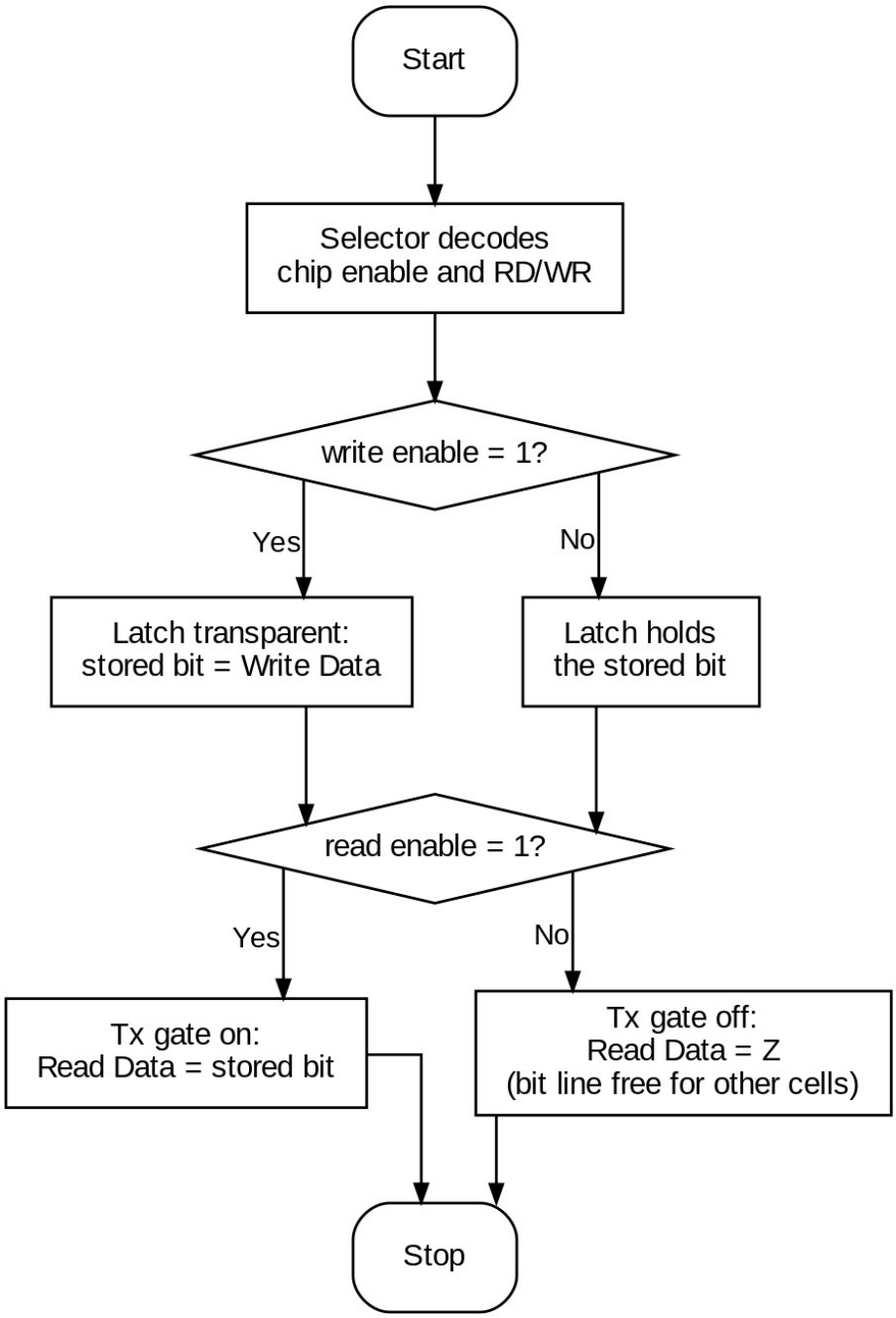


Figure 12: `cache_cell`: operation flowchart

Code

Listing: two_way/blocks/cache_cell.vhd

```
-- Cache cell (one bit), Figure 1 of the project specification
--   Write Data -> positive level sensitive latch (Dlatch) -> Tx gate -> Read Data
--   The selector turns Chip enable and RD/WR into the latch's Write enable and the
--   Tx gate's Read enable. Read Data is high impedance unless the cell is being read,
--   so the Read Data outputs of many cells can share one bit line.
library ieee;
use ieee.std_logic_1164.all;

entity cache_cell is
    port (
        write_data  : in  std_logic;
        chip_enable : in  std_logic;
        rd_wrn      : in  std_logic;   -- 1 = read, 0 = write
        read_data   : out std_logic    -- 'Z' when not read
    );
end entity cache_cell;

architecture structural of cache_cell is
    signal read_enable, read_enable_n, write_enable : std_logic;
    signal stored, stored_n                         : std_logic;
begin
    u_sel   : entity work.selector
        port map (chip_enable => chip_enable, rd_wrn => rd_wrn,
                  read_enable => read_enable, write_enable => write_enable);
    u_latch : entity work.Dlatch port map (d => write_data, clk => write_enable, q => stored, qbar =>
stored_n);
    u_inv   : entity work.not_gate port map (a => read_enable, y => read_enable_n);
    u_tx    : entity work.tx
        port map (sel => read_enable, selnot => read_enable_n, input => stored, output => read_data);
end architecture structural;
```

Testbench

- Write 1, read it back; write 0, read it back.
- With Chip enable = 0: Read Data is high impedance and a write request does not change the cell.
- As in the chip, RD/WR and Write Data change only while Chip enable is off, so the selector never produces a write glitch.

Listing: two_way/testbench/blocks/tb_cache_cell.vhd

```
-- Testbench: cache_cell (with its selector), Figures 1 and 2 of the specification
--   write 1 and 0, read back, hold while Chip enable = 0, Read Data high impedance unless read
library ieee;
use ieee.std_logic_1164.all;
use work.tb_pkg.all;

entity tb_cache_cell is
end entity tb_cache_cell;

architecture sim of tb_cache_cell is
    signal write_data, chip_enable, rd_wrn : std_logic := '0';
    signal read_data                       : std_logic;
begin
    dut : entity work.cache_cell
        port map (write_data => write_data, chip_enable => chip_enable, rd_wrn => rd_wrn, read_data =>
read_data);

    stimulus : process
        type step_t is record
            ce, rw, wd, rd : std_logic;   -- Chip enable, RD/WR, Write Data, expected Read Data
        end record;
        type steps_t is array (natural range <>) of step_t;
        constant STEPS : steps_t := (
            ('1', '0', '1', 'Z'),   -- write 1 (Read Data off while writing)
            ('1', '1', '0', '1'),   -- read: 1
            ('0', '1', '0', 'Z'),   -- chip enable off: Read Data high impedance
            ('0', '0', '0', 'Z'),   -- RD/WR = write but chip enable off: no write
            ('1', '1', '0', '1'),   -- still 1
```

```
            ('1', '0', '0', 'Z'),   -- write 0
            ('1', '1', '1', '0'),   -- read: 0
            ('0', '0', '1', 'Z'),   -- no write while chip enable is off
            ('1', '1', '1', '0')    -- still 0
        );
        variable errors : natural := 0;
    begin
        for i in STEPS'range loop
            -- as in the chip: RD/WR and Write Data change only while Chip enable is off, so the
            -- selector never makes a write glitch (the specification's write timing rule)
            chip_enable <= '0';
            wait for 1 ns;
            rd_wrn <= STEPS(i).rw; write_data <= STEPS(i).wd;
            wait for 1 ns;
            chip_enable <= STEPS(i).ce;
            wait for 8 ns;
            if read_data /= STEPS(i).rd then
                report "FAIL step " & str(i) & ": Read Data = " & str(read_data)
                     & ", expected " & str(STEPS(i).rd) severity error;
                errors := errors + 1;
            end if;
        end loop;
        if errors = 0 then
            report "tb_cache_cell: ALL TESTS PASSED";
        else
            report "tb_cache_cell: " & str(errors) & " TEST(S) FAILED" severity error;
        end if;
        wait;
    end process stimulus;
end architecture sim;
```

Simulation results

Simulation result: PASS — `tb_cache_cell`: all checks passed, simulation ended at 90 ns.

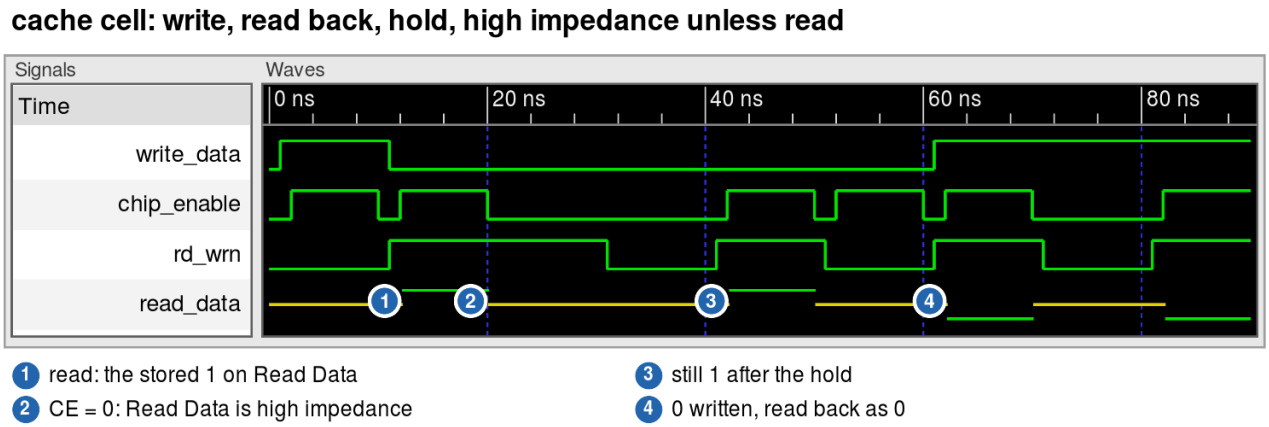


Figure 13: `tb_cache_cell`: simulation waveform

Read Data shows the stored bit only while the cell is read (Chip enable = 1, RD/WR = 1); the mid-level line is high impedance.

4.5 Decoder, multiplexers and comparator

Low-level combinational blocks built from the basic gates: the 2-to-4 decoder (block offset and byte offset), the 1-bit and N-bit 2:1 multiplexers (cache write address and data, register load), and the N-bit equality comparator used for the tag compare.

Listing: two_way/blocks/decoder2to4.vhd

```
-- 2-to-4 decoder with enable: y(i) = en and (a = i)
library ieee;
```

```
use ieee.std_logic_1164.all;

entity decoder2to4 is
    port (
        a  : in  std_logic_vector(1 downto 0);
        en : in  std_logic;
        y  : out std_logic_vector(3 downto 0)   -- one-hot
    );
end entity decoder2to4;

architecture structural of decoder2to4 is
    signal a_n : std_logic_vector(1 downto 0);
begin
    u_inv0 : entity work.not_gate port map (a => a(0), y => a_n(0));
    u_inv1 : entity work.not_gate port map (a => a(1), y => a_n(1));

    u_y0 : entity work.and3_gate port map (a => en, b => a_n(1), c => a_n(0), y => y(0));
    u_y1 : entity work.and3_gate port map (a => en, b => a_n(1), c => a(0),   y => y(1));
    u_y2 : entity work.and3_gate port map (a => en, b => a(1),   c => a_n(0), y => y(2));
    u_y3 : entity work.and3_gate port map (a => en, b => a(1),   c => a(0),   y => y(3));
end architecture structural;
```

Listing: two_way/blocks/mux2.vhd

```
-- 2:1 multiplexer (1 bit) from basic gates: y = (a and not s) or (b and s)
library ieee;
use ieee.std_logic_1164.all;

entity mux2 is
    port (
        a, b : in  std_logic;   -- a selected when s = 0, b when s = 1
        s    : in  std_logic;
        y    : out std_logic
    );
end entity mux2;

architecture structural of mux2 is
    signal s_n, a_sel, b_sel : std_logic;
begin
    u_inv : entity work.not_gate  port map (a => s, y => s_n);
    u_a   : entity work.and2_gate port map (a => a, b => s_n, y => a_sel);
    u_b   : entity work.and2_gate port map (a => b, b => s,   y => b_sel);
    u_or  : entity work.or2_gate  port map (a => a_sel, b => b_sel, y => y);
end architecture structural;
```

Listing: two_way/blocks/mux2_n.vhd

```
-- N-bit 2:1 multiplexer: one mux2 per bit
library ieee;
use ieee.std_logic_1164.all;

entity mux2_n is
    generic (N : positive := 8);
    port (
        a, b : in  std_logic_vector(N-1 downto 0);   -- a when s = 0, b when s = 1
        s    : in  std_logic;
        y    : out std_logic_vector(N-1 downto 0)
    );
end entity mux2_n;

architecture structural of mux2_n is
begin
    g_bit : for i in 0 to N-1 generate
        u_mux : entity work.mux2 port map (a => a(i), b => b(i), s => s, y => y(i));
    end generate g_bit;
end architecture structural;
```

Listing: two_way/blocks/mux4_n.vhd

```
-- N-bit 4:1 multiplexer: tree of three mux2_n
library ieee;
use ieee.std_logic_1164.all;

entity mux4_n is
    generic (N : positive := 8);
    port (
        d0, d1, d2, d3 : in  std_logic_vector(N-1 downto 0);
```

```
        s             : in  std_logic_vector(1 downto 0);
        y             : out std_logic_vector(N-1 downto 0)
    );
end entity mux4_n;

architecture structural of mux4_n is
    signal m01, m23 : std_logic_vector(N-1 downto 0);
begin
    u_01 : entity work.mux2_n generic map (N => N) port map (a => d0,  b => d1,  s => s(0), y => m01);
    u_23 : entity work.mux2_n generic map (N => N) port map (a => d2,  b => d3,  s => s(0), y => m23);
    u_y  : entity work.mux2_n generic map (N => N) port map (a => m01, b => m23, s => s(1), y => y);
end architecture structural;
```

Listing: two_way/blocks/and_reduce_n.vhd

```
-- N-input AND (reduction): chain of 2-input AND gates, y = a(0) and a(1) and ... a(N-1)
library ieee;
use ieee.std_logic_1164.all;

entity and_reduce_n is
    generic (N : positive := 4);
    port (
        a : in  std_logic_vector(N-1 downto 0);
        y : out std_logic
    );
end entity and_reduce_n;

architecture structural of and_reduce_n is
    signal chain : std_logic_vector(N-1 downto 0);
begin
    chain(0) <= a(0);
    g_and : for i in 1 to N-1 generate
        u_and : entity work.and2_gate port map (a => chain(i-1), b => a(i), y => chain(i));
    end generate g_and;
    y <= chain(N-1);
end architecture structural;
```

Listing: two_way/blocks/comparator_n.vhd

```
-- N-bit equality comparator (tag compare): one XNOR per bit + AND reduction
library ieee;
use ieee.std_logic_1164.all;

entity comparator_n is
    generic (N : positive := 5);
    port (
        a, b : in  std_logic_vector(N-1 downto 0);
        eq   : out std_logic                       -- 1 when a = b
    );
end entity comparator_n;

architecture structural of comparator_n is
    signal same : std_logic_vector(N-1 downto 0);
begin
    g_bit : for i in 0 to N-1 generate
        u_xnor : entity work.xnor2_gate port map (a => a(i), b => b(i), y => same(i));
    end generate g_bit;

    u_and : entity work.and_reduce_n generic map (N => N) port map (a => same, y => eq);
end architecture structural;
```

Simulation results

Simulation result: PASS — `tb_decoder2to4`: all checks passed, simulation ended at 80 ns.

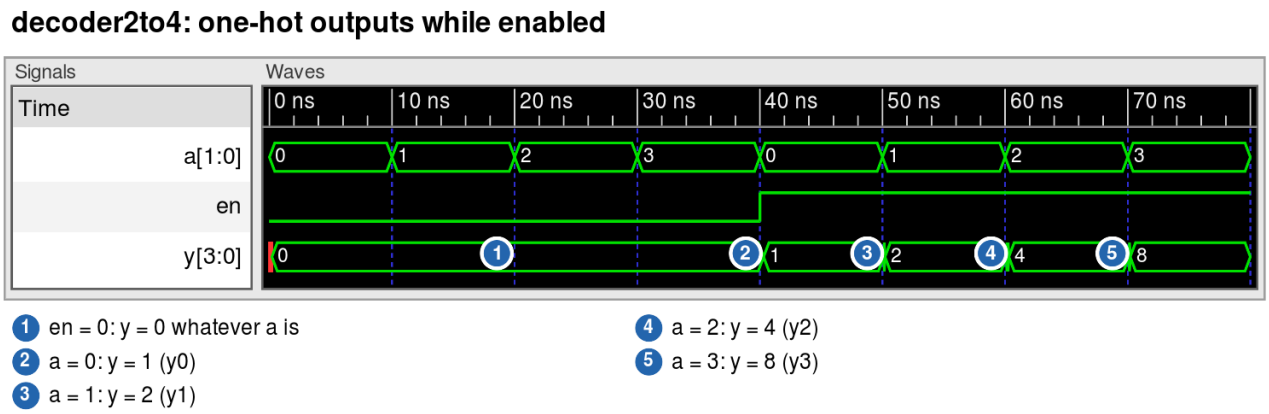


Figure 14: `tb_decoder2to4`: simulation waveform

With en = 0 every output is 0; with en = 1 only the output selected by a is 1 (one-hot 1, 2, 4, 8).

Simulation result: **PASS** — `tb_mux2`: all checks passed, simulation ended at 80 ns.

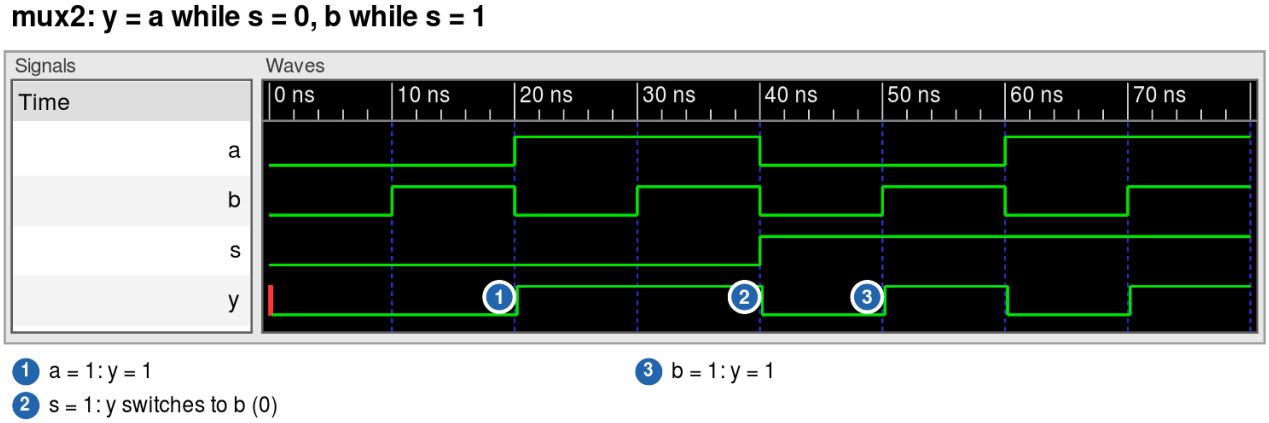


Figure 15: `tb_mux2`: simulation waveform

y follows a while s = 0 and b while s = 1, for all eight input combinations.

Simulation result: **PASS** — `tb_mux2_n`: all checks passed, simulation ended at 100 ns.

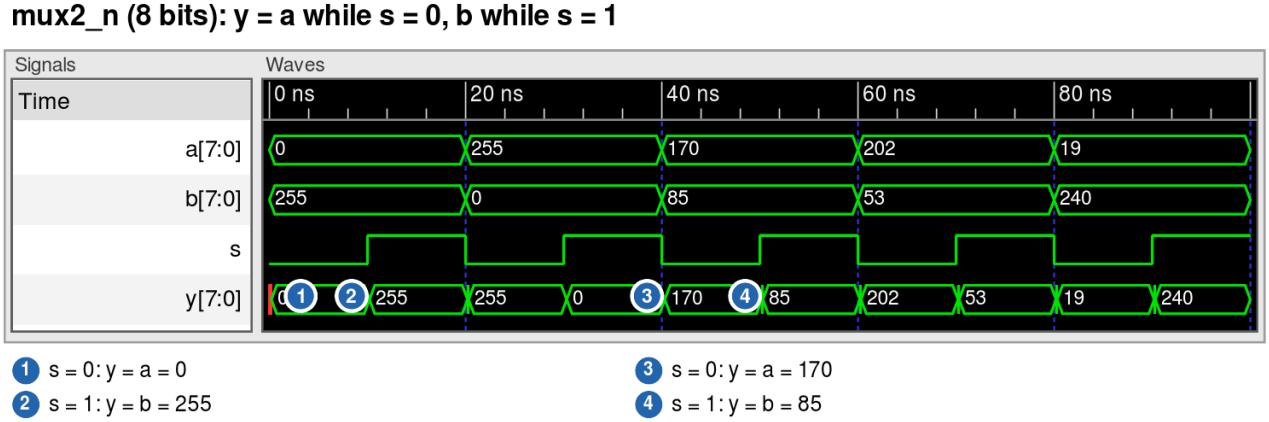


Figure 16: `tb_mux2_n`: simulation waveform

Every bit of y follows a while s = 0 and b while s = 1.

Simulation result: **PASS** — `tb_mux4_n`: all checks passed, simulation ended at 120 ns.

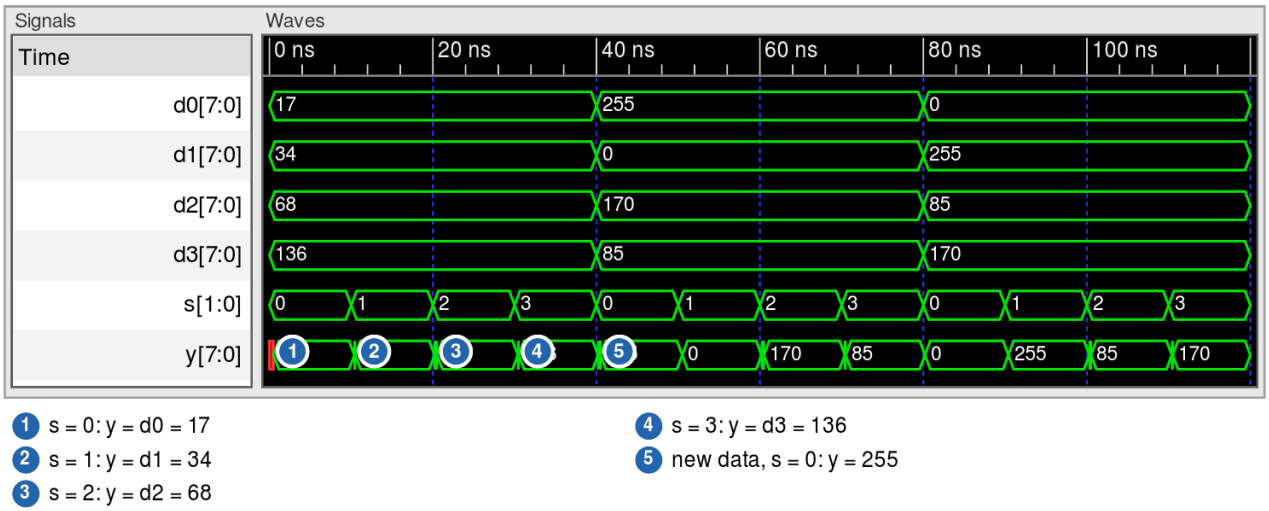


Figure 17: `tb_mux4_n`: simulation waveform

y follows d0, d1, d2 or d3 as the select s is 00, 01, 10, 11.

Simulation result: **PASS** — `tb_and_reduce_n`: all checks passed, simulation ended at 160 ns.

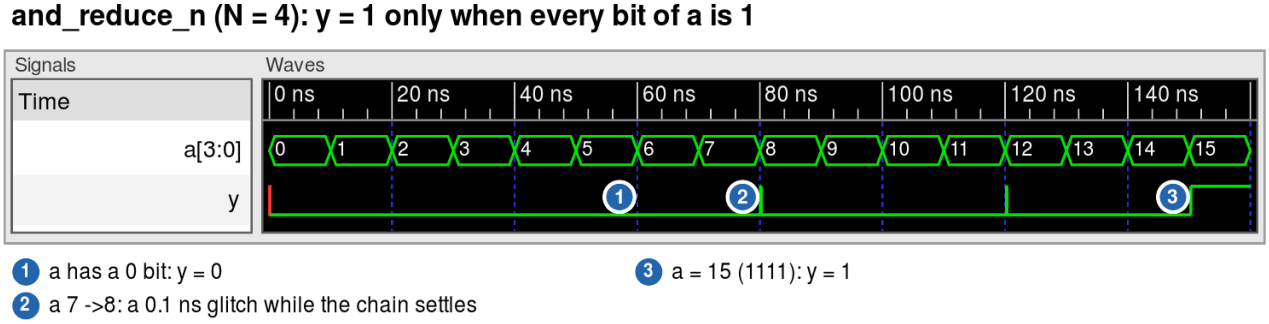


Figure 18: `tb_and_reduce_n`: simulation waveform

y is 1 only while every bit of a is 1.

Simulation result: **PASS** — `tb_comparator_n`: all checks passed, simulation ended at 10240 ns.

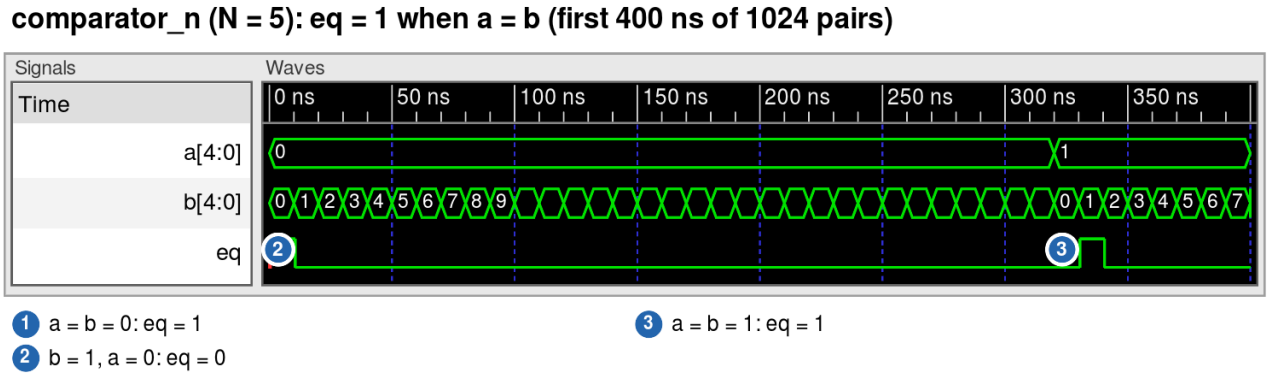


Figure 19: `tb_comparator_n`: simulation waveform

eq is 1 exactly when a = b; the first 400 ns of the test of all 1024 pairs.

4.6 Registers

`dff_en` is a register bit with load: a `mux2` feeds back the flip-flop output when load = 0. `register_n` is N of them; the chip uses one for the CPU address (6 bits), one for the CPU data (8 bits) and one for the output register (8 bits), plus a `dff_en` for RD/WR.

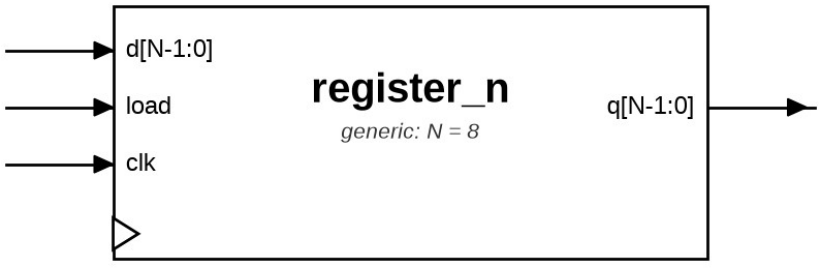


Figure 20: `register_n`: inputs and outputs

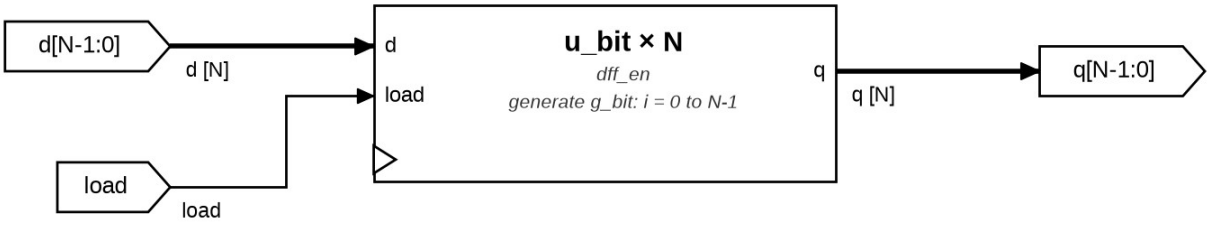


Figure 21: `register_n`: internal structure

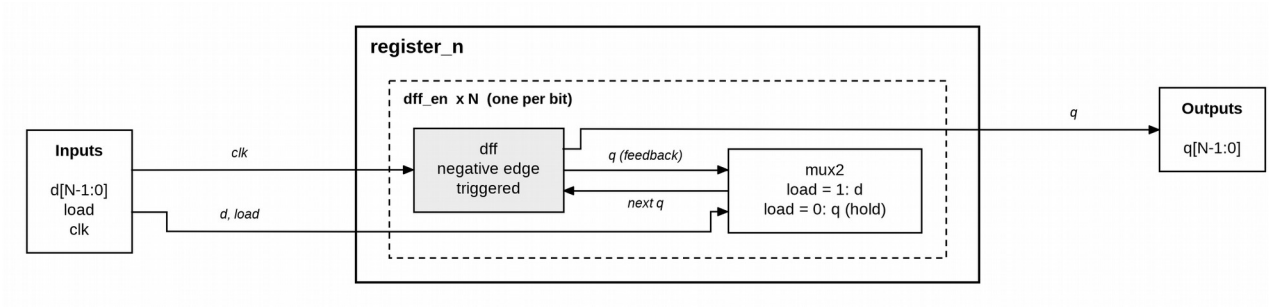


Figure 22: `register_n`: block diagram

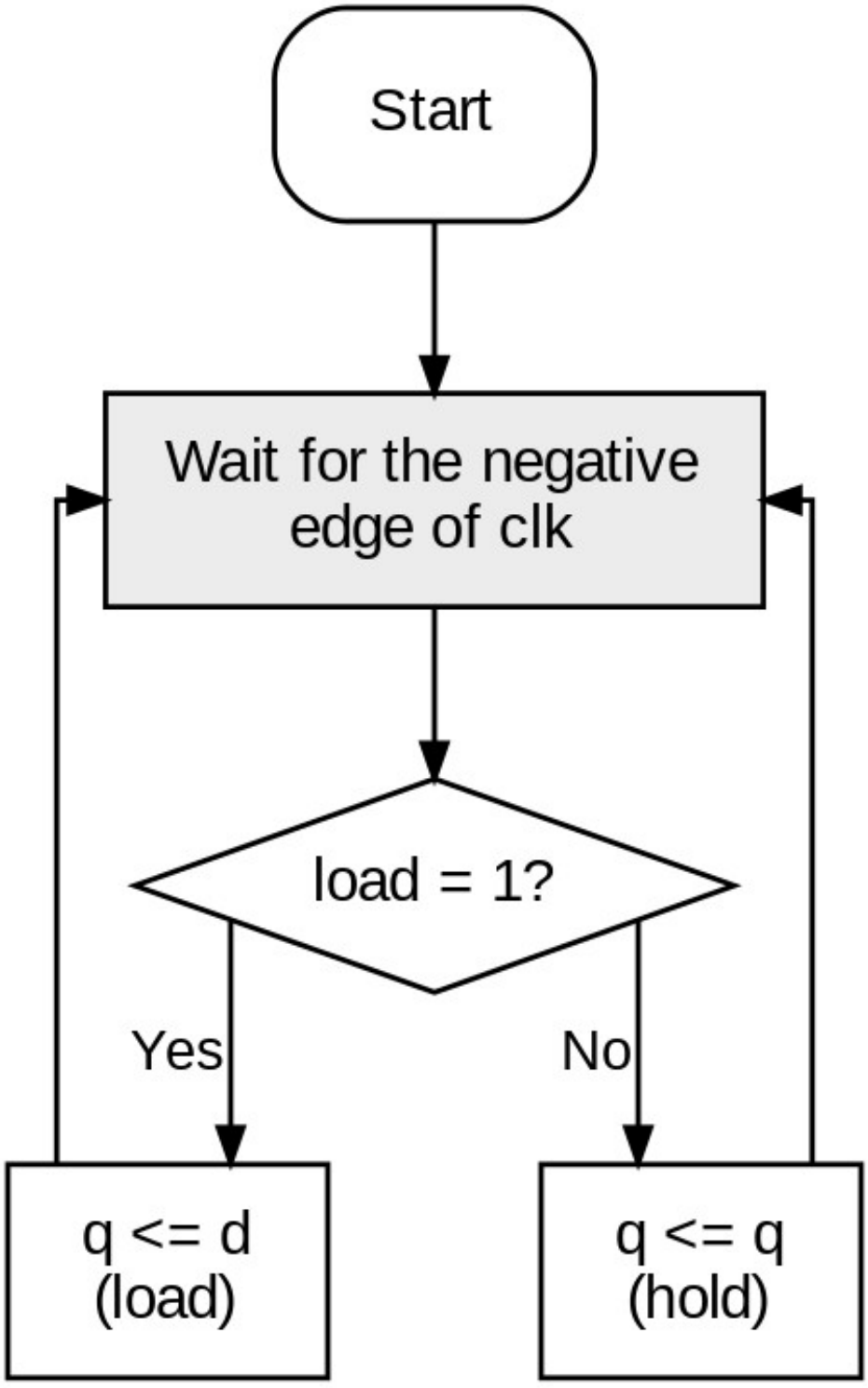


Figure 23: `register_n`: operation flowchart

Code

Listing: two_way/blocks/dff_en.vhd

```
-- Register bit with load: negative edge D flip-flop (dff) with a mux2 holding q when load = 0
--   next q = d when load = 1, else q (the chip works on the negative edge of clk)
library ieee;
use ieee.std_logic_1164.all;

entity dff_en is
    port (
        d    : in  std_logic;
        load : in  std_logic;
        clk  : in  std_logic;
        q    : out std_logic
    );
end entity dff_en;

architecture structural of dff_en is
    signal q_i, q_n, d_sel : std_logic;
begin
    u_mux : entity work.mux2 port map (a => q_i, b => d, s => load, y => d_sel);
    u_ff  : entity work.dff  port map (d => d_sel, clk => clk, q => q_i, qbar => q_n);
    q <= q_i;
end architecture structural;
```

Listing: two_way/blocks/register_n.vhd

```
-- N-bit register with load (negative edge): one dff_en per bit
--   Used for the CPU address / data / read_write input registers and the output register.
library ieee;
use ieee.std_logic_1164.all;

entity register_n is
    generic (N : positive := 8);
    port (
        d    : in  std_logic_vector(N-1 downto 0);
        load : in  std_logic;
        clk  : in  std_logic;
        q    : out std_logic_vector(N-1 downto 0)
    );
end entity register_n;

architecture structural of register_n is
begin
    g_bit : for i in 0 to N-1 generate
        u_bit : entity work.dff_en port map (d => d(i), load => load, clk => clk, q => q(i));
    end generate g_bit;
end architecture structural;
```

Testbench

- N = 8: load, hold twice with new data at the input, load again; inputs change on the positive edge and q is checked after the negative edge.

Listing: two_way/testbench/blocks/tb_register_n.vhd

```
-- Testbench: register_n (N = 8), loads on the negative edge only when load = 1
library ieee;
use ieee.std_logic_1164.all;
use work.tb_pkg.all;

entity tb_register_n is
end entity tb_register_n;

architecture sim of tb_register_n is
    signal clk, load : std_logic := '1';
    signal d, q      : std_logic_vector(7 downto 0) := (others => '0');
    signal done      : boolean := false;
begin
    dut : entity work.register_n generic map (N => 8) port map (d => d, load => load, clk => clk, q => q);

    clk_gen : process   -- same clock as chip_test: high at 10k ns, low at 10k + 5 ns
    begin
        while not done loop
            clk <= '1'; wait for 5 ns;
```

```
            clk <= '0'; wait for 5 ns;
        end loop;
        wait;
    end process clk_gen;

    stimulus : process
        type step_t is record
            load : std_logic;
            d, q : std_logic_vector(7 downto 0);   -- input, expected q after the negative edge
        end record;
        type steps_t is array (natural range <>) of step_t;
        constant STEPS : steps_t := (
            ('1', "10100101", "10100101"),   -- load
            ('0', "11110000", "10100101"),   -- hold
            ('0', "00001111", "10100101"),   -- hold
            ('1', "00111100", "00111100"),   -- load
            ('1', "11111111", "11111111"));
        variable errors : natural := 0;
    begin
        for i in STEPS'range loop
            wait until rising_edge(clk);          -- inputs change on the positive edge
            load <= STEPS(i).load; d <= STEPS(i).d;
            wait until falling_edge(clk);         -- register loads on the negative edge
            wait for 2 ns;
            if q /= STEPS(i).q then
                report "FAIL step " & str(i) & ": q=" & str(q) & " expected " & str(STEPS(i).q) severity
error;
                errors := errors + 1;
            end if;
        end loop;
        if errors = 0 then
            report "tb_register_n: ALL TESTS PASSED";
        else
            report "tb_register_n: " & str(errors) & " TEST(S) FAILED" severity error;
        end if;
        done <= true;
        wait;
    end process stimulus;
end architecture sim;
```

Simulation results

Simulation result: **PASS** — `tb_register_n`: all checks passed, simulation ended at 60 ns.

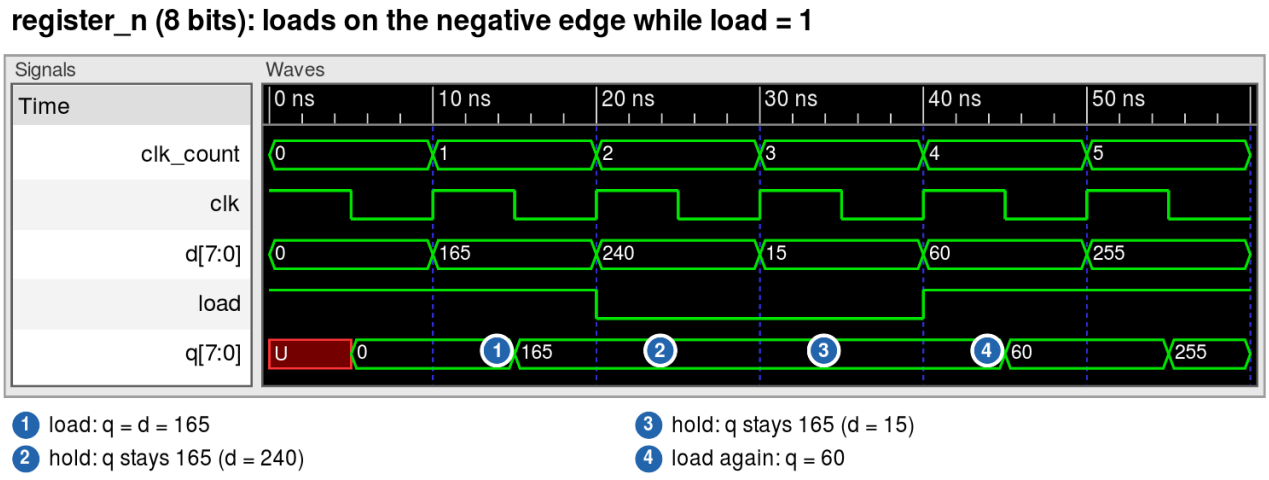


Figure 24: `tb_register_n`: simulation waveform

q changes only on negative edges where load = 1.

Simulation result: **PASS** — `tb_dff_en`: all checks passed, simulation ended at 70 ns.

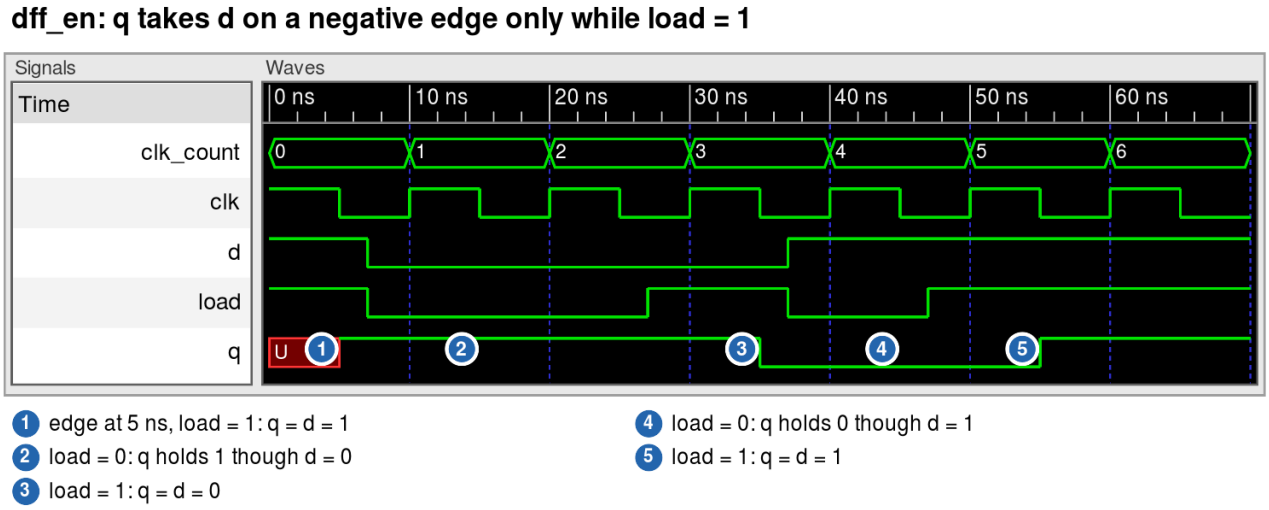


Figure 25: `tb_dff_en`: simulation waveform

q takes d on a negative edge only while load = 1, and holds its value otherwise.

4.7 Write-signal flip-flop and clock cycle counter

`pos_dff` is the course dff clocked by the inverted clock, i.e. a positive edge flip-flop; it makes the cache write signal. `counter` is a 5-bit synchronous counter (half adders and negative edge flip-flops): `restart = 1` loads 1, `count = 1` counts up, and `count = 0` with `restart = 0` clears it. The state machine counts the clock cycles of every request with it: the counter starts at 1 on the negative edge that accepts a request and, after the edge where `busy` goes low, holds 2 (read hit), 3 (write hit or miss) or 19 (read miss).

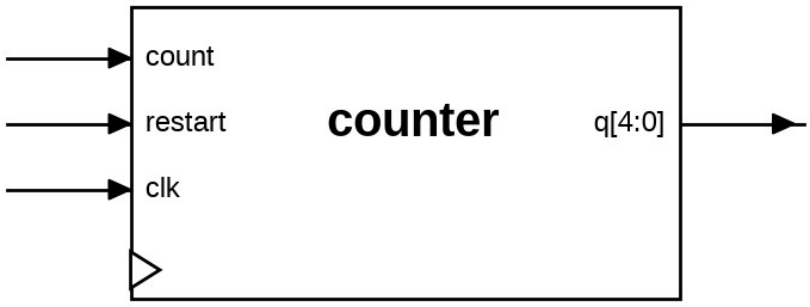


Figure 26: `counter`: inputs and outputs

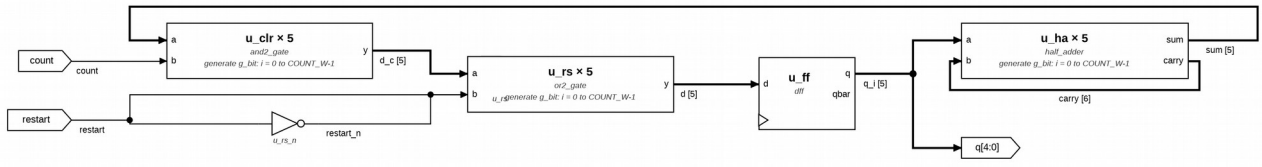


Figure 27: `counter`: internal structure

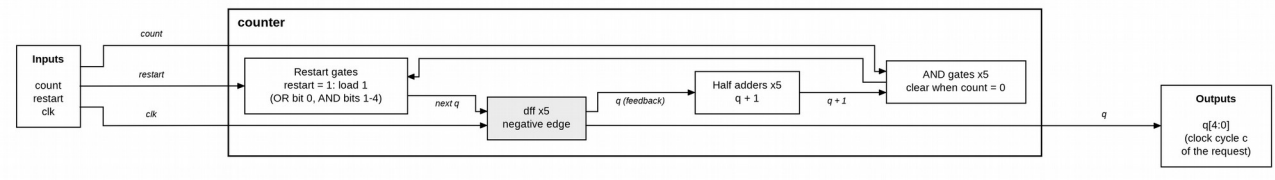


Figure 28: `counter`: block diagram

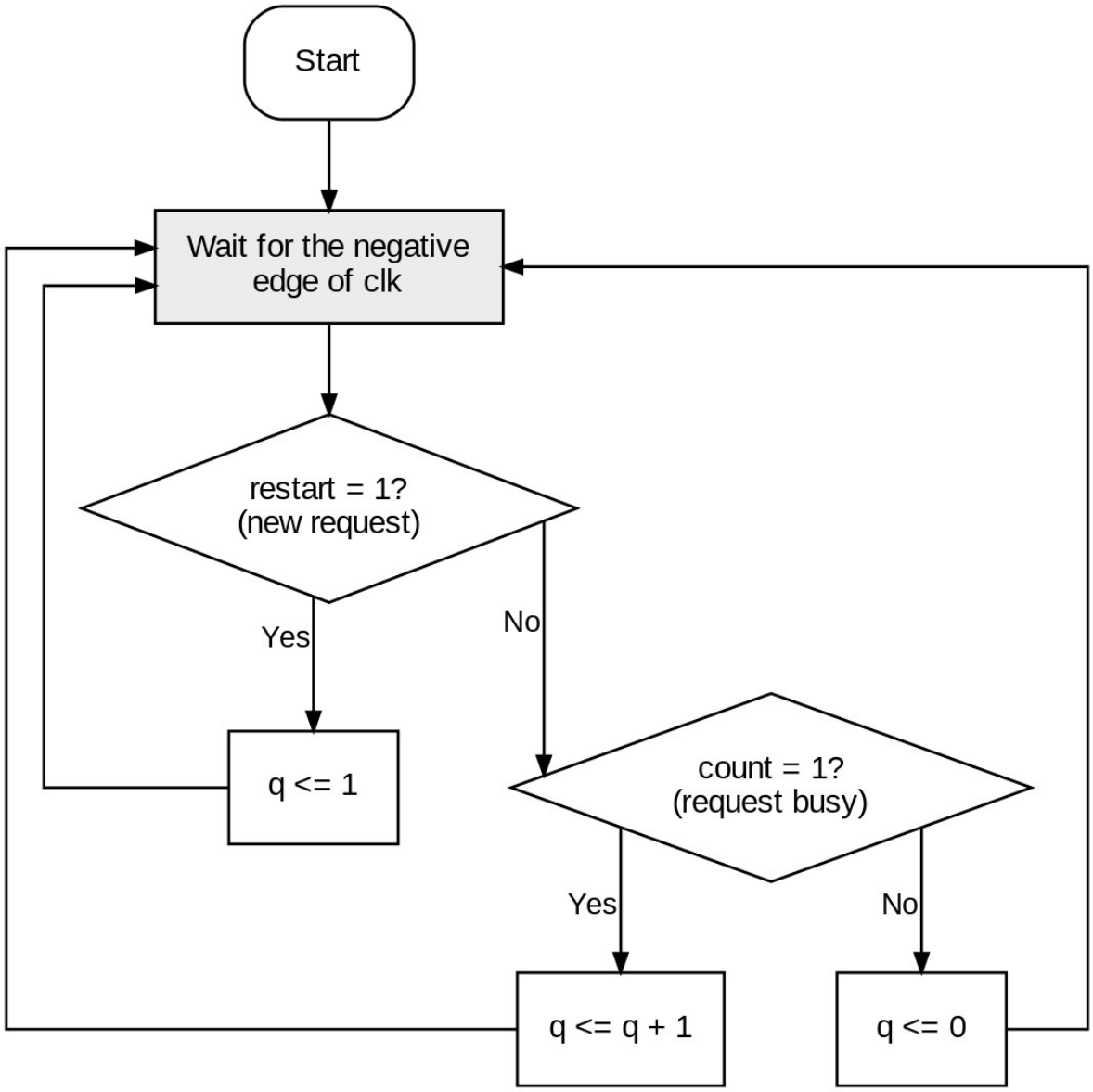


Figure 29: `counter`: operation flowchart

Code

Listing: two_way/blocks/pos_dff.vhd

```
-- Positive edge D flip-flop: the negative edge dff clocked by the inverted clock
--   Used to make the cache write signal, which the specification requires to be on for
--   exactly one clock cycle, from a positive edge to the next positive edge.
```

```
library ieee;
use ieee.std_logic_1164.all;

entity pos_dff is
    port (
        d   : in  std_logic;
        clk : in  std_logic;
        q   : out std_logic
    );
end entity pos_dff;

architecture structural of pos_dff is
    signal clk_n, q_n : std_logic;
begin
    u_inv : entity work.not_gate port map (a => clk, y => clk_n);
    u_ff  : entity work.dff port map (d => d, clk => clk_n, q => q, qbar => q_n);
end architecture structural;
```

Listing: two_way/blocks/half_adder.vhd

```
-- Half adder: sum = a xor b, carry = a and b
library ieee;
use ieee.std_logic_1164.all;

entity half_adder is
    port (
        a, b  : in  std_logic;
        sum   : out std_logic;
        carry : out std_logic
    );
end entity half_adder;

architecture structural of half_adder is
begin
    u_sum   : entity work.xor2_gate port map (a => a, b => b, y => sum);
    u_carry : entity work.and2_gate port map (a => a, b => b, y => carry);
end architecture structural;
```

Listing: two_way/blocks/counter.vhd

```
-- 5-bit clock cycle counter of a request (negative edge)
--   restart = 1 : count <= 1   (the first negative edge of a request)
--   count   = 1 : count <= count + 1        count = 0 and restart = 0 : count <= 0 (synchronous clear)
--   The state machine counts the clock cycles of every request with it, so after the negative
--   edge where busy goes low it holds the request's length: read hit 2, write hit/miss 3,
--   read miss 19. During a read miss it also times the memory access and the four byte writes.
library ieee;
use ieee.std_logic_1164.all;
use work.cache_pkg.all;

entity counter is
    port (
        count   : in  std_logic;
        restart : in  std_logic;
        clk     : in  std_logic;
        q       : out std_logic_vector(COUNT_W-1 downto 0)
    );
end entity counter;

architecture structural of counter is
    signal q_i, q_n, sum, d, d_c : std_logic_vector(COUNT_W-1 downto 0);
    signal carry                 : std_logic_vector(COUNT_W downto 0);
    signal restart_n             : std_logic;
begin
    carry(0) <= '1';                                        -- add one
    u_rs_n : entity work.not_gate port map (a => restart, y => restart_n);
    g_bit : for i in 0 to COUNT_W-1 generate
        u_ha  : entity work.half_adder port map (a => q_i(i), b => carry(i), sum => sum(i), carry =>
carry(i+1));
        u_clr : entity work.and2_gate  port map (a => sum(i), b => count, y => d_c(i));
        -- restart: bit 0 is set, the other bits are cleared
        g_b0 : if i = 0 generate
            u_rs : entity work.or2_gate  port map (a => d_c(i), b => restart, y => d(i));
        end generate g_b0;
        g_bn : if i > 0 generate
            u_rs : entity work.and2_gate port map (a => d_c(i), b => restart_n, y => d(i));
        end generate g_bn;
```

```
        u_ff  : entity work.dff        port map (d => d(i), clk => clk, q => q_i(i), qbar => q_n(i));
    end generate g_bit;
    q <= q_i;
end architecture structural;
```

Testbench

- Clear, count from 1 to 20 (past 19, a read miss) and clear; restart loads 1 and the count runs to 19; restart straight from 19 (a back-to-back request) loads 1 again; then clear.

Listing: two_way/testbench/blocks/tb_counter.vhd

```
-- Testbench: counter (5 bits), counts on the negative edge while count = 1, clears when 0,
-- restart = 1 loads 1
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use work.tb_pkg.all;

entity tb_counter is
end entity tb_counter;

architecture sim of tb_counter is
    signal clk, count : std_logic := '1';
    signal restart    : std_logic := '0';
    signal q          : std_logic_vector(4 downto 0);
    signal done       : boolean := false;
begin
    dut : entity work.counter port map (count => count, restart => restart, clk => clk, q => q);

    clk_gen : process
    begin
        while not done loop
            clk <= '1'; wait for 5 ns;
            clk <= '0'; wait for 5 ns;
        end loop;
        wait;
    end process clk_gen;

    stimulus : process
        variable errors : natural := 0;
        procedure expect (n : natural) is
        begin
            if q /= conv_std_logic_vector(n, 5) then
                report "FAIL: count=" & str(q) & " expected " & str(conv_std_logic_vector(n, 5)) severity
error;
                errors := errors + 1;
            end if;
        end procedure;
    begin
        count <= '0';
        wait until falling_edge(clk); wait for 2 ns;
        expect(0);                                  -- cleared
        count <= '1';
        for i in 1 to 20 loop                       -- counts 1 .. 20 (past 19, a read miss)
            wait until falling_edge(clk); wait for 2 ns;
            expect(i);
        end loop;
        count <= '0';
        wait until falling_edge(clk); wait for 2 ns;
        expect(0);                                  -- synchronous clear
        restart <= '1';                             -- restart from 0
        wait until falling_edge(clk); wait for 2 ns;
        expect(1);
        restart <= '0'; count <= '1';
        for i in 2 to 19 loop
            wait until falling_edge(clk); wait for 2 ns;
            expect(i);
        end loop;
        count <= '0'; restart <= '1';               -- restart from 19 (back-to-back request)
        wait until falling_edge(clk); wait for 2 ns;
        expect(1);
        restart <= '0';
```

```
        wait until falling_edge(clk); wait for 2 ns;
        expect(0);
        if errors = 0 then
            report "tb_counter: ALL TESTS PASSED";
        else
            report "tb_counter: " & str(errors) & " TEST(S) FAILED" severity error;
        end if;
        done <= true;
        wait;
    end process stimulus;
end architecture sim;
```

Simulation results

Simulation result: PASS — `tb_counter`: all checks passed, simulation ended at 430 ns.

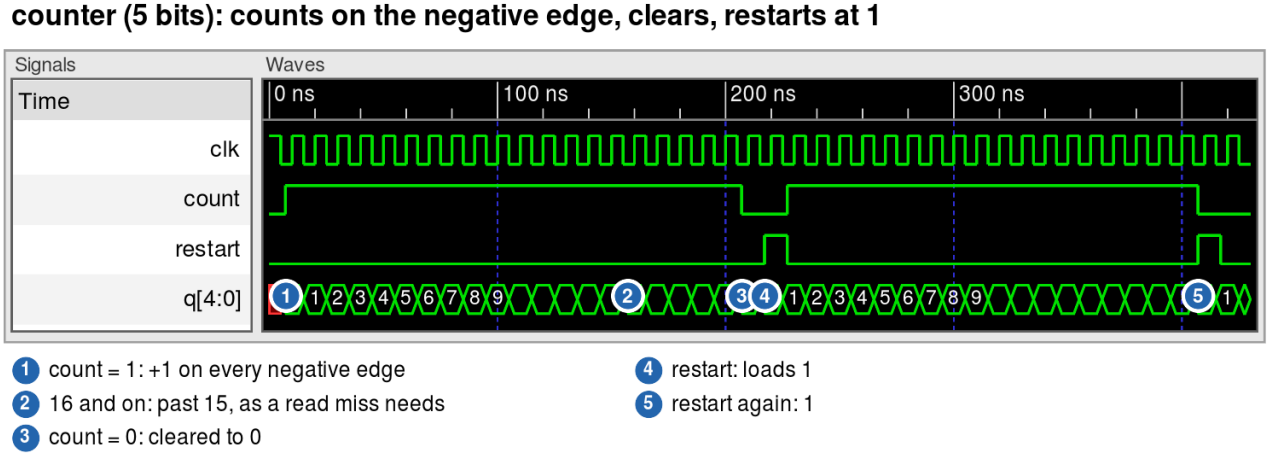


Figure 30: `tb_counter`: simulation waveform

q counts up on every negative edge while count = 1, is loaded with 1 when restart = 1 (also straight from 19, as for a back-to-back request) and returns to 0 when count = 0.

Simulation result: PASS — `tb_pos_dff`: all checks passed, simulation ended at 35 ns.

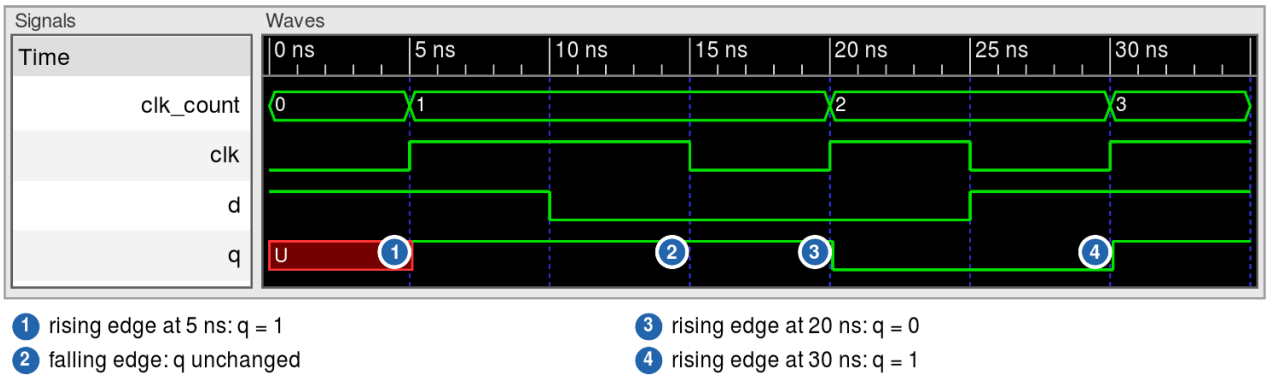


Figure 31: `tb_pos_dff`: simulation waveform

q takes d on the positive edges of clk (the course dff clocked by the inverted clock).

Simulation result: PASS — `tb_half_adder`: all checks passed, simulation ended at 40 ns.

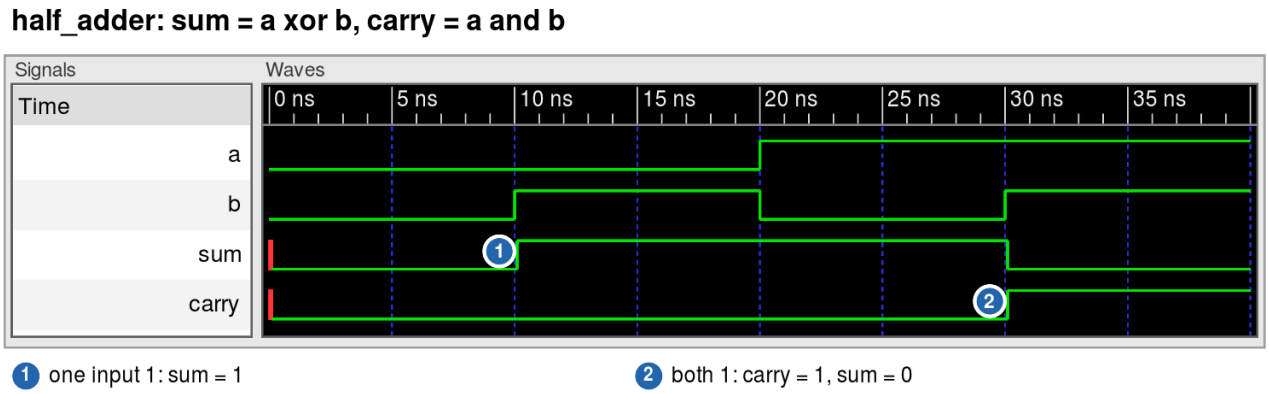


Figure 32: `tb_half_adder`: simulation waveform

sum = a xor b and carry = a and b for all four inputs.

5 Cache

5.1 Cache block

A cache block is 4 bytes (`cache_byte`, 8 cache cells each), a 2-bit tag and a valid bit, all cache cells. Byte i is enabled by `block_select` · `byte_select(i)`; the tag and valid cells by `block_select`. Reset also enables the valid cell, so a single reset pulse clears all four valid bits. The cell outputs drive shared buses (`read_data`, `tag_out`, `valid_out`) that are high impedance when the block is not selected.

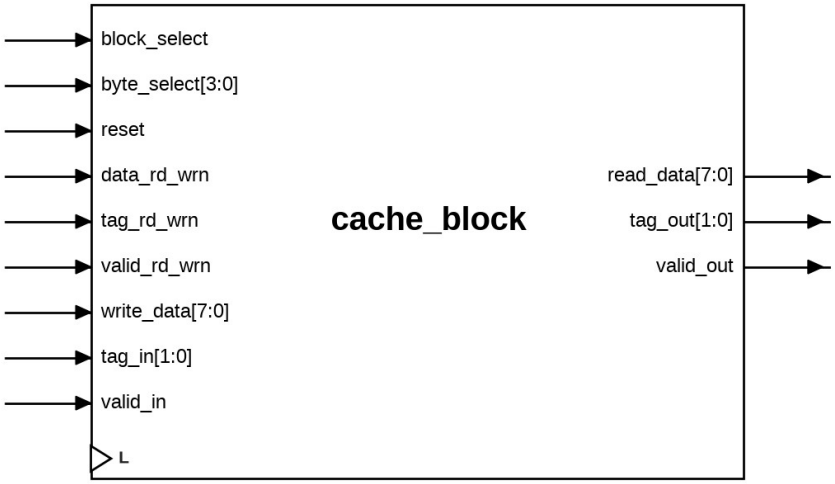


Figure 33: `cache_block`: inputs and outputs

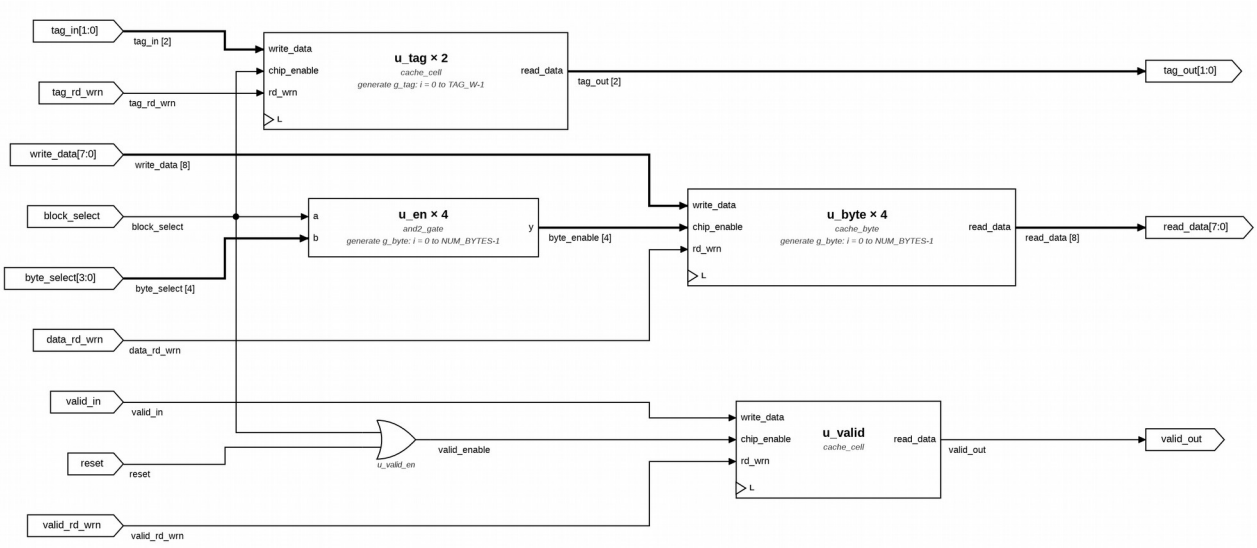


Figure 34: `cache_block`: internal structure

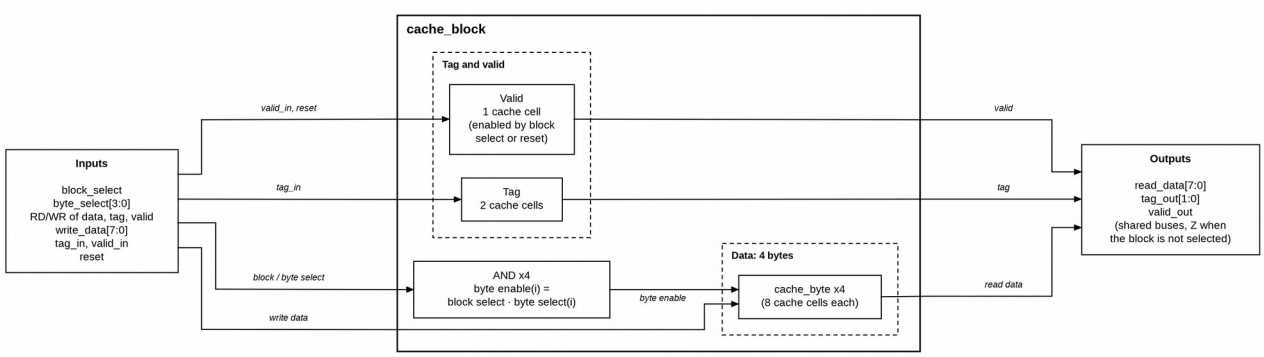


Figure 35: `cache_block`: block diagram

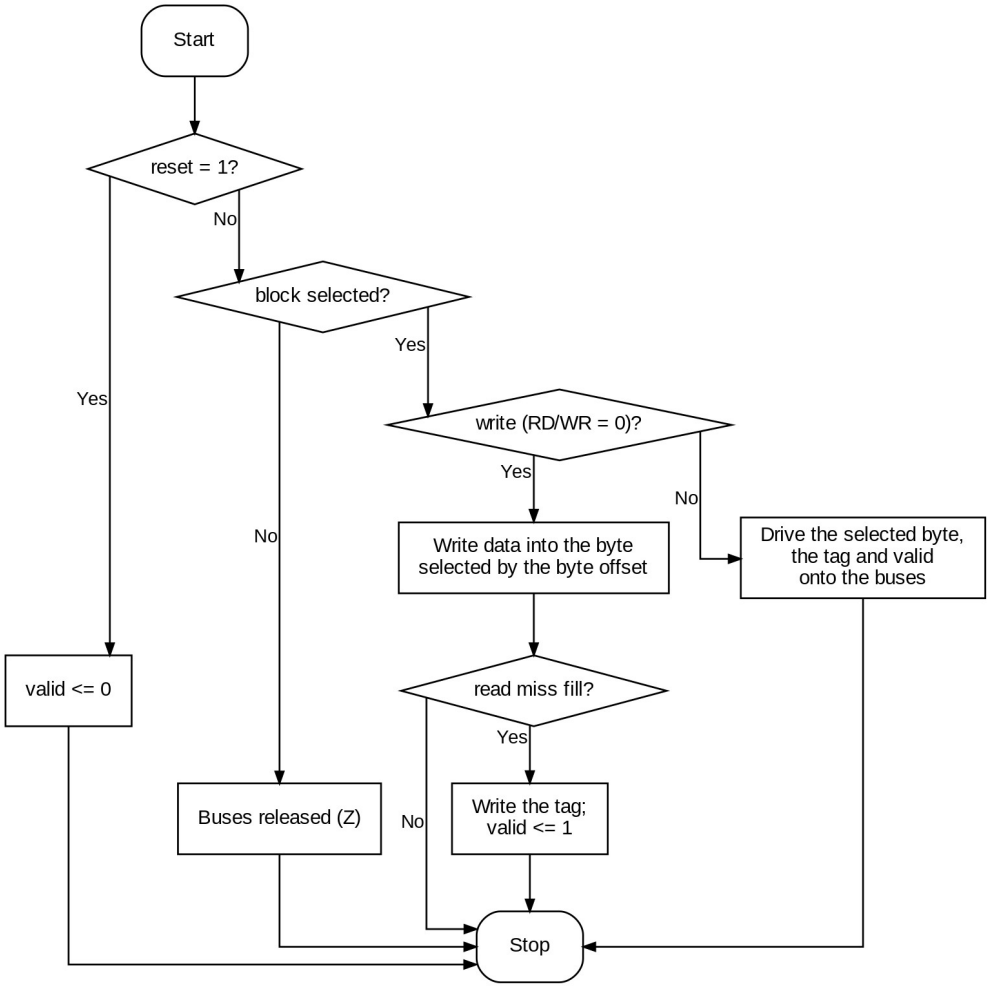


Figure 36: `cache_block`: operation flowchart

Code

Listing: two_way/cache/cache_byte.vhd

```
-- One byte of the cache: 8 cache cells sharing Chip enable and RD/WR
library ieee;
use ieee.std_logic_1164.all;
use work.cache_pkg.all;
```

```
entity cache_byte is
    port (
        write_data  : in  std_logic_vector(DATA_W-1 downto 0);
        chip_enable : in  std_logic;
        rd_wrn      : in  std_logic;
        read_data   : out std_logic_vector(DATA_W-1 downto 0)   -- 'Z' unless read
    );
end entity cache_byte;

architecture structural of cache_byte is
begin
    g_bit : for i in 0 to DATA_W-1 generate
        u_cell : entity work.cache_cell
            port map (write_data => write_data(i), chip_enable => chip_enable, rd_wrn => rd_wrn,
                      read_data => read_data(i));
    end generate g_bit;
end architecture structural;
```

Listing: two_way/cache/cache_block.vhd

```
-- One cache block: valid bit, 2-bit tag and 4 data bytes ("4 words/block"), all cache cells
--   Data byte i is enabled by block_select and byte_select(i) (the decoded byte offset).
--   The tag and valid cells are enabled by block_select; reset also enables the valid cell
--   so that reset clears every valid bit at once.
library ieee;
use ieee.std_logic_1164.all;
use work.cache_pkg.all;

entity cache_block is
    port (
        block_select : in  std_logic;
        byte_select  : in  std_logic_vector(NUM_BYTES-1 downto 0);   -- one-hot byte offset
        reset        : in  std_logic;
        data_rd_wrn  : in  std_logic;   -- 0 while the write signal is on
        tag_rd_wrn   : in  std_logic;   -- 0 while writing a block from memory (read miss)
        valid_rd_wrn : in  std_logic;   -- 0 while writing a block from memory, or reset
        write_data   : in  std_logic_vector(DATA_W-1 downto 0);
        tag_in       : in  std_logic_vector(TAG_W-1 downto 0);
        valid_in     : in  std_logic;
        read_data    : out std_logic_vector(DATA_W-1 downto 0);      -- shared buses ('Z' when
        tag_out      : out std_logic_vector(TAG_W-1 downto 0);       --  this block is not
        valid_out    : out std_logic                                 --  selected)
    );
end entity cache_block;

architecture structural of cache_block is
    signal byte_enable : std_logic_vector(NUM_BYTES-1 downto 0);
    signal valid_enable : std_logic;
begin
    -- ---------------- Data bytes ----------------
    g_byte : for i in 0 to NUM_BYTES-1 generate
        u_en   : entity work.and2_gate port map (a => block_select, b => byte_select(i), y =>
byte_enable(i));
        u_byte : entity work.cache_byte
            port map (write_data => write_data, chip_enable => byte_enable(i), rd_wrn => data_rd_wrn,
                      read_data => read_data);
    end generate g_byte;

    -- ---------------- Tag ----------------
    g_tag : for i in 0 to TAG_W-1 generate
        u_tag : entity work.cache_cell
            port map (write_data => tag_in(i), chip_enable => block_select, rd_wrn => tag_rd_wrn,
                      read_data => tag_out(i));
    end generate g_tag;

    -- ---------------- Valid ----------------
    u_valid_en : entity work.or2_gate port map (a => block_select, b => reset, y => valid_enable);
    u_valid    : entity work.cache_cell
        port map (write_data => valid_in, chip_enable => valid_enable, rd_wrn => valid_rd_wrn,
                  read_data => valid_out);
end architecture structural;
```

Testbench

- Reset clears valid.

- Write the 4 bytes with the tag and valid (as a read miss does), read every byte back.
- A single-byte write (write hit) changes only that byte and not the tag.
- Block not selected: all its buses are high impedance; reset clears valid again.

Listing: two_way/testbench/cache/tb_cache_block.vhd

```
-- Testbench: cache_block - write the 4 bytes, tag and valid, read them back, buses high
-- impedance when the block is not selected, reset clears valid
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use work.tb_pkg.all;

entity tb_cache_block is
end entity tb_cache_block;

architecture sim of tb_cache_block is
    signal block_select, reset : std_logic := '0';
    signal byte_select         : std_logic_vector(3 downto 0) := "0000";
    signal data_rd_wrn, tag_rd_wrn, valid_rd_wrn : std_logic := '1';
    signal write_data          : std_logic_vector(7 downto 0) := (others => '0');
    signal tag_in              : std_logic_vector(1 downto 0) := "00";
    signal valid_in            : std_logic := '0';
    signal read_data           : std_logic_vector(7 downto 0);
    signal tag_out             : std_logic_vector(1 downto 0);
    signal valid_out           : std_logic;
begin
    dut : entity work.cache_block
        port map (block_select => block_select, byte_select => byte_select, reset => reset,
                  data_rd_wrn => data_rd_wrn, tag_rd_wrn => tag_rd_wrn, valid_rd_wrn => valid_rd_wrn,
                  write_data => write_data, tag_in => tag_in, valid_in => valid_in,
                  read_data => read_data, tag_out => tag_out, valid_out => valid_out);

    stimulus : process
        variable errors : natural := 0;
        procedure check (cond : boolean; msg : string) is
        begin
            if not cond then
                report "FAIL: " & msg severity error;
                errors := errors + 1;
            end if;
        end procedure;
        function onehot (i : natural) return std_logic_vector is
            variable v : std_logic_vector(3 downto 0) := "0000";
        begin
            v(i) := '1';
            return v;
        end function;
    begin
        -- 1. reset clears valid (block not even selected)
        reset <= '1'; valid_rd_wrn <= '0'; valid_in <= '0';
        wait for 10 ns;
        reset <= '0'; valid_rd_wrn <= '1';
        block_select <= '1';
        wait for 10 ns;
        check(valid_out = '0', "valid = 0 after reset");

        -- 2. write the 4 bytes with the tag and valid (as during a read miss)
        tag_in <= "10"; valid_in <= '1';
        for i in 0 to 3 loop
            byte_select <= onehot(i); write_data <= conv_std_logic_vector(16#A0# + i, 8);
            wait for 5 ns;
            data_rd_wrn <= '0'; tag_rd_wrn <= '0'; valid_rd_wrn <= '0';   -- write signal on
            wait for 10 ns;
            data_rd_wrn <= '1'; tag_rd_wrn <= '1'; valid_rd_wrn <= '1';   -- write signal off
            wait for 5 ns;
        end loop;

        -- 3. read every byte back
        for i in 0 to 3 loop
            byte_select <= onehot(i);
            wait for 5 ns;
```

```
            check(read_data = conv_std_logic_vector(16#A0# + i, 8),
                  "byte " & str(i) & " = " & str(read_data));
        end loop;
        check(tag_out = "10" and valid_out = '1', "tag = 10 and valid = 1");

        -- 4. a write to byte 2 alone (write hit) changes only byte 2
        byte_select <= onehot(2); write_data <= "11111111"; tag_in <= "01";
        wait for 5 ns;
        data_rd_wrn <= '0'; wait for 10 ns; data_rd_wrn <= '1';
        wait for 5 ns;
        check(read_data = "11111111", "byte 2 written");
        byte_select <= onehot(1); wait for 5 ns;
        check(read_data = conv_std_logic_vector(16#A1#, 8), "byte 1 unchanged");
        check(tag_out = "10", "tag unchanged by a data write");

        -- 5. block not selected: shared buses released
        block_select <= '0'; wait for 5 ns;
        check(read_data = "ZZZZZZZZ" and tag_out = "ZZ" and valid_out = 'Z', "buses high impedance");

        -- 6. reset clears valid again
        reset <= '1'; valid_rd_wrn <= '0'; valid_in <= '0'; wait for 10 ns;
        reset <= '0'; valid_rd_wrn <= '1'; block_select <= '1'; wait for 5 ns;
        check(valid_out = '0', "reset invalidates the block");

        if errors = 0 then
            report "tb_cache_block: ALL TESTS PASSED";
        else
            report "tb_cache_block: " & str(errors) & " TEST(S) FAILED" severity error;
        end if;
        wait;
    end process stimulus;
end architecture sim;
```

Simulation results

Simulation result: **PASS** — `tb_cache_block`: all checks passed, simulation ended at 165 ns.

cache_block: 4 bytes, the tag and the valid bit

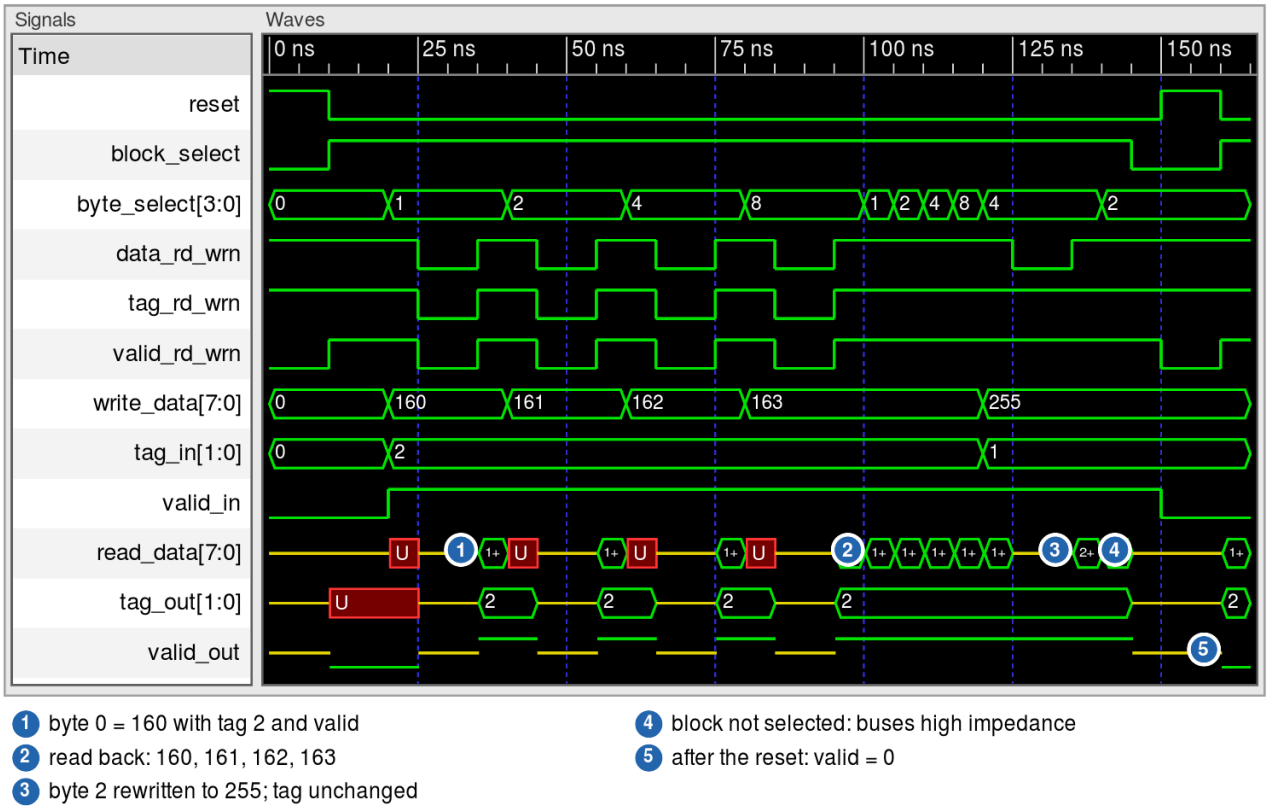


Figure 37: `tb_cache_block`: simulation waveform

Each write is a pulse on the RD/WR inputs with the data stable around it; reads appear on the shared buses only while the block is selected.

Simulation result: PASS — `tb_cache_byte`: all checks passed, simulation ended at 210 ns.

cache_byte: 8 cache cells, written and read back as a byte

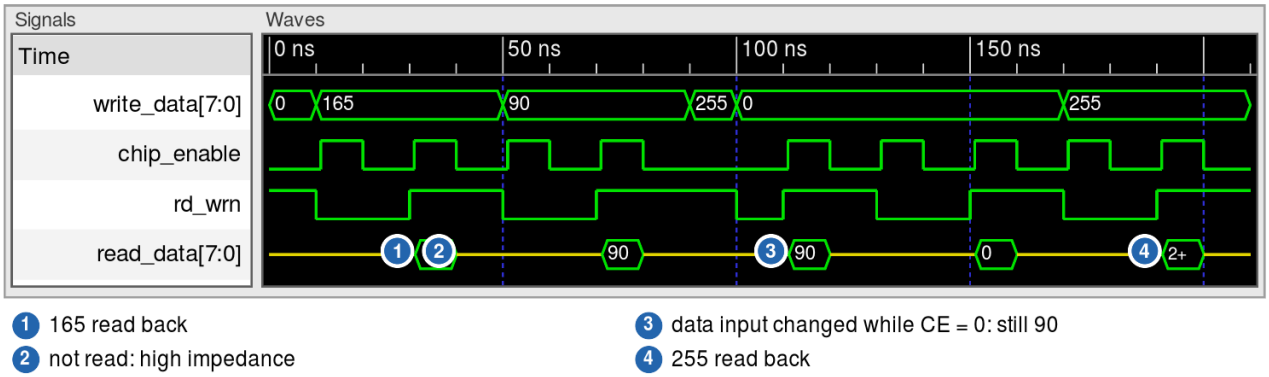


Figure 38: `tb_cache_byte`: simulation waveform

Writes store write_data while chip enable = 1 and RD/WR = 0; the byte appears on read_data only while it is read.

5.2 Cache way

One way of the cache: 4 cache blocks (16 bytes). The block select and byte select come from the cache's decoders, which both ways share. The way's own write signal writes the selected byte; during a read miss (fill = 1) it also writes the tag and sets the valid bit. The selected byte, tag and valid bit are always read onto the way's buses. During a read miss (fill = 1) a Dlatch on every output (8 data bits, 2 tag bits, the valid bit) holds the values from before the fill: while a tag, valid or data cell is written its Read Data is high impedance, and the byte select steps through the bytes being written, so without the latches the hit/miss logic would see Z and X until the block is complete. The latches pass the cells again as soon as fill = 0, for the read of the requested byte.

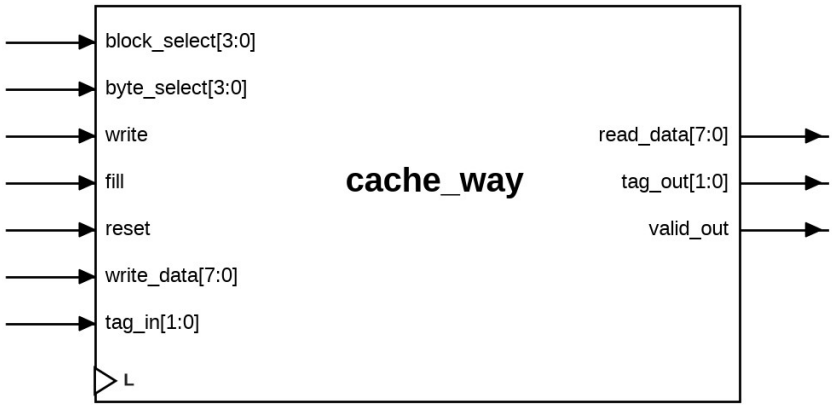


Figure 39: `cache_way`: inputs and outputs

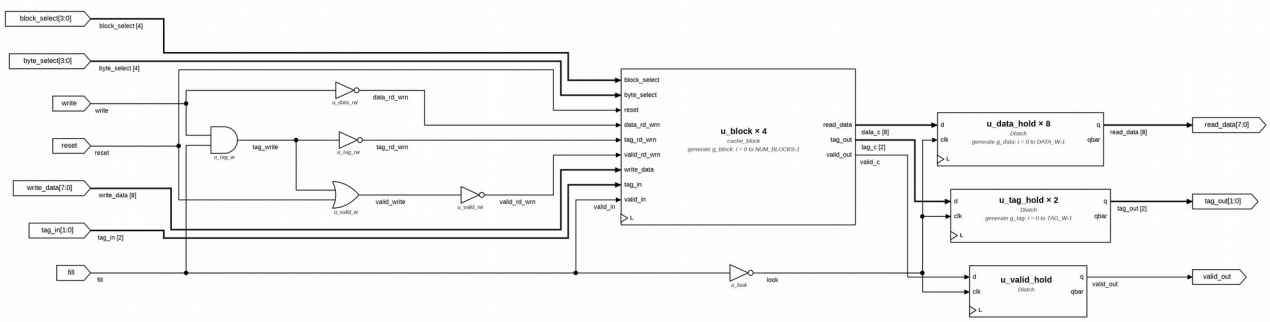


Figure 40: `cache_way`: internal structure

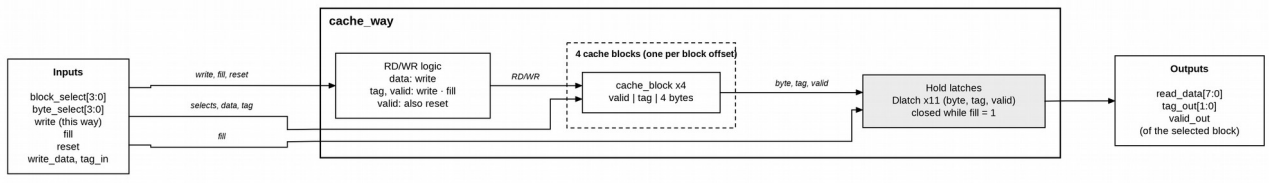


Figure 41: `cache_way`: block diagram

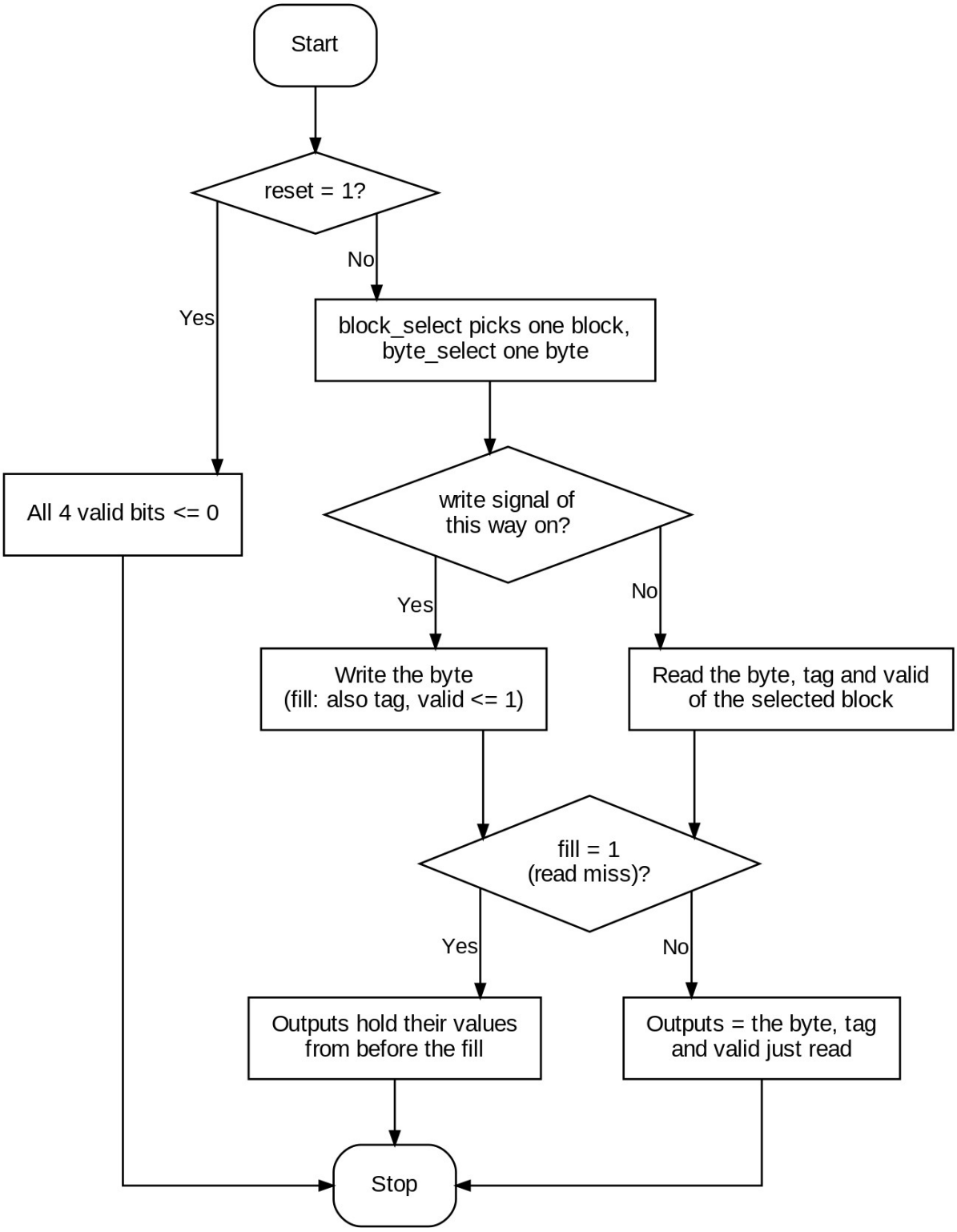


Figure 42: `cache_way`: operation flowchart

Code

Listing: two_way/cache/cache_way.vhd

```
-- One way of the two-way cache: 4 cache blocks (16 bytes)
--   block_select / byte_select come from the cache's shared decoders (one-hot block offset
--   and byte offset of the address).
--   write = 1 writes write_data into the selected byte of this way; during a read miss
--   (fill = 1) it also stores the tag and sets the valid bit. reset clears every valid bit.
--   The selected byte, and the tag and valid bit of the selected block, are always read, except
--   during a read miss (fill = 1): latches then hold them at their values from before the fill,
--   so they do not show the bytes being written, nor the 'Z' of the tag / valid / data cells
--   while they are written, until the whole block is in.
library ieee;
use ieee.std_logic_1164.all;
use work.cache_pkg.all;

entity cache_way is
    port (
        block_select : in  std_logic_vector(NUM_BLOCKS-1 downto 0);
        byte_select  : in  std_logic_vector(NUM_BYTES-1 downto 0);
        write        : in  std_logic;   -- write signal of this way
        fill         : in  std_logic;   -- read miss: the block comes from memory
        reset        : in  std_logic;
        write_data   : in  std_logic_vector(DATA_W-1 downto 0);
        tag_in       : in  std_logic_vector(TAG_W-1 downto 0);
        read_data    : out std_logic_vector(DATA_W-1 downto 0);
        tag_out      : out std_logic_vector(TAG_W-1 downto 0);   -- tag of the selected block
        valid_out    : out std_logic                             -- valid bit of the selected block
    );
end entity cache_way;

architecture structural of cache_way is
    signal tag_write, valid_write                : std_logic;
    signal data_rd_wrn, tag_rd_wrn, valid_rd_wrn   : std_logic;
    signal valid_in                              : std_logic;
    -- the cells' outputs, and the latches that pass them while fill = 0
    signal data_c, data_n                        : std_logic_vector(DATA_W-1 downto 0);
    signal tag_c, tag_n                          : std_logic_vector(TAG_W-1 downto 0);
    signal valid_c, valid_n, look                : std_logic;
begin
    -- ---------------- RD/WR of the cells ----------------
    -- data: written whenever the write signal of this way is on
    u_data_rw  : entity work.not_gate  port map (a => write, y => data_rd_wrn);
    -- tag and valid: written with the data bytes of a read miss
    u_tag_w    : entity work.and2_gate port map (a => write, b => fill, y => tag_write);
    u_tag_rw   : entity work.not_gate  port map (a => tag_write, y => tag_rd_wrn);
    -- valid: also written (to 0) by reset. The value written is fill (1 during a read miss,
    -- 0 otherwise), not "not reset", which would rise before the valid cell's write enable
    -- falls when reset ends and the latch would capture a 1 (a hold-time race).
    u_valid_w  : entity work.or2_gate  port map (a => tag_write, b => reset, y => valid_write);
    u_valid_rw : entity work.not_gate  port map (a => valid_write, y => valid_rd_wrn);
    valid_in <= fill;

    -- ---------------- Cache blocks ----------------
    g_block : for i in 0 to NUM_BLOCKS-1 generate
        u_block : entity work.cache_block
            port map (block_select => block_select(i), byte_select => byte_select, reset => reset,
                      data_rd_wrn => data_rd_wrn, tag_rd_wrn => tag_rd_wrn, valid_rd_wrn => valid_rd_wrn,
                      write_data => write_data, tag_in => tag_in, valid_in => valid_in,
                      read_data => data_c, tag_out => tag_c, valid_out => valid_c);
    end generate g_block;

    -- ---------------- Outputs, held during a read miss ----------------
    u_look : entity work.not_gate port map (a => fill, y => look);
    g_data : for i in 0 to DATA_W-1 generate
        u_data_hold : entity work.Dlatch port map (d => data_c(i), clk => look, q => read_data(i), qbar =>
data_n(i));
    end generate g_data;
    g_tag : for i in 0 to TAG_W-1 generate
        u_tag_hold : entity work.Dlatch port map (d => tag_c(i), clk => look, q => tag_out(i), qbar =>
tag_n(i));
    end generate g_tag;
    u_valid_hold : entity work.Dlatch port map (d => valid_c, clk => look, q => valid_out, qbar =>
valid_n);
end architecture structural;
```

Testbench

- Reset invalidates every block.
- A read miss into block 1 with tag 10 (4 bytes, tag and valid), read back.
- Other blocks untouched; a write hit to one byte keeps the tag and valid bit; reset again.

Listing: two_way/testbench/cache/tb_cache_way.vhd

```
-- Testbench: cache_way - reset, a block written from memory (tag and valid too), read back,
-- a single-byte write, other blocks untouched, reset again
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use work.tb_pkg.all;

entity tb_cache_way is
end entity tb_cache_way;

architecture sim of tb_cache_way is
    signal block_select, byte_select : std_logic_vector(3 downto 0) := "0001";
    signal write, fill, reset        : std_logic := '0';
    signal write_data, read_data     : std_logic_vector(7 downto 0) := (others => '0');
    signal tag_in, tag_out           : std_logic_vector(1 downto 0) := "00";
    signal valid_out                 : std_logic;
begin
    dut : entity work.cache_way
        port map (block_select => block_select, byte_select => byte_select, write => write, fill => fill,
                  reset => reset, write_data => write_data, tag_in => tag_in, read_data => read_data,
                  tag_out => tag_out, valid_out => valid_out);

    stimulus : process
        variable errors : natural := 0;
        procedure check (cond : boolean; msg : string) is
        begin
            if not cond then
                report "FAIL: " & msg severity error;
                errors := errors + 1;
            end if;
        end procedure;
        function onehot (i : natural) return std_logic_vector is
            variable v : std_logic_vector(3 downto 0) := "0000";
        begin
            v(i) := '1';
            return v;
        end function;
        procedure sel (blk, byt : natural) is
        begin
            block_select <= onehot(blk); byte_select <= onehot(byt);
            wait for 5 ns;
        end procedure;
        procedure write_pulse is   -- address and data stable 5 ns before and after
        begin
            wait for 5 ns;
            write <= '1'; wait for 10 ns; write <= '0';
            wait for 5 ns;
        end procedure;
    begin
        -- 1. reset: every block invalid
        reset <= '1'; wait for 10 ns; reset <= '0';
        for b in 0 to 3 loop
            sel(b, 0);
            check(valid_out = '0', "block " & str(b) & " invalid after reset");
        end loop;

        -- 2. read miss into block 1 with tag 10: 4 bytes, tag and valid
        tag_in <= "10"; fill <= '1';
        for i in 0 to 3 loop
            block_select <= onehot(1); byte_select <= onehot(i);
            write_data <= conv_std_logic_vector(16#40# + i, 8);
            write_pulse;
        end loop;
        fill <= '0';
```

```
        for i in 0 to 3 loop
            sel(1, i);
            check(read_data = conv_std_logic_vector(16#40# + i, 8), "block 1 byte " & str(i) & " = " &
str(read_data));
        end loop;
        check(tag_out = "10" and valid_out = '1', "block 1: tag 10, valid");

        -- 3. other blocks untouched
        sel(0, 0); check(valid_out = '0', "block 0 still invalid");
        sel(3, 0); check(valid_out = '0', "block 3 still invalid");

        -- 4. write hit to block 1 byte 2 (fill = 0): tag and valid unchanged
        block_select <= onehot(1); byte_select <= onehot(2); write_data <= "11001100"; tag_in <= "01";
        write_pulse;
        sel(1, 2);
        check(read_data = "11001100" and tag_out = "10" and valid_out = '1', "byte 2 written, tag/valid
unchanged");

        -- 5. reset invalidates block 1
        reset <= '1'; wait for 10 ns; reset <= '0';
        sel(1, 0);
        check(valid_out = '0', "block 1 invalid after the second reset");

        if errors = 0 then
            report "tb_cache_way: ALL TESTS PASSED";
        else
            report "tb_cache_way: " & str(errors) & " TEST(S) FAILED" severity error;
        end if;
        wait;
    end process stimulus;
end architecture sim;
```

Simulation results

Simulation result: PASS — `tb_cache_way`: all checks passed, simulation ended at 180 ns.

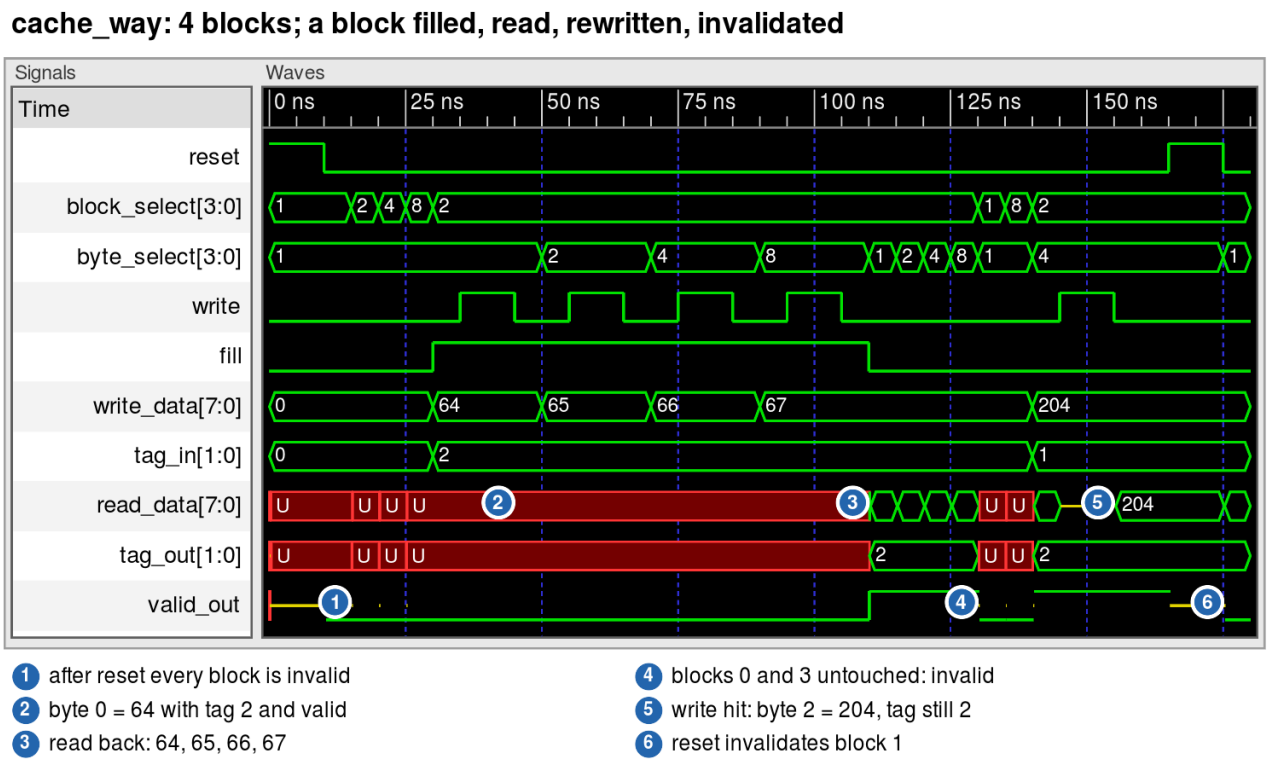


Figure 43: `tb_cache_way`: simulation waveform

Each write is a 10 ns pulse with block select, byte select and data stable 5 ns before and after it.

5.3 Hit/Miss

Hit/Miss of one way: hit = valid · (tag of the address = tag stored in the block of that way); miss = not hit. The tag compare is a 2-bit `comparator_n`. The cache has one per way.



Figure 44: `hit_miss`: inputs and outputs

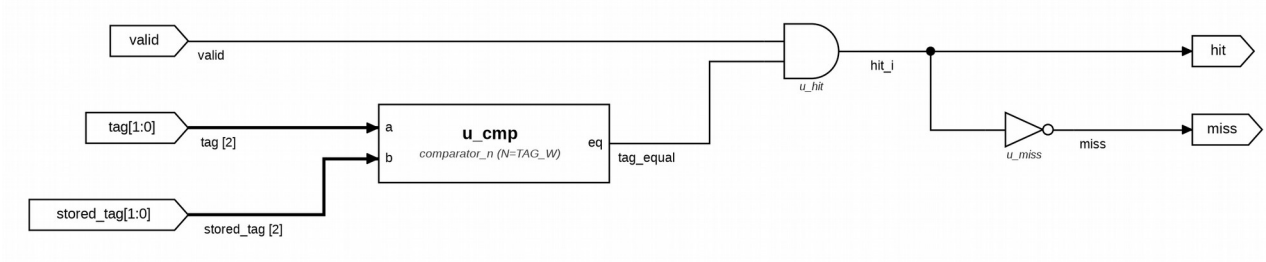


Figure 45: `hit_miss`: internal structure

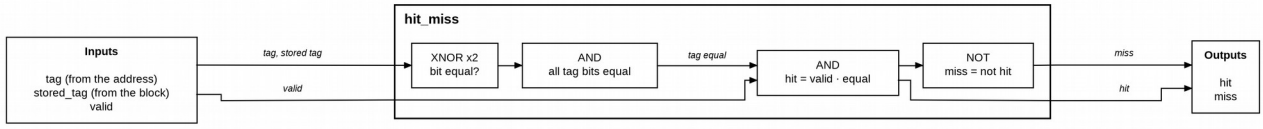


Figure 46: `hit_miss`: block diagram

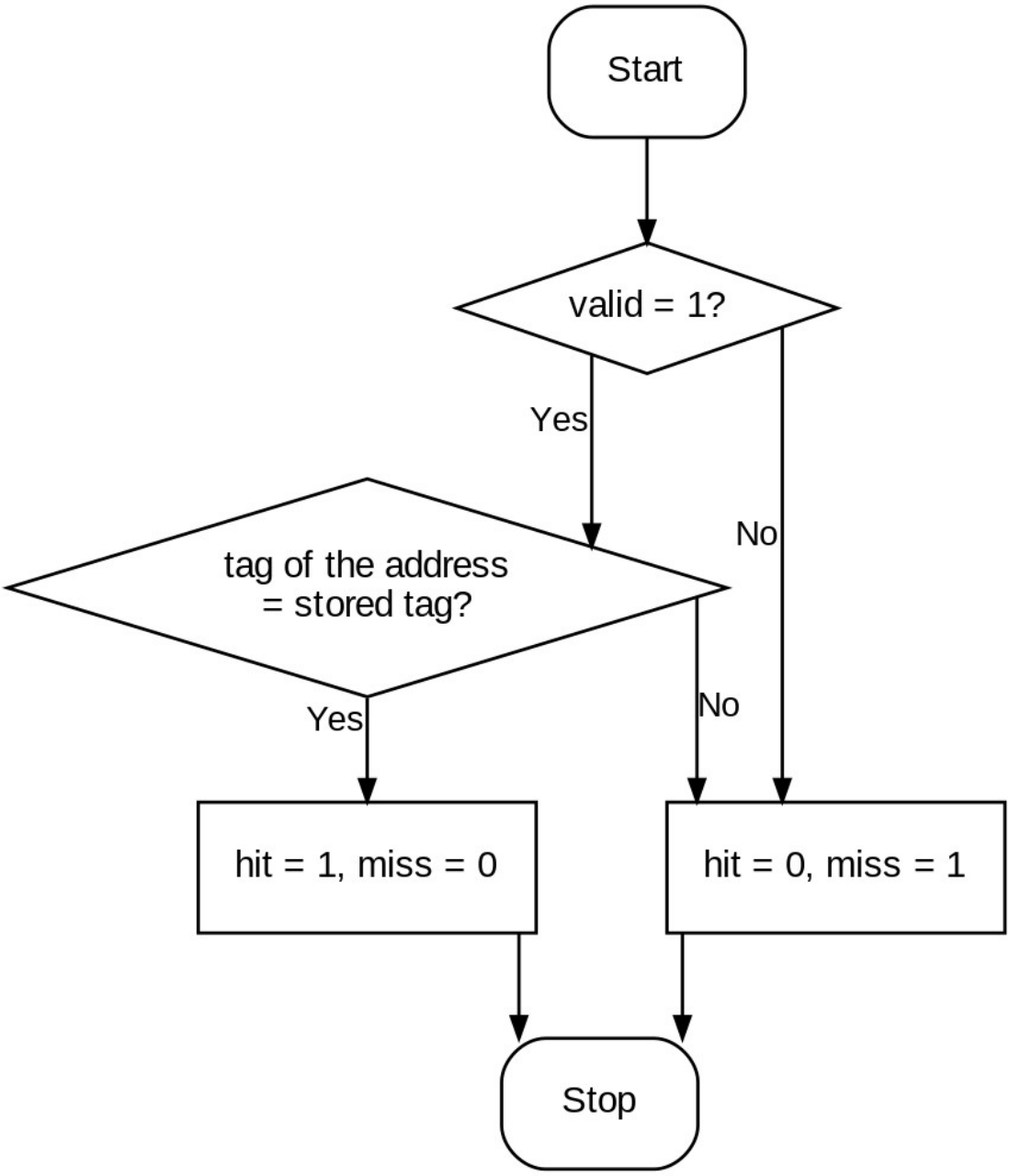


Figure 47: `hit_miss`: operation flowchart

Code

Listing: two_way/cache/hit_miss.vhd

```
-- Hit/Miss: compare the tag bits of the address with the tag stored in the addressed block,
-- and check that the block is valid.   hit = valid and (tag = stored tag),  miss = not hit
library ieee;
use ieee.std_logic_1164.all;
use work.cache_pkg.all;

entity hit_miss is
    port (
        tag        : in  std_logic_vector(TAG_W-1 downto 0);   -- from the CPU address
```

```
        stored_tag : in  std_logic_vector(TAG_W-1 downto 0);   -- from the cache
        valid      : in  std_logic;
        hit        : out std_logic;
        miss       : out std_logic
    );
end entity hit_miss;

architecture structural of hit_miss is
    signal tag_equal, hit_i : std_logic;
begin
    u_cmp  : entity work.comparator_n generic map (N => TAG_W) port map (a => tag, b => stored_tag, eq =>
tag_equal);
    u_hit  : entity work.and2_gate port map (a => valid, b => tag_equal, y => hit_i);
    u_miss : entity work.not_gate  port map (a => hit_i, y => miss);
    hit <= hit_i;
end architecture structural;
```

Testbench

- All 32 combinations of address tag, stored tag and valid.

Listing: two_way/testbench/cache/tb_hit_miss.vhd

```
-- Testbench: hit_miss - all 32 combinations of tag, stored tag and valid
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use work.tb_pkg.all;

entity tb_hit_miss is
end entity tb_hit_miss;

architecture sim of tb_hit_miss is
    signal tag, stored_tag : std_logic_vector(1 downto 0) := "00";
    signal valid, hit, miss : std_logic := '0';
begin
    dut : entity work.hit_miss
        port map (tag => tag, stored_tag => stored_tag, valid => valid, hit => hit, miss => miss);

    stimulus : process
        variable errors   : natural := 0;
        variable expected : std_logic;
    begin
        for v in 0 to 1 loop
            for t in 0 to 3 loop
                for s in 0 to 3 loop
                    tag <= conv_std_logic_vector(t, 2); stored_tag <= conv_std_logic_vector(s, 2);
                    if v = 1 then valid <= '1'; else valid <= '0'; end if;
                    wait for 10 ns;
                    if v = 1 and t = s then expected := '1'; else expected := '0'; end if;
                    if hit /= expected or miss /= not expected then
                        report "FAIL: tag=" & str(tag) & " stored=" & str(stored_tag) & " valid=" &
str(valid)
                             & " -> hit=" & str(hit) & " miss=" & str(miss) severity error;
                        errors := errors + 1;
                    end if;
                end loop;
            end loop;
        end loop;
        if errors = 0 then
            report "tb_hit_miss: ALL TESTS PASSED";
        else
            report "tb_hit_miss: " & str(errors) & " TEST(S) FAILED" severity error;
        end if;
        wait;
    end process stimulus;
end architecture sim;
```

Simulation results

Simulation result: PASS — `tb_hit_miss`: all checks passed, simulation ended at 320 ns.

hit_miss: hit when the tags match and the block is valid

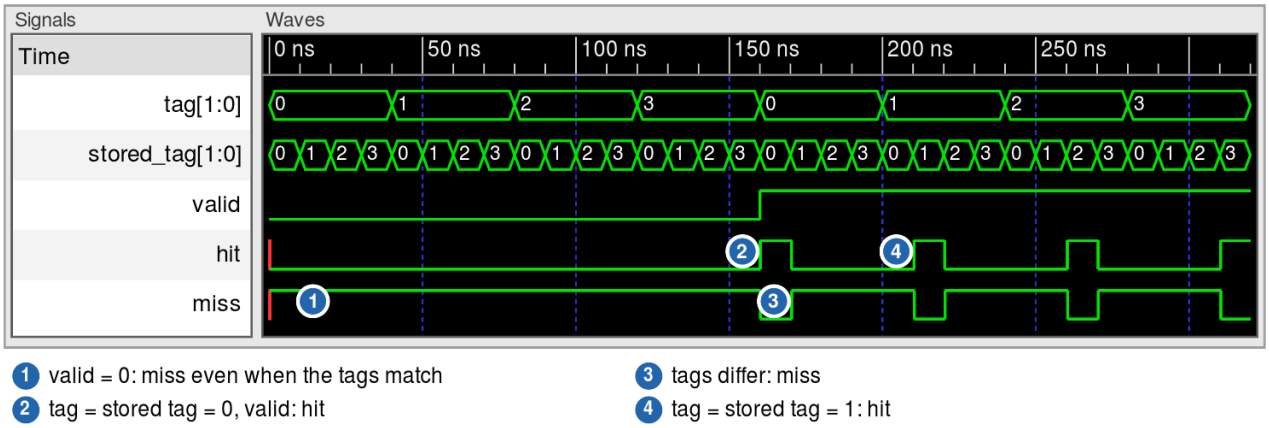


Figure 48: `tb_hit_miss`: simulation waveform

hit is 1 only in the steps where valid = 1 and the two tags are equal.

5.4 Way select

Combines the two ways: hit = hit0 + hit1, miss = not hit, hit_way = hit1 (the way that holds the block). The victim (the way a read miss fills) is an invalid way first, otherwise the LRU way: victim = valid0 · (not valid1 + lru_way).

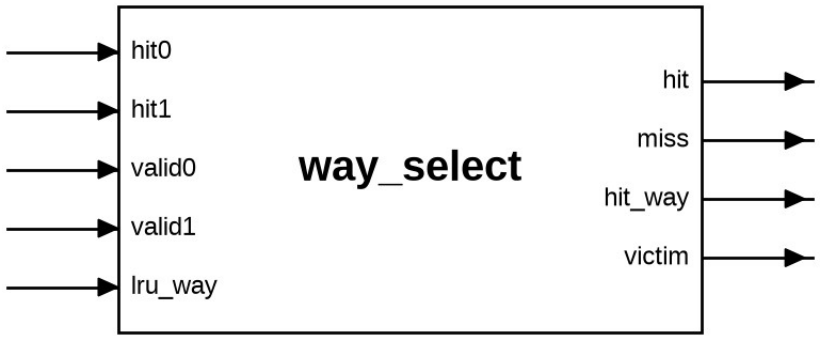


Figure 49: `way_select`: inputs and outputs

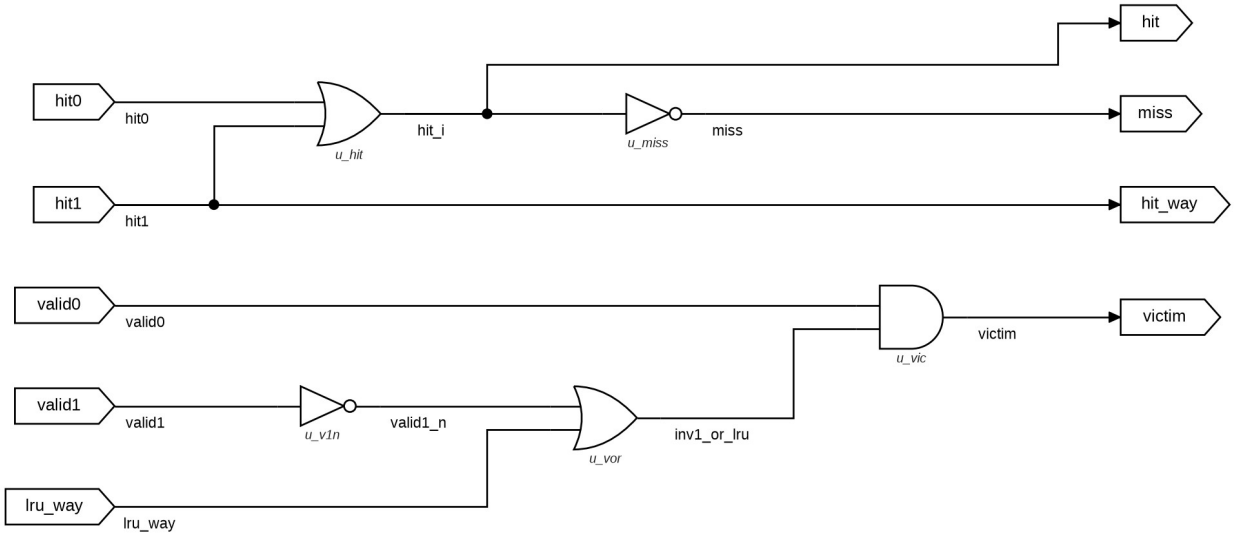


Figure 50: `way_select`: internal structure

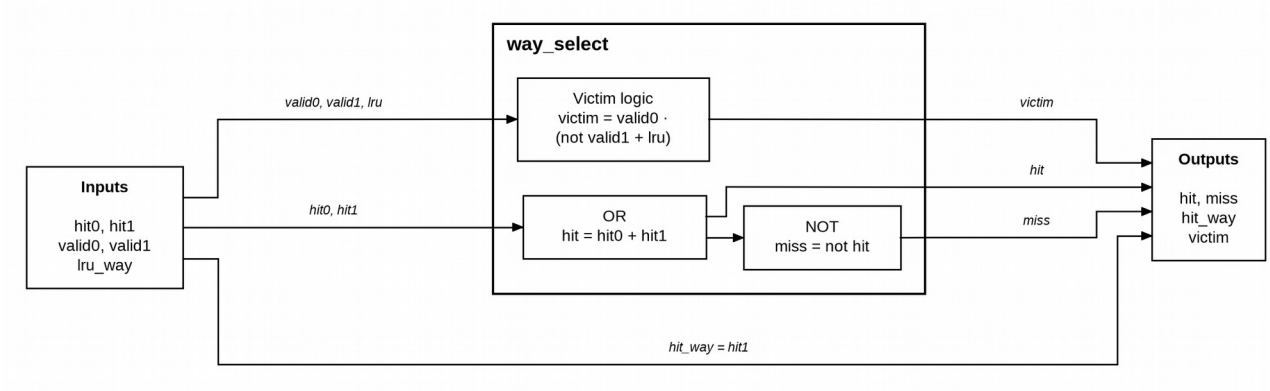


Figure 51: `way_select`: block diagram

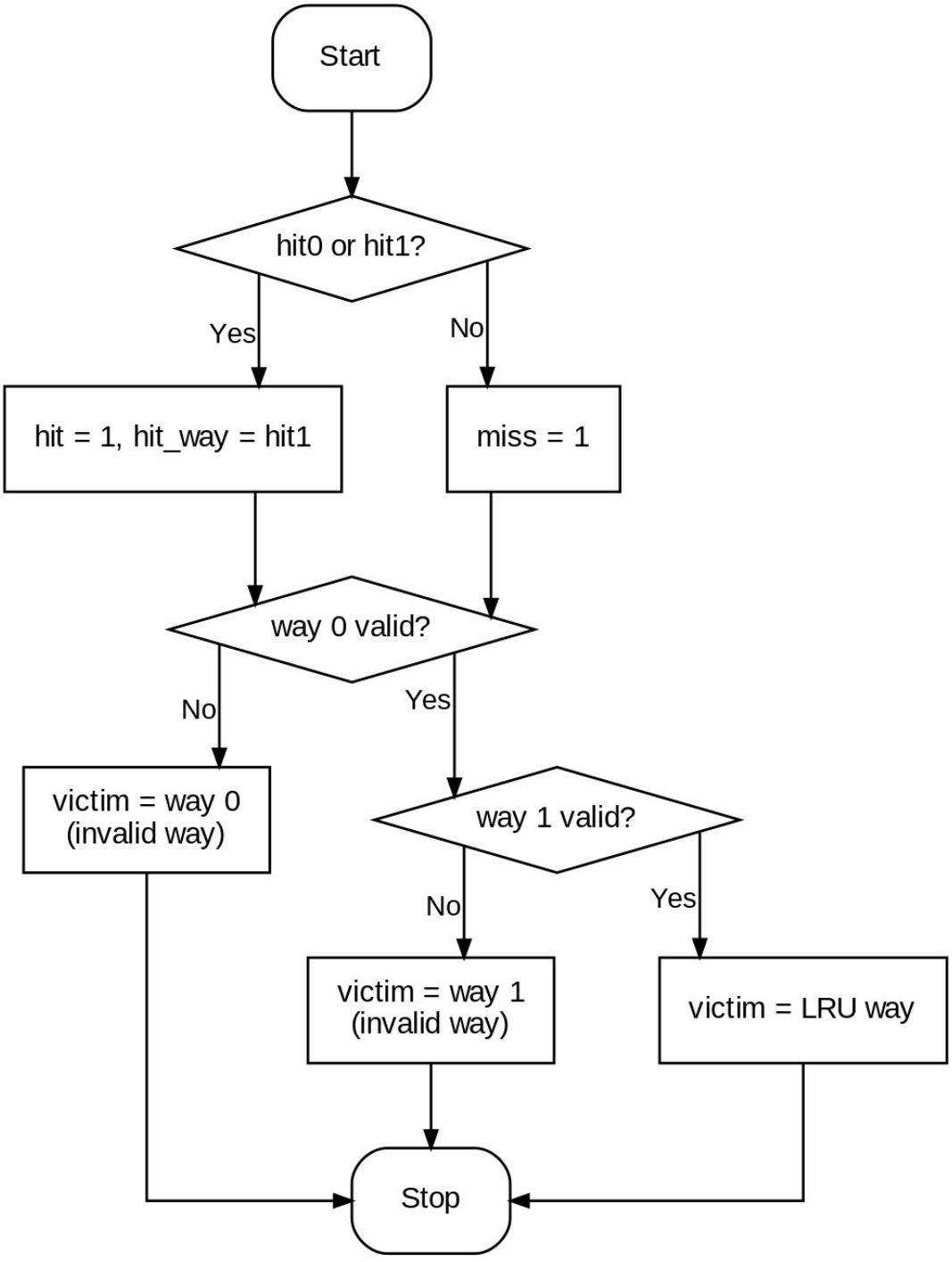


Figure 52: `way_select`: operation flowchart

Code

Listing: two_way/cache/way_select.vhd

```
-- Way selection of the two-way cache
--   hit     = hit0 or hit1,  miss = not hit,  hit_way = hit1 (the way that holds the block)
--   victim  = the way a read miss fills: an invalid way first, else the LRU way
--           = valid0 and (not valid1 or lru_way)
```

```
library ieee;
use ieee.std_logic_1164.all;

entity way_select is
    port (
        hit0, hit1     : in  std_logic;   -- Hit/Miss of each way
        valid0, valid1 : in  std_logic;   -- valid bit of the addressed block in each way
        lru_way        : in  std_logic;
        hit            : out std_logic;
        miss           : out std_logic;
        hit_way        : out std_logic;
        victim         : out std_logic
    );
end entity way_select;

architecture structural of way_select is
    signal hit_i, valid1_n, inv1_or_lru : std_logic;
begin
    u_hit  : entity work.or2_gate  port map (a => hit0, b => hit1, y => hit_i);
    u_miss : entity work.not_gate  port map (a => hit_i, y => miss);
    u_v1n  : entity work.not_gate  port map (a => valid1, y => valid1_n);
    u_vor  : entity work.or2_gate  port map (a => valid1_n, b => lru_way, y => inv1_or_lru);
    u_vic  : entity work.and2_gate port map (a => valid0, b => inv1_or_lru, y => victim);
    hit     <= hit_i;
    hit_way <= hit1;
end architecture structural;
```

Testbench

- All 32 combinations of hit0, hit1, valid0, valid1 and lru_way against a reference model.

Listing: two_way/testbench/cache/tb_way_select.vhd

```
-- Testbench: way_select - all 32 combinations of hit0, hit1, valid0, valid1 and lru_way
-- compared with a reference model (victim = invalid way first, else the LRU way)
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use work.tb_pkg.all;

entity tb_way_select is
end entity tb_way_select;

architecture sim of tb_way_select is
    signal hit0, hit1, valid0, valid1, lru_way : std_logic := '0';
    signal hit, miss, hit_way, victim          : std_logic;
begin
    dut : entity work.way_select
        port map (hit0 => hit0, hit1 => hit1, valid0 => valid0, valid1 => valid1, lru_way => lru_way,
                  hit => hit, miss => miss, hit_way => hit_way, victim => victim);

    stimulus : process
        variable v      : std_logic_vector(4 downto 0);
        variable e_hit, e_vic : std_logic;
        variable errors : natural := 0;
    begin
        for i in 0 to 31 loop
            v := conv_std_logic_vector(i, 5);
            hit0 <= v(4); hit1 <= v(3); valid0 <= v(2); valid1 <= v(1); lru_way <= v(0);
            wait for 10 ns;
            e_hit := v(4) or v(3);
            if v(2) = '0' then e_vic := '0';          -- way 0 invalid
            elsif v(1) = '0' then e_vic := '1';       -- way 1 invalid
            else e_vic := v(0);                       -- both valid: the LRU way
            end if;
            if hit /= e_hit or miss /= not e_hit or hit_way /= v(3) or victim /= e_vic then
                report "FAIL: hit0 hit1 valid0 valid1 lru = " & str(v) & " -> hit=" & str(hit)
                     & " hit_way=" & str(hit_way) & " victim=" & str(victim) severity error;
                errors := errors + 1;
            end if;
        end loop;
        if errors = 0 then
            report "tb_way_select: ALL TESTS PASSED";
        else
```

```
            report "tb_way_select: " & str(errors) & " TEST(S) FAILED" severity error;
        end if;
        wait;
    end process stimulus;
end architecture sim;
```

Simulation results

Simulation result: **PASS** — `tb_way_select`: all checks passed, simulation ended at 320 ns.

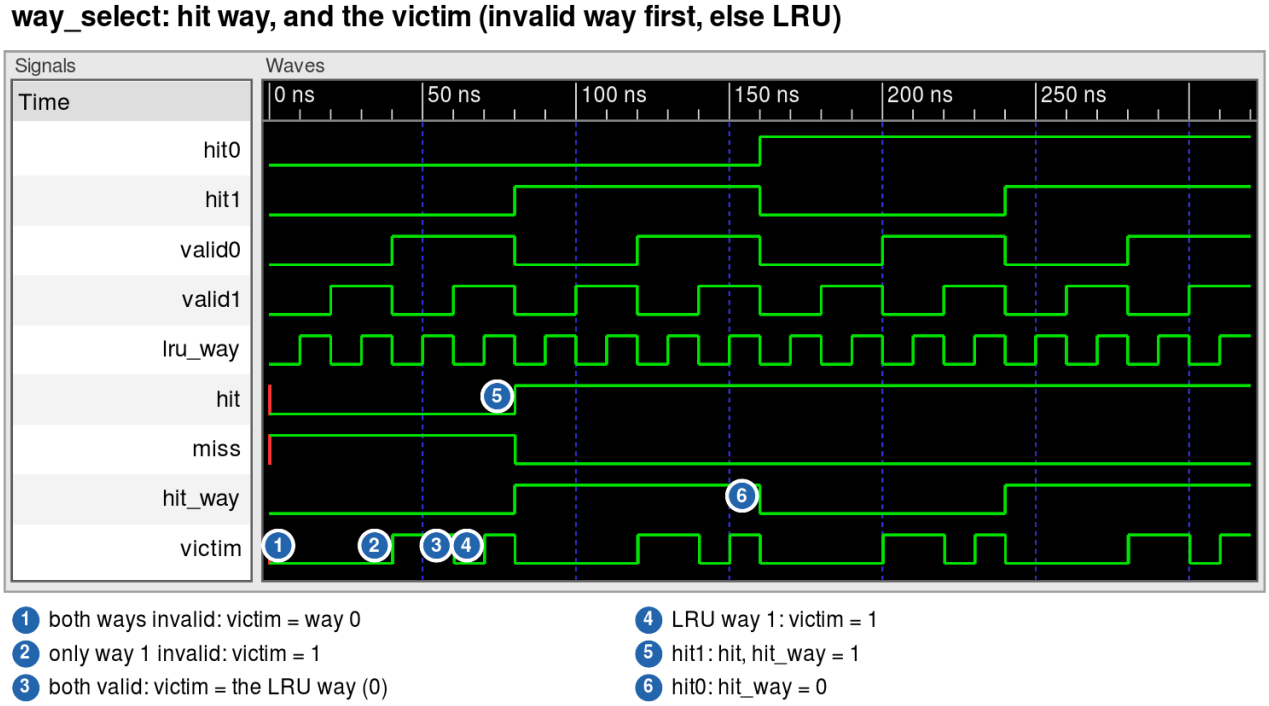


Figure 53: `tb_way_select`: simulation waveform

victim is 0 while way 0 is invalid, 1 while only way 1 is invalid, and follows lru_way when both are valid.

5.5 LRU

One LRU bit per block offset, held in negative edge registers: the bit is the way to replace next. When update = 1, the bit of the addressed block offset becomes the other way (lru <= not used_way); a decoder enabled by update loads only that bit. A 4:1 mux reads the bit of the addressed block offset. Reset clears all bits. With two ways one bit gives exact LRU.

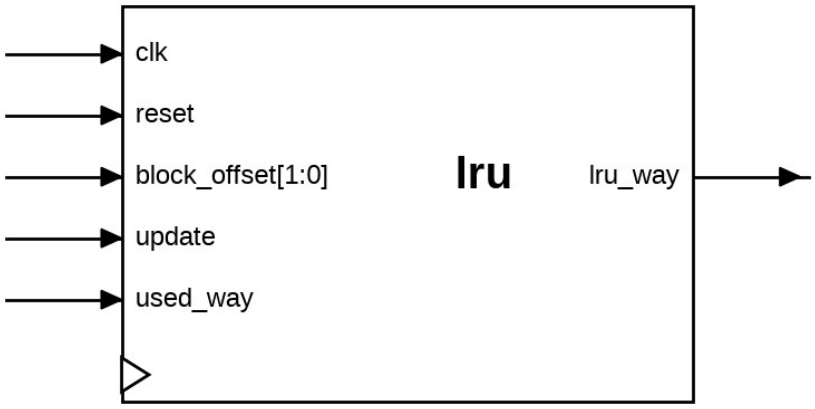


Figure 54: `lru`: inputs and outputs

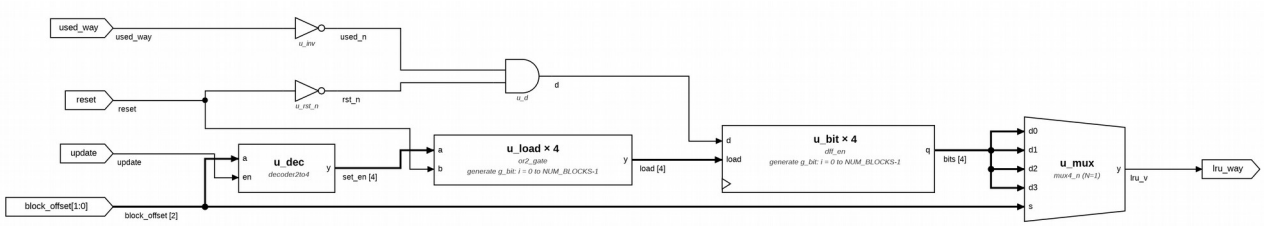


Figure 55: `lru`: internal structure

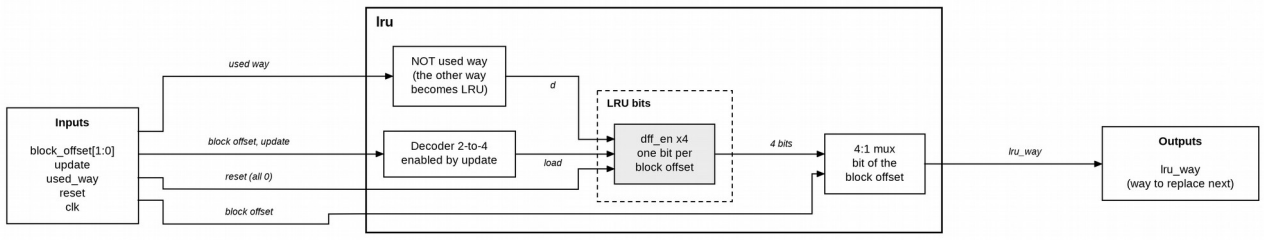


Figure 56: `lru`: block diagram

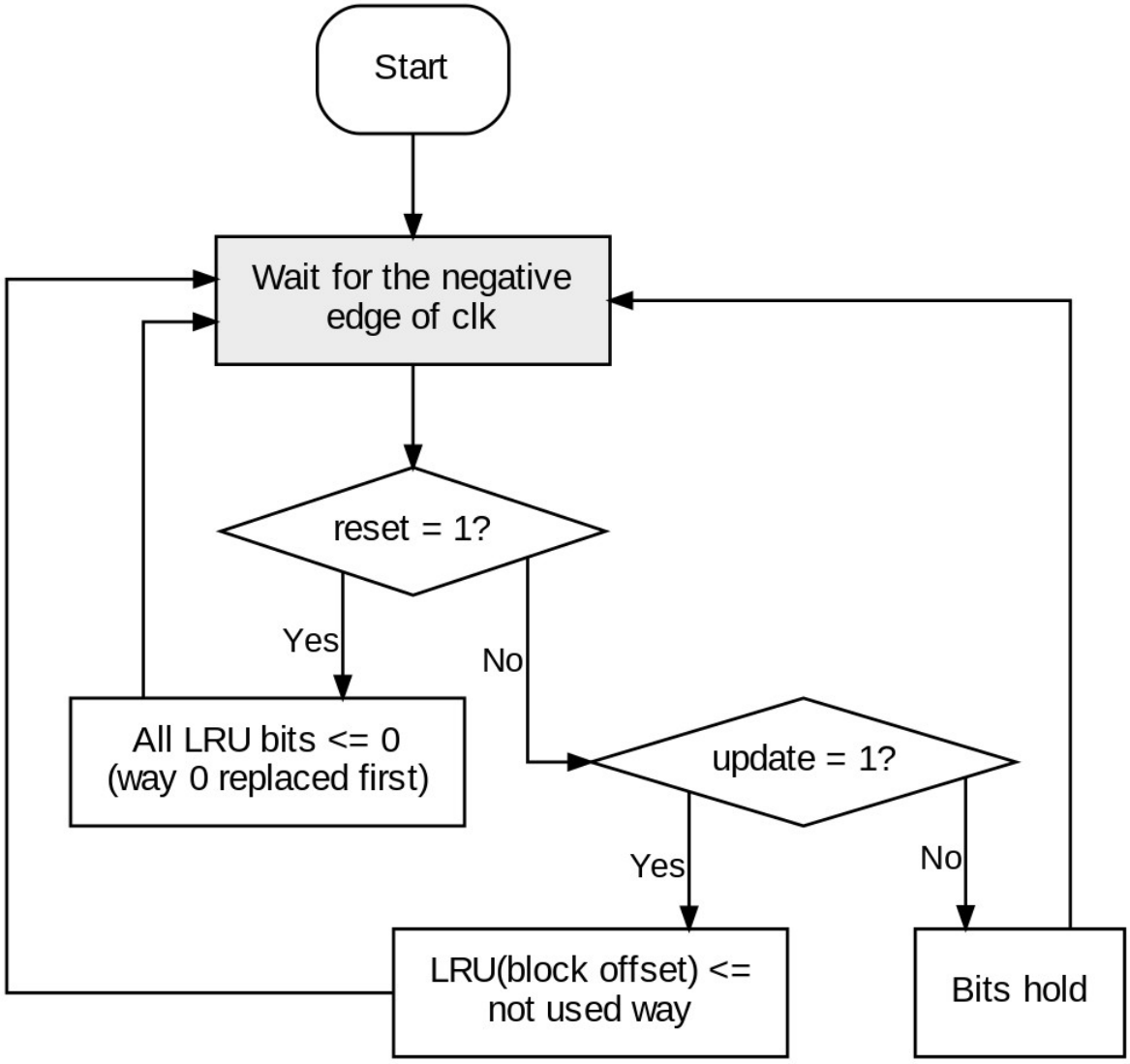


Figure 57: `lru`: operation flowchart

Code

Listing: two_way/cache/lru.vhd

```
-- LRU (least recently used) bits: one bit per block offset, lru = the way to replace next
--   On the negative edge with update = 1, the bit of the addressed block offset becomes the
--   other way:  lru(block offset) <= not used_way.  reset clears every bit (way 0 first).
--   For two ways one bit per block offset gives exact LRU.
library ieee;
use ieee.std_logic_1164.all;
use work.cache_pkg.all;

entity lru is
    port (
        clk          : in  std_logic;
        reset        : in  std_logic;
        block_offset : in  std_logic_vector(1 downto 0);
        update       : in  std_logic;   -- the addressed block was used in this cycle
        used_way     : in  std_logic;   -- the way that was used
        lru_way      : out std_logic    -- LRU way of the addressed block offset
    );
```

```
end entity lru;

architecture structural of lru is
    signal set_en, load, bits : std_logic_vector(NUM_BLOCKS-1 downto 0);
    signal used_n, rst_n, d   : std_logic;
    signal lru_v              : std_logic_vector(0 downto 0);
begin
    -- only the addressed bit is enabled, and only when update = 1
    u_dec   : entity work.decoder2to4 port map (a => block_offset, en => update, y => set_en);
    u_inv   : entity work.not_gate    port map (a => used_way, y => used_n);
    u_rst_n : entity work.not_gate    port map (a => reset, y => rst_n);
    u_d     : entity work.and2_gate   port map (a => used_n, b => rst_n, y => d);   -- reset loads 0

    g_bit : for i in 0 to NUM_BLOCKS-1 generate
        u_load : entity work.or2_gate port map (a => set_en(i), b => reset, y => load(i));
        u_bit  : entity work.dff_en   port map (d => d, load => load(i), clk => clk, q => bits(i));
    end generate g_bit;

    u_mux : entity work.mux4_n generic map (N => 1)
        port map (d0 => bits(0 downto 0), d1 => bits(1 downto 1), d2 => bits(2 downto 2),
                  d3 => bits(3 downto 3), s => block_offset, y => lru_v);
    lru_way <= lru_v(0);
end architecture structural;
```

Testbench

- Reset; a block used in way 0 then way 1; update = 0 holds; every block offset updated; reset again — every bit read back after each step.

Listing: two_way/testbench/cache/tb_lru.vhd

```
-- Testbench: lru - reset, update of the addressed bit only, update = 0 holds, all bits readable
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use work.tb_pkg.all;

entity tb_lru is
end entity tb_lru;

architecture sim of tb_lru is
    signal clk                          : std_logic := '1';
    signal reset, update, used_way, lru_way : std_logic := '0';
    signal block_offset                 : std_logic_vector(1 downto 0) := "00";
    signal done                         : boolean := false;
begin
    dut : entity work.lru
        port map (clk => clk, reset => reset, block_offset => block_offset, update => update,
                  used_way => used_way, lru_way => lru_way);

    clk_gen : process
    begin
        while not done loop
            clk <= '1'; wait for 5 ns;
            clk <= '0'; wait for 5 ns;
        end loop;
        wait;
    end process clk_gen;

    stimulus : process
        type bits_t is array (0 to 3) of std_logic;
        variable model  : bits_t := (others => '0');
        variable errors : natural := 0;
        procedure check_all (msg : string) is
        begin
            for i in 0 to 3 loop
                block_offset <= conv_std_logic_vector(i, 2);
                wait for 1 ns;
                if lru_way /= model(i) then
                    report "FAIL: " & msg & ": block " & str(i) & " lru=" & str(lru_way)
                         & " expected " & str(model(i)) severity error;
                    errors := errors + 1;
                end if;
            end loop;
        end procedure;
```

```
        -- one clock: inputs set on the positive edge, sampled on the negative edge
        procedure use_block (blk : natural; upd, way : std_logic) is
        begin
            wait until rising_edge(clk);
            block_offset <= conv_std_logic_vector(blk, 2); update <= upd; used_way <= way;
            wait until falling_edge(clk);
            wait for 1 ns;
            update <= '0';
            if upd = '1' then model(blk) := not way; end if;
        end procedure;
    begin
        wait until rising_edge(clk);
        reset <= '1';
        wait until falling_edge(clk); wait for 1 ns;
        reset <= '0';
        check_all("after reset");
        use_block(2, '1', '0');  check_all("block 2 used way 0");
        use_block(2, '1', '1');  check_all("block 2 used way 1");
        use_block(1, '0', '0');  check_all("no update");
        use_block(0, '1', '0'); use_block(1, '1', '1'); use_block(3, '1', '0');
        check_all("every block updated");
        wait until rising_edge(clk); reset <= '1';
        wait until falling_edge(clk); wait for 1 ns; reset <= '0';
        model := (others => '0');
        check_all("after the second reset");
        if errors = 0 then
            report "tb_lru: ALL TESTS PASSED";
        else
            report "tb_lru: " & str(errors) & " TEST(S) FAILED" severity error;
        end if;
        done <= true;
        wait;
    end process stimulus;
end architecture sim;
```

Simulation results

Simulation result: **PASS** — `tb_lru`: all checks passed, simulation ended at 100 ns.

lru: one bit per block offset (step bN wM: block N used way M)

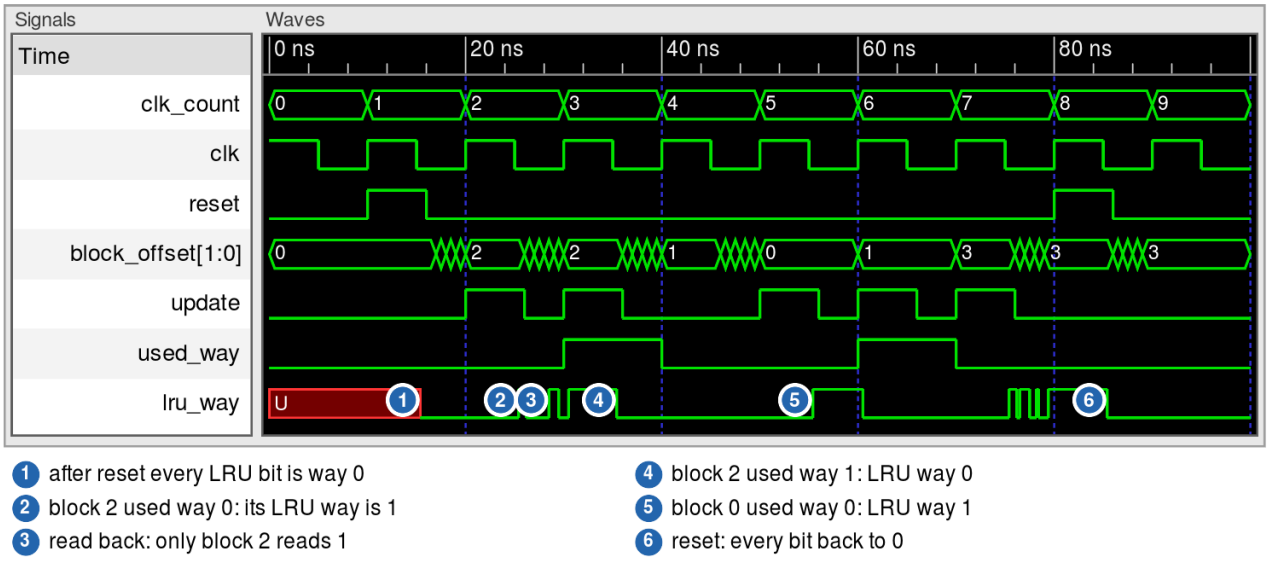


Figure 58: `tb_lru`: simulation waveform

lru_way of the addressed block offset changes only on negative edges with update = 1.

Truth tables

Every gate output of `lru` for all 32 input combinations. bits(i)+ is the value of bit i after the next negative edge of clk; – means the bit holds its value (load(i) = 0), and bi is the present value of bit i.

Table 5: `lru`: every gate output for every input (bits(3..0)+ after the next negative edge)

reset	update	block offset	used way	set_en(3..0) u_dec	used_nu_inv	rst_nu_rst_n	d u_d	load(3..0) u_load	bits(3..0)+ u_bit	lru_way u_mux
0	0	00	0	0000	1	1	1	0000	– – – –	b0
0	0	00	1	0000	0	1	0	0000	– – – –	b0
0	0	01	0	0000	1	1	1	0000	– – – –	b1
0	0	01	1	0000	0	1	0	0000	– – – –	b1
0	0	10	0	0000	1	1	1	0000	– – – –	b2
0	0	10	1	0000	0	1	0	0000	– – – –	b2
0	0	11	0	0000	1	1	1	0000	– – – –	b3
0	0	11	1	0000	0	1	0	0000	– – – –	b3
0	1	00	0	0001	1	1	1	0001	– – – 1	b0
0	1	00	1	0001	0	1	0	0001	– – – 0	b0
0	1	01	0	0010	1	1	1	0010	– – 1 –	b1
0	1	01	1	0010	0	1	0	0010	– – 0 –	b1
0	1	10	0	0100	1	1	1	0100	– 1 – –	b2
0	1	10	1	0100	0	1	0	0100	– 0 – –	b2
0	1	11	0	1000	1	1	1	1000	1 – – –	b3
0	1	11	1	1000	0	1	0	1000	0 – – –	b3
1	0	00	0	0000	1	0	0	1111	0 0 0 0	b0
1	0	00	1	0000	0	0	0	1111	0 0 0 0	b0
1	0	01	0	0000	1	0	0	1111	0 0 0 0	b1
1	0	01	1	0000	0	0	0	1111	0 0 0 0	b1
1	0	10	0	0000	1	0	0	1111	0 0 0 0	b2
1	0	10	1	0000	0	0	0	1111	0 0 0 0	b2
1	0	11	0	0000	1	0	0	1111	0 0 0 0	b3
1	0	11	1	0000	0	0	0	1111	0 0 0 0	b3
1	1	00	0	0001	1	0	0	1111	0 0 0 0	b0
1	1	00	1	0001	0	0	0	1111	0 0 0 0	b0
1	1	01	0	0010	1	0	0	1111	0 0 0 0	b1
1	1	01	1	0010	0	0	0	1111	0 0 0 0	b1
1	1	10	0	0100	1	0	0	1111	0 0 0 0	b2
1	1	10	1	0100	0	0	0	1111	0 0 0 0	b2
1	1	11	0	1000	1	0	0	1111	0 0 0 0	b3
1	1	11	1	1000	0	0	0	1111	0 0 0 0	b3

One LRU bit, `u_bit(i)` (`dff_en`: a mux2 that keeps q when load = 0, and a negative edge dff):

Table 6: `dff_en` in `lru`: one LRU bit

load(i)	d	bits(i)	mux2 output	bits(i)+
0	x	0	0	0
0	x	1	1	1
1	0	x	0	0
1	1	x	1	1

Read, `u_mux` (`mux4_n`, combinational): lru_way = bits(block offset).

Table 7: `mux4_n` in `lru`: LRU way of the addressed block offset

block offset	lru_way
00	bits(0)
01	bits(1)

10	bits(2)
11	bits(3)

With the state machine (CHECK cycle): lru_update = CHECK · (hit + RD/WR) and used_way = hit ? hit_way : victim, so the addressed bit becomes not used_way after a read hit, write hit or read miss, and does not change after a write miss.

Table 8: `lru` update for each kind of access

Access	hit	RD/WR	lru_update	used_way	Addressed bit after
Read hit in way 0	1	1	1	0 (hit_way)	1
Read hit in way 1	1	1	1	1 (hit_way)	0
Read miss, victim way 0	0	1	1	0 (victim)	1
Read miss, victim way 1	0	1	1	1 (victim)	0
Write hit in way 0	1	0	1	0 (hit_way)	1
Write hit in way 1	1	0	1	1 (hit_way)	0
Write miss	0	0	0	x	unchanged

5.6 Two-way cache (32 bytes)

The cache has a single address input, the latched CPU address, and splits it into tag (bits 5–4), block offset (3–2) and byte offset (1–0). The shared decoders select the block and the byte in both ways at once; during a read miss a mux2_n takes the byte offset from fill_byte (the counter). Each way's Hit/Miss compares the tag; way_select gives hit, hit_way and the victim. The cache's hit and miss outputs come through a Dlatch that is open only while latch_way (the CHECK cycle) or reset is 1: the state machine reads the compare at the end of CHECK, and the outputs keep that result until the next request's CHECK; reset clears them.

Replacement: on the negative edge that ends the tag compare, `latch_way` keeps the victim in the `fill_way` register (the valid and LRU bits change during the fill), and `lru_update` makes the way used (the way that hit, or the victim of a read miss) the most recently used. The write signal goes to the way that hit (write hit) or to `fill_way` (read miss). The byte read comes from the way that hit.

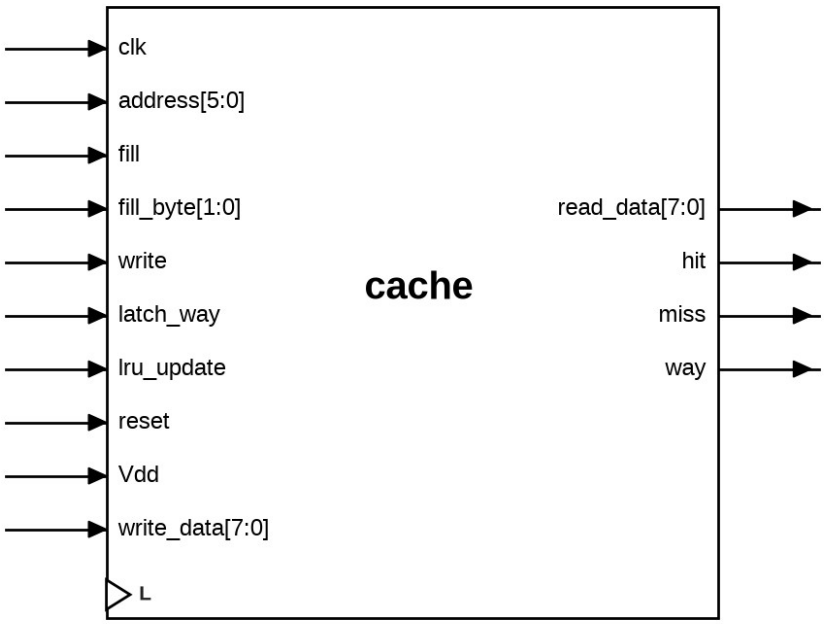


Figure 59: cache: inputs and outputs

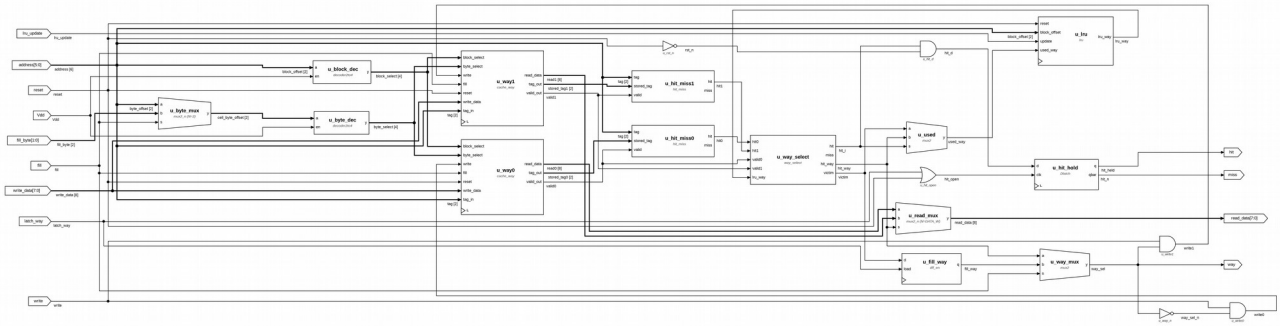


Figure 60: cache: internal structure

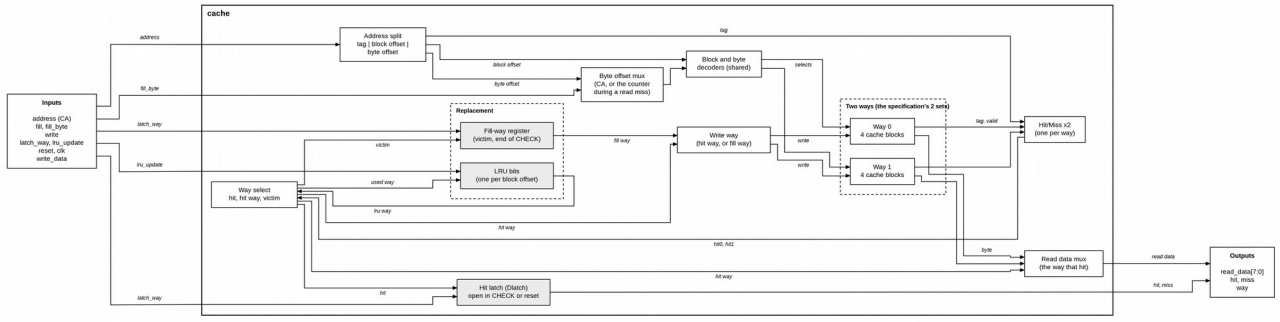


Figure 61: cache: block diagram

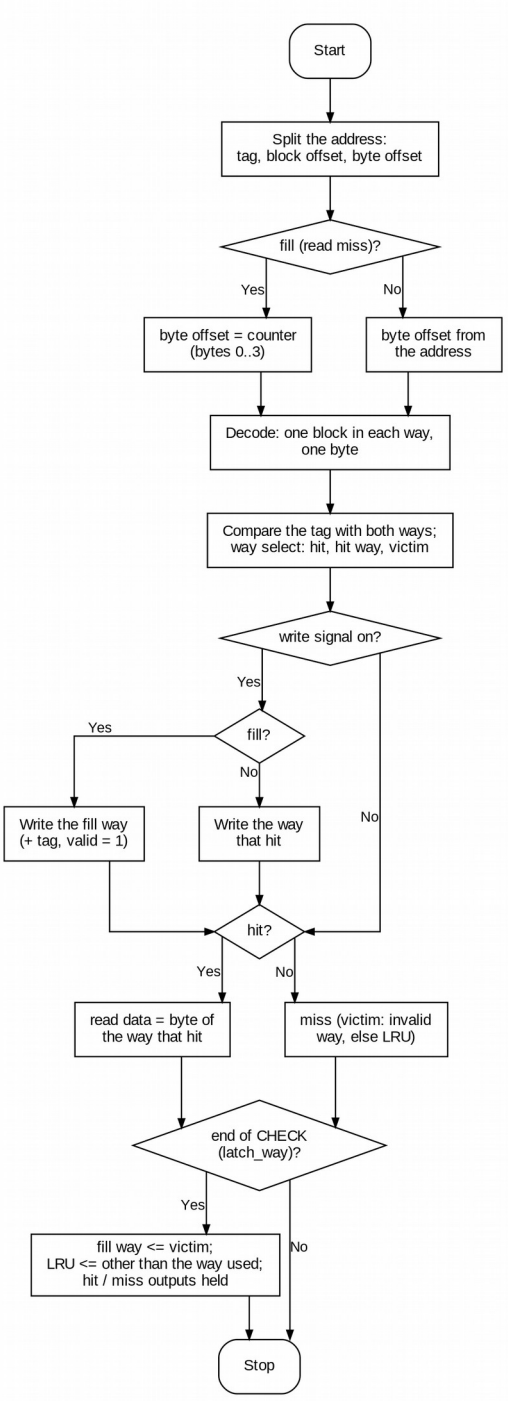


Figure 62: cache: operation flowchart

Code

Listing: two_way/cache/cache.vhd

```
-- The two-way set associative cache (32 bytes) addressed by the CPU address
--   address = | tag | block offset | byte offset |   (5..4, 3..2, 1..0)
--   The block offset selects one block in each of the 2 ways (the specification's "2 sets");
--   the tag is compared with the tags of both blocks (Hit/Miss of each way). On a hit the
--   byte is read from the way that hit. A read miss fills the victim way: an invalid way,
--   else the least recently used one. LRU: the way used by a hit or filled by a read miss
--   becomes the most recently used.
--
```

```
--   Control (from the state machine):
--   write      : write signal (one clock cycle, positive edge to positive edge). It writes the
--                way that hit (write hit) or, during a read miss (fill = 1), the victim way,
--                together with its tag and valid bit; the byte offset then comes from fill_byte.
--   latch_way  : at the end of the tag/valid compare (negative edge), keep the victim way for
--                the read miss that may follow (the valid and LRU bits change while it fills).
--   lru_update : at the same negative edge, mark the way used as most recently used.
--   reset      : clears every valid bit and the LRU bits.
--
--   hit / miss are the compare of the CHECK cycle, held until the next request's CHECK: a
--   latch passes the compare only while latch_way (CHECK) or reset is on. The compare itself
--   is X while a read miss writes the tag and valid cells (their read outputs are then 'Z').
library ieee;
use ieee.std_logic_1164.all;
use work.cache_pkg.all;

entity cache is
    port (
        clk        : in  std_logic;
        address    : in  std_logic_vector(ADDR_W-1 downto 0);   -- latched CPU address (CA)
        fill       : in  std_logic;   -- read miss: the block comes from memory
        fill_byte  : in  std_logic_vector(1 downto 0);          -- byte of the block being written
        write      : in  std_logic;   -- the write signal
        latch_way  : in  std_logic;
        lru_update : in  std_logic;
        reset      : in  std_logic;
        Vdd        : in  std_logic;
        write_data : in  std_logic_vector(DATA_W-1 downto 0);
        read_data  : out std_logic_vector(DATA_W-1 downto 0);   -- the addressed byte
        hit        : out std_logic;
        miss       : out std_logic;
        way        : out std_logic    -- the way being written / read (for observation)
    );
end entity cache;

architecture structural of cache is
    -- fields of the address
    signal tag, block_offset, byte_offset : std_logic_vector(1 downto 0);
    signal cell_byte_offset               : std_logic_vector(1 downto 0);   -- byte actually selected
    signal block_select, byte_select      : std_logic_vector(3 downto 0);

    -- the two ways
    signal read0, read1                   : std_logic_vector(DATA_W-1 downto 0);
    signal stored_tag0, stored_tag1       : std_logic_vector(TAG_W-1 downto 0);
    signal valid0, valid1, hit0, hit1     : std_logic;
    signal miss0, miss1                   : std_logic;

    -- way selection and replacement
    signal hit_i, hit_way, victim, lru_way, used_way, fill_way : std_logic;
    signal way_sel, way_sel_n, write0, write1              : std_logic;
    signal miss_i, rst_n, hit_d, hit_open, hit_held, hit_n   : std_logic;
begin
    -- ---------------- Address: | tag | block offset | byte offset | ----------------
    tag          <= address(5 downto 4);
    block_offset <= address(3 downto 2);
    byte_offset  <= address(1 downto 0);

    -- read miss: write bytes 0..3 of the block in turn; otherwise the CPU's byte offset
    u_byte_mux : entity work.mux2_n generic map (N => 2)
        port map (a => byte_offset, b => fill_byte, s => fill, y => cell_byte_offset);

    -- ---------------- Decoders (shared by both ways) ----------------
    u_block_dec : entity work.decoder2to4 port map (a => block_offset,     en => Vdd, y => block_select);
    u_byte_dec  : entity work.decoder2to4 port map (a => cell_byte_offset, en => Vdd, y => byte_select);

    -- ---------------- Write signal of each way ----------------
    -- write hit: the way that hit; read miss: the victim kept in fill_way
    u_way_mux : entity work.mux2 port map (a => hit_way, b => fill_way, s => fill, y => way_sel);
    u_way_n   : entity work.not_gate  port map (a => way_sel, y => way_sel_n);
    u_write0  : entity work.and2_gate port map (a => write, b => way_sel_n, y => write0);
    u_write1  : entity work.and2_gate port map (a => write, b => way_sel,   y => write1);

    -- ---------------- The two ways ----------------
    u_way0 : entity work.cache_way
        port map (block_select => block_select, byte_select => byte_select, write => write0, fill => fill,
```

```
                reset => reset, write_data => write_data, tag_in => tag,
                read_data => read0, tag_out => stored_tag0, valid_out => valid0);
    u_way1 : entity work.cache_way
        port map (block_select => block_select, byte_select => byte_select, write => write1, fill => fill,
                reset => reset, write_data => write_data, tag_in => tag,
                read_data => read1, tag_out => stored_tag1, valid_out => valid1);

    -- ---------------- Hit/Miss of each way, way selection ----------------
    u_hit_miss0 : entity work.hit_miss
        port map (tag => tag, stored_tag => stored_tag0, valid => valid0, hit => hit0, miss => miss0);
    u_hit_miss1 : entity work.hit_miss
        port map (tag => tag, stored_tag => stored_tag1, valid => valid1, hit => hit1, miss => miss1);
    u_way_select : entity work.way_select
        port map (hit0 => hit0, hit1 => hit1, valid0 => valid0, valid1 => valid1, lru_way => lru_way,
                hit => hit_i, miss => miss_i, hit_way => hit_way, victim => victim);

    -- ---------------- Replacement: LRU bits and the way being filled ----------------
    -- the way used by this request: the way that hit, or the victim of a read miss
    u_used : entity work.mux2 port map (a => victim, b => hit_way, s => hit_i, y => used_way);
    u_lru  : entity work.lru
        port map (clk => clk, reset => reset, block_offset => block_offset, update => lru_update,
                used_way => used_way, lru_way => lru_way);
    u_fill_way : entity work.dff_en port map (d => victim, load => latch_way, clk => clk, q => fill_way);

    -- ---------------- Read data: the byte from the way that hit ----------------
    u_read_mux : entity work.mux2_n generic map (N => DATA_W)
        port map (a => read0, b => read1, s => hit_way, y => read_data);

    -- ---------------- hit / miss, held from the end of CHECK; reset clears them ----------------
    u_rst_n    : entity work.not_gate  port map (a => reset, y => rst_n);
    u_hit_d    : entity work.and2_gate port map (a => hit_i, b => rst_n, y => hit_d);
    u_hit_open : entity work.or2_gate  port map (a => latch_way, b => reset, y => hit_open);
    u_hit_hold : entity work.Dlatch    port map (d => hit_d, clk => hit_open, q => hit_held, qbar =>
hit_n);

    hit  <= hit_held;
    miss <= hit_n;
    way  <= way_sel;
end architecture structural;
```

Testbench

- Reset; A (tag 0, block 1) read miss fills way 0; B (tag 1, same block) fills way 1.
- Both A and B hit, each read from its own way.
- C (tag 2) read miss replaces the LRU way (B); B then misses (a write miss, which does not change the LRU); A and C hit.
- A write hit writes only the way that hit; another block offset misses; reset empties the cache.

Listing: two_way/testbench/cache/tb_cache.vhd

```
-- Testbench: two-way cache - driven as the state machine drives it (inputs on the positive edge,
-- sampled on the negative edge, write signal from positive edge to positive edge):
--   two tags in the same block offset live in the two ways; reads come from the way that hit;
--   the victim is the invalid way, then the LRU way; a write miss does not change the LRU;
--   a write hit writes only the way that hit; reset invalidates everything.
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use work.tb_pkg.all;

entity tb_cache is
end entity tb_cache;

architecture sim of tb_cache is
    signal clk                      : std_logic := '1';
    signal address                  : std_logic_vector(5 downto 0) := (others => '0');
    signal fill, write, reset       : std_logic := '0';
    signal latch_way, lru_update    : std_logic := '0';
    signal fill_byte                : std_logic_vector(1 downto 0) := "00";
    signal Vdd                      : std_logic := '1';
    signal write_data, read_data    : std_logic_vector(7 downto 0) := (others => '0');
```

```
    signal hit, miss, way             : std_logic;
    signal done                       : boolean := false;
begin
    dut : entity work.cache
        port map (clk => clk, address => address, fill => fill, fill_byte => fill_byte, write => write,
                  latch_way => latch_way, lru_update => lru_update, reset => reset, Vdd => Vdd,
                  write_data => write_data, read_data => read_data, hit => hit, miss => miss, way => way);

    clk_gen : process
    begin
        while not done loop
            clk <= '1'; wait for 5 ns;
            clk <= '0'; wait for 5 ns;
        end loop;
        wait;
    end process clk_gen;

    stimulus : process
        variable errors : natural := 0;
        variable got_hit : std_logic;
        procedure check (cond : boolean; msg : string) is
        begin
            if not cond then
                report "FAIL: " & msg severity error;
                errors := errors + 1;
            end if;
        end procedure;
        function addr (tag, blk, byt : natural) return std_logic_vector is
        begin
            return conv_std_logic_vector(tag * 16 + blk * 4 + byt, 6);
        end function;
        -- one CHECK cycle: address and latch_way (the CHECK state: the cache passes its compare
        -- to hit) on a positive edge, and on the negative edge the victim kept and lru_update
        -- (if hit, or a read)
        procedure access_block (tag, blk, byt : natural; rd : boolean) is
        begin
            wait until rising_edge(clk);
            address <= addr(tag, blk, byt);
            latch_way <= '1';
            wait for 2 ns;
            got_hit := hit;
            if hit = '1' or rd then lru_update <= '1'; else lru_update <= '0'; end if;
            wait until falling_edge(clk);
            wait for 1 ns;
            latch_way <= '0'; lru_update <= '0';
        end procedure;
        -- read miss: the 4 bytes from memory into the victim way, one write pulse each
        procedure fill_block (base : natural; expect_way : std_logic; msg : string) is
        begin
            fill <= '1';
            wait for 1 ns;
            check(way = expect_way, msg & ": filling way " & str(way) & ", expected way " &
str(expect_way));
            for i in 0 to 3 loop
                fill_byte <= conv_std_logic_vector(i, 2); write_data <= conv_std_logic_vector(base + i,
8);
                wait until rising_edge(clk); write <= '1';
                wait until rising_edge(clk); write <= '0';
                wait until falling_edge(clk);
            end loop;
            fill <= '0';
        end procedure;
        procedure expect_hit (tag, blk, byt : natural; data : natural; w : std_logic; msg : string) is
        begin
            access_block(tag, blk, byt, true);
            check(got_hit = '1', msg & ": miss, expected a hit");
            check(way = w, msg & ": way " & str(way) & ", expected way " & str(w));
            check(read_data = conv_std_logic_vector(data, 8), msg & ": data " & str(read_data));
        end procedure;
    begin
        -- 1. reset
        wait until rising_edge(clk); reset <= '1';
        wait until rising_edge(clk); reset <= '0';

        -- 2. A (tag 0, block 1): read miss, victim = way 0 (both invalid)
```

```
        access_block(0, 1, 0, true);
        check(got_hit = '0', "A misses after reset");
        fill_block(16#10#, '0', "A");
        -- 3. B (tag 1, block 1): read miss, victim = way 1 (still invalid)
        access_block(1, 1, 0, true);
        check(got_hit = '0', "B misses");
        fill_block(16#20#, '1', "B");

        -- 4. both blocks are in the cache, each in its own way
        expect_hit(0, 1, 2, 16#12#, '0', "A byte 2");
        expect_hit(1, 1, 3, 16#23#, '1', "B byte 3");
        expect_hit(0, 1, 0, 16#10#, '0', "A byte 0");            -- A used last: B is LRU

        -- 5. C (tag 2, block 1): read miss, the LRU way (B, way 1) is replaced
        access_block(2, 1, 0, true);
        check(got_hit = '0', "C misses");
        fill_block(16#30#, '1', "C replaces the LRU way (B)");

        -- 6. B was evicted; a write miss does not change the LRU
        access_block(1, 1, 0, false);
        check(got_hit = '0', "B misses after the replacement (write miss)");
        expect_hit(0, 1, 1, 16#11#, '0', "A still in way 0");
        expect_hit(2, 1, 2, 16#32#, '1', "C in way 1");

        -- 7. write hit to C byte 2 (way 1 only)
        access_block(2, 1, 2, false);
        check(got_hit = '1', "C write hit");
        write_data <= "10101010";
        wait until rising_edge(clk); write <= '1';
        wait until rising_edge(clk); write <= '0';
        expect_hit(2, 1, 2, 16#AA#, '1', "C byte 2 written");
        expect_hit(0, 1, 2, 16#12#, '0', "A byte 2 unchanged (other way)");

        -- 8. another block offset is still empty; reset empties everything
        access_block(0, 3, 0, false);
        check(got_hit = '0', "block 3 misses");
        wait until rising_edge(clk); reset <= '1';
        wait until rising_edge(clk); reset <= '0';
        access_block(0, 1, 0, false);
        check(got_hit = '0', "A misses after the second reset");

        if errors = 0 then
            report "tb_cache: ALL TESTS PASSED";
        else
            report "tb_cache: " & str(errors) & " TEST(S) FAILED" severity error;
        end if;
        done <= true;
        wait;
    end process stimulus;
end architecture sim;
```

Simulation results

Simulation result: PASS — `tb_cache`: all checks passed, simulation ended at 450 ns.

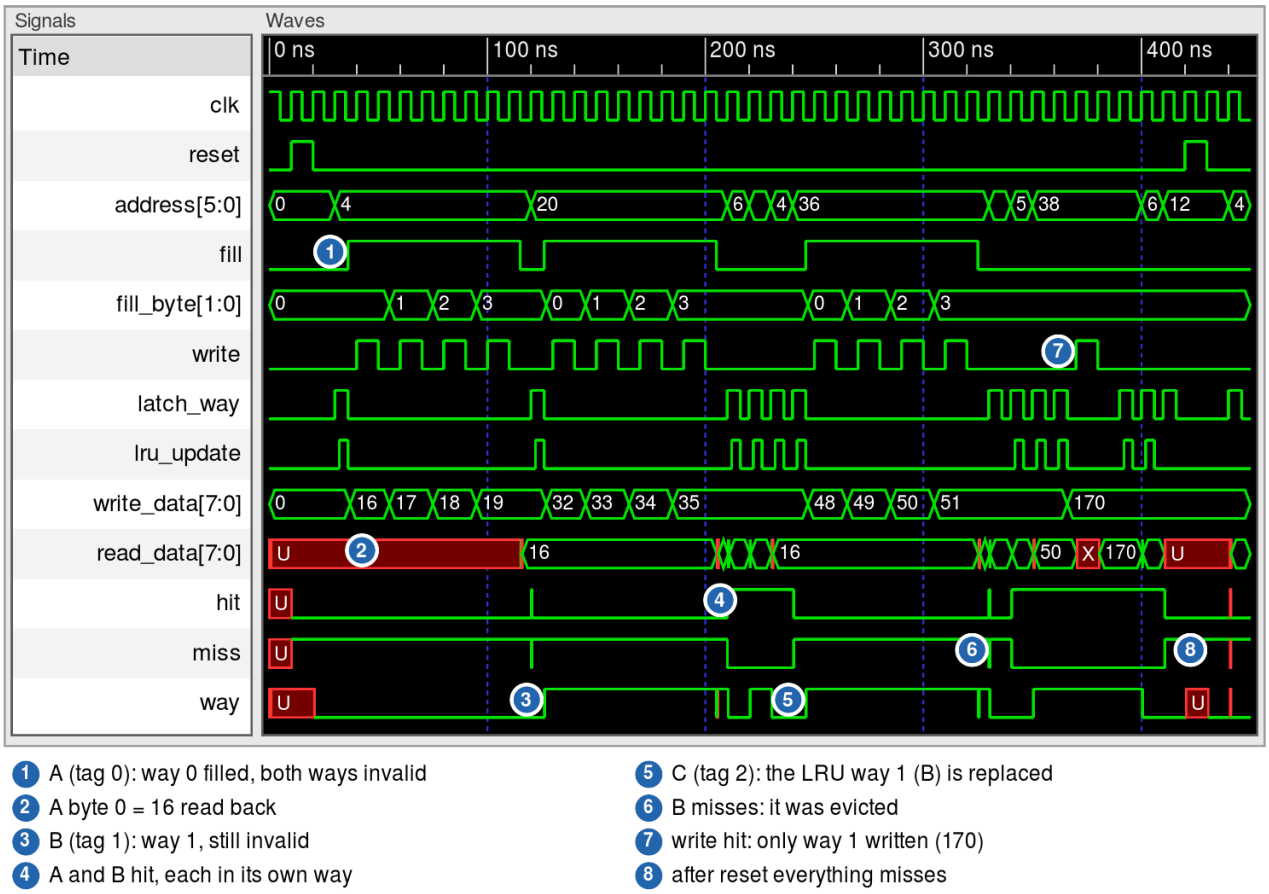


Figure 63: `tb_cache`: simulation waveform

The testbench drives latch_way and lru_update as the state machine does. way shows the way being read or written: 0 for A, 1 for B, and 1 again when C replaces B. latch_way is raised with the address, as in the CHECK state, because the cache passes its compare to hit only while latch_way = 1.

6 State machine

The state machine is one-hot: five negative edge flip-flops, one per state, with the next-state logic built from gates. The counter gives the clock cycle c of the request: c = 1 after the negative edge that accepts the request (CHECK); it counts in CHECK, WRITE and READ_MISS, and after the edge where busy goes low it holds 2 (read hit), 3 (write) or 19 (read miss). It is cleared on the next negative edge, or started at 1 again by a new request.

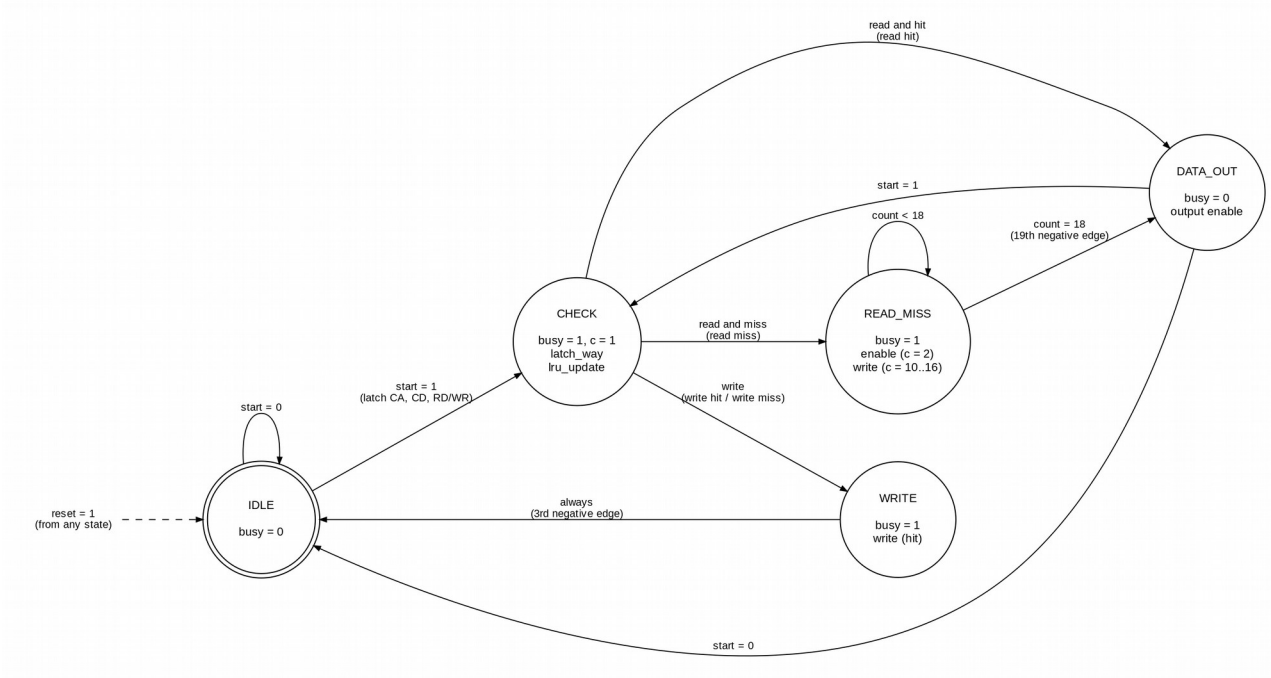


Figure 64: State machine diagram

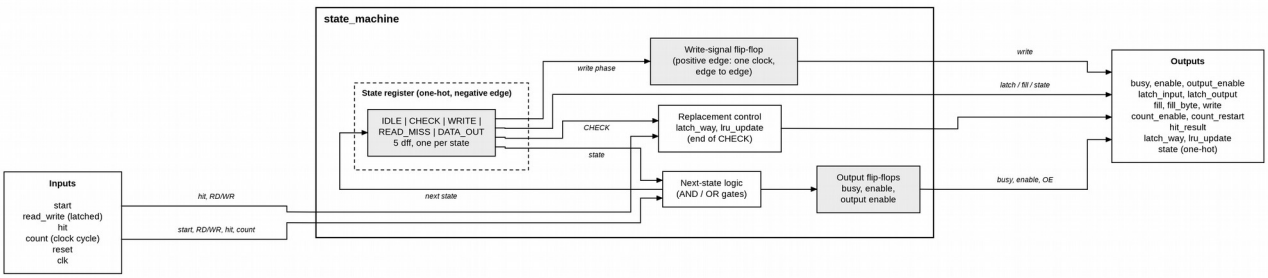


Figure 65: `state_machine`: block diagram

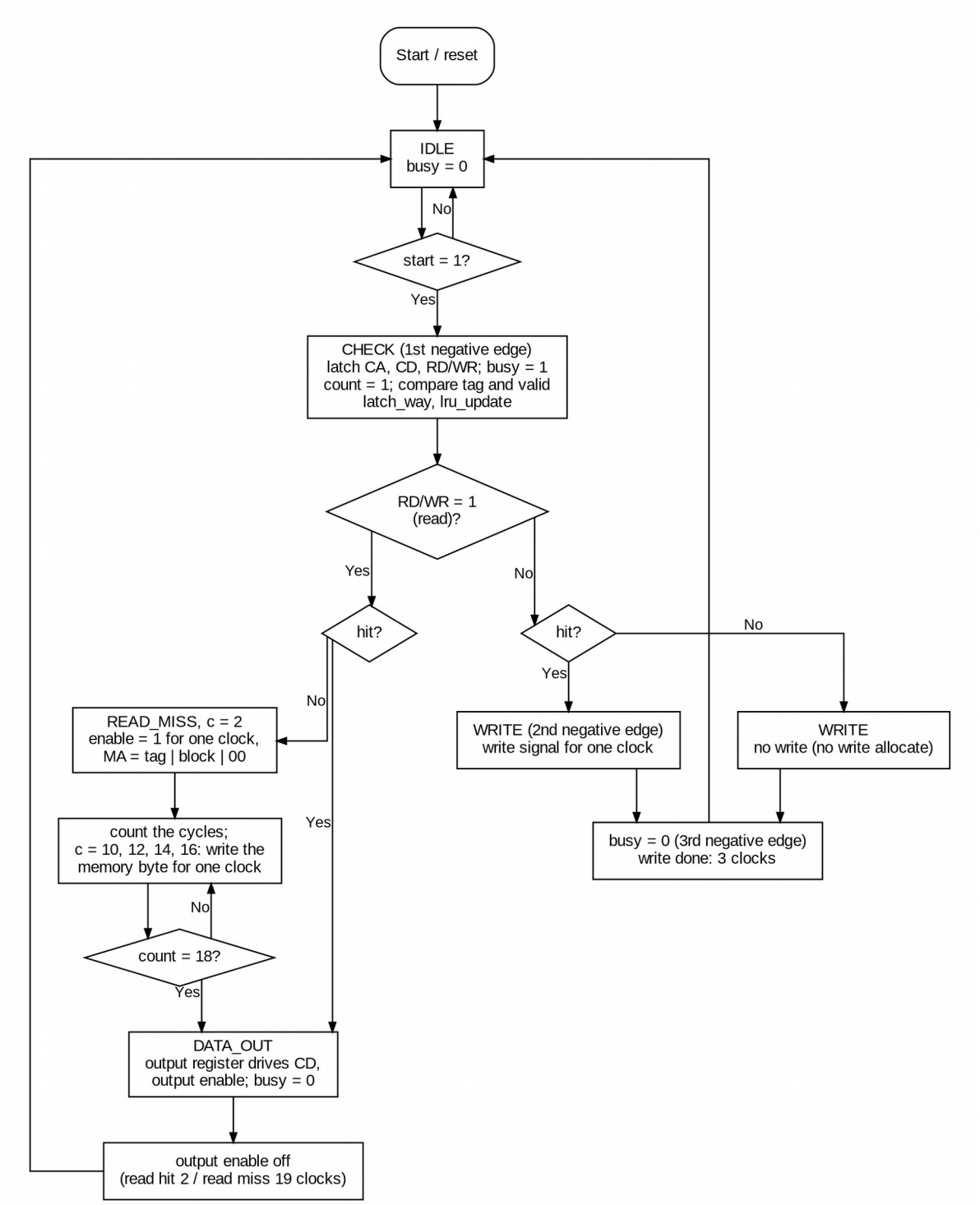


Figure 66: `state_machine`: operation flowchart

6.1 States

Table 9: States of the state machine

State	What happens	Why it is necessary
IDLE	Waits for start. busy = 0.	Reset state; the chip must not react until a request arrives.

CHECK	Entered on the negative edge after start: CA, CD and RD/WR are latched, busy goes high. During this cycle the addressed block's tag and valid bit and the addressed byte are read and compared (hit/miss) in both ways. On the negative edge that ends CHECK, latch_way keeps the victim way and lru_update marks the way used as most recently used.	The inputs are removed by the CPU on the next positive edge, so they must be latched; the hit/ miss decision needs one cycle.
WRITE	Write hit: the write signal writes CD into the addressed byte (positive edge to positive edge). Write miss: nothing is written (no write allocate). busy stays high.	The specification requires 3 clocks for a write hit or miss, with the write on the second negative edge.
READ_MISS	c = 2: enable = 1 with MA = tag, block offset, 00. c = 10, 12, 14, 16: the memory byte is valid, the write signal writes it (bytes 0–3; tag and valid with them). c = 18: the requested byte is read from the cache. The ways hold their tag, valid and read byte while the block is filled.	The memory needs 8 cycles and delivers one byte every 2 cycles; the block must be in the cache before the byte is returned.
DATA_OUT	The output register drives CD (output enable) for one clock; busy = 0. A new start can be accepted here.	The CPU reads the data on the positive edge after output enable; it must stay stable for one cycle.

Next-state equations (· = AND, + = OR):

- CHECK′ = start · (IDLE + DATA_OUT) · not reset
- WRITE′ = CHECK · not RD/WR · not reset
- READ_MISS′ = (CHECK · RD/WR · miss + READ_MISS · not c18) · not reset (c18 = count(4) · count(1))
- DATA_OUT′ = (CHECK · RD/WR · hit + READ_MISS · c18) · not reset
- IDLE′ = reset + (IDLE + DATA_OUT) · not start · not reset + WRITE
- latch_way = CHECK; lru_update = CHECK · (hit + RD/WR) (no LRU change on a write miss)
- count_enable = (CHECK + WRITE + READ_MISS) · not reset; count_restart = CHECK′ (the counter starts at 1 with every request)
- hit_result = the hit of the CHECK cycle (dff_en loaded in CHECK or reset), held until the next request's CHECK

6.2 Timing

Table 10: Timing of each operation

Negative edge after start	Read hit	Write hit / write miss	Read miss
1st	latch inputs, busy = 1 (CHECK)	latch inputs, busy = 1 (CHECK)	latch inputs, busy = 1 (CHECK)
2nd	data → output register, output enable, busy = 0	WRITE; write signal on next positive edge (hit only)	enable = 1, MA driven (READ_MISS, c = 2)
3rd	output enable off	busy = 0 (IDLE)	enable = 0, MA high impedance
10th, 12th, 14th, 16th			memory byte 0, 1, 2, 3 valid; written from the next positive edge for one cycle
18th			requested byte read from the cache (c = 18)
19th			data → output register, output enable, busy = 0
20th			output enable off

6.3 Inputs, outputs and connections

Table 11: State machine inputs and outputs

Signal	Dir	Connected to	Function
clk, reset	in	chip ports	Clock (negative edge) and synchronous reset
start	in	chip port	Request from the CPU
read_write	in	dff_en (RD/WR register)	Latched RD/WR of the request
hit	in	cache (its Hit/Miss logic)	Hit of the latched address (the cache holds it from the end of CHECK)
count	in	counter	Clock cycle of the request (1 in CHECK)
busy	out	chip port busy	Handshake to the CPU
enable	out	chip port mem_en; tx gates of MA	Memory read request; drives MA
output_enable	out	tx gates of CD	Drives the output register onto CD for one cycle
latch_input	out	register_n (CA, CD), dff_en (RD/WR)	Latch the CPU request
latch_output	out	register_n (output register)	Latch the byte read from the cache
count_enable	out	counter	Count in CHECK, WRITE and READ_MISS, clear otherwise
count_restart	out	counter	Start the count at 1

			when a request is accepted
fill	out	cache; mux2_n (write data)	Read miss: byte offset from the counter, data from MD, write tag and valid
fill_byte	out	cache (byte offset mux)	Read miss: byte of the block being written (from the counter)
hit_result	out	chip (hit, miss)	Hit/miss of the request, kept from the end of CHECK
write	out	cache	Write signal (one clock, positive edge to positive edge)
latch_way	out	cache (fill_way register)	Keep the victim way for a read miss
lru_update	out	cache (lru)	Mark the way used as most recently used
state	out	(observation)	One-hot state, for the waveforms

6.4 Code

Listing: two_way/control/state_machine.vhd

```
-- State machine of the cache chip (one-hot, negative edge dffs, next-state logic from gates)
--
--   IDLE      : wait for start.                                      start          -> CHECK
--   CHECK     : CPU signals latched, busy on; tag/valid compare.  read  and hit  -> DATA_OUT   (read hit)
--                                                                    read  and miss -> READ_MISS  (read
miss)
--                                                                    write          -> WRITE      (write
hit/miss)
--   WRITE     : write hit: the write signal is on for one cycle;  always         -> IDLE  (busy off)
--               write miss: nothing is written (no write allocate)
--   READ_MISS : memory enable for one cycle, wait for the memory, write the 4 bytes from memory,
--               then read the requested byte.
--   DATA_OUT  : output enable on for one cycle, busy off.         start          -> CHECK, else IDLE
--
--   The counter gives the clock cycle c of the request: c = 1 after the first negative edge
--   (CHECK), and after the negative edge where busy goes low c = 2 (read hit), 3 (write hit/miss)
--   or 19 (read miss). It is cleared on the next negative edge, or restarted at 1 by a new request.
--   Read miss:  c = 2              enable on (memory address on MA)
--               c = 10,12,14,16    a byte from memory is valid: write signal on from the next
--                                  positive edge for one cycle (bytes 0,1,2,3 of the block)
--               c = 18             read the requested byte              c = 18 -> DATA_OUT
--   hit_result: the hit/miss of the request, kept from the end of CHECK until the next request's
--               CHECK (the tag/valid compare itself changes while a read miss writes the block).
--
--   Read hit: 2 clocks, write hit/miss: 3 clocks, read miss: 19 clocks (from start to the end).
--   Two-way cache: at the end of CHECK, latch_way keeps the victim way and lru_update marks the
--   way used (read hit, write hit, read miss) as the most recently used.
library ieee;
use ieee.std_logic_1164.all;
use work.cache_pkg.all;

entity state_machine is
    port (
        clk           : in  std_logic;
        reset         : in  std_logic;
        start         : in  std_logic;
        read_write    : in  std_logic;   -- latched RD/WR: 1 = read
        hit           : in  std_logic;
        count         : in  std_logic_vector(COUNT_W-1 downto 0);
        busy          : out std_logic;
```

```
        enable        : out std_logic;   -- memory enable
        output_enable : out std_logic;
        latch_input   : out std_logic;   -- load the CPU address / data / RD/WR registers
        latch_output  : out std_logic;   -- load the output register from the cache
        count_enable  : out std_logic;   -- run the cycle counter (else clear it)
        count_restart : out std_logic;   -- start the cycle counter at 1 (new request)
        fill_byte     : out std_logic_vector(1 downto 0);   -- read miss: byte of the block being written
        hit_result    : out std_logic;   -- hit (1) or miss (0) of the request, from the end of CHECK
        fill          : out std_logic;   -- read miss: write bytes from memory (MD) into the block
        write         : out std_logic;   -- cache write signal (positive edge to positive edge)
        latch_way     : out std_logic;   -- two-way cache: keep the victim way (end of CHECK)
        lru_update    : out std_logic;   -- two-way cache: the way used becomes most recently used
        state         : out std_logic_vector(4 downto 0)   -- one-hot: DATA_OUT READ_MISS WRITE CHECK IDLE
    );
end entity state_machine;

architecture structural of state_machine is
    signal s_idle, s_check, s_write, s_miss, s_out : std_logic;          -- state flip-flops
    signal n_idle, n_check, n_write, n_miss, n_out : std_logic;          -- next state
    signal q_n                                     : std_logic_vector(7 downto 0);
    signal rst_n, rd_n, miss, c0_n, c1_n, c41_n     : std_logic;
    signal waiting, accept, rd_ok, rd_hit, rd_miss, wr_req : std_logic;
    signal miss_on, miss_done, idle_keep, idle_set  : std_logic;
    signal n_miss_i, n_out_i                        : std_logic;
    signal busy_d, enable_d, b1                     : std_logic;
    signal hit_r, wr_hit, byte_valid, wphase_a, wphase : std_logic;
    signal hit_d, hit_load, busy_st, c21, c321, c16    : std_logic;
    signal byte_c                                      : std_logic;
    signal used                                    : std_logic;
begin
    u_rst_n : entity work.not_gate port map (a => reset,      y => rst_n);
    u_rd_n  : entity work.not_gate port map (a => read_write, y => rd_n);
    u_miss  : entity work.not_gate port map (a => hit,        y => miss);
    u_c0_n  : entity work.not_gate port map (a => count(0),   y => c0_n);
    u_c1_n  : entity work.not_gate port map (a => count(1),   y => c1_n);

    -- ---------------- Next-state logic ----------------
    -- a request is accepted in IDLE or DATA_OUT
    u_wait   : entity work.or2_gate  port map (a => s_idle,  b => s_out,  y => waiting);
    u_accept : entity work.and2_gate port map (a => start,   b => waiting, y => accept);
    -- CHECK -> DATA_OUT / READ_MISS / WRITE
    u_rd_ok  : entity work.and2_gate port map (a => s_check, b => read_write, y => rd_ok);
    u_rd_hit : entity work.and2_gate port map (a => rd_ok,   b => hit,  y => rd_hit);
    u_rd_ms  : entity work.and2_gate port map (a => rd_ok,   b => miss, y => rd_miss);
    u_wr_req : entity work.and2_gate port map (a => s_check, b => rd_n, y => wr_req);
    -- READ_MISS until the counter reaches 18 (10010: the only c in 2..18 with bits 4 and 1 set)
    u_c41_n  : entity work.nand2_gate port map (a => count(4), b => count(1), y => c41_n);
    u_ms_on  : entity work.and2_gate port map (a => s_miss,  b => c41_n,  y => miss_on);
    u_ms_dn  : entity work.and3_gate port map (a => s_miss,  b => count(4), c => count(1), y =>
miss_done);

    u_n_chk  : entity work.and2_gate port map (a => accept,  b => rst_n, y => n_check);
    u_n_wr   : entity work.and2_gate port map (a => wr_req,  b => rst_n, y => n_write);
    u_n_ms_i : entity work.or2_gate  port map (a => rd_miss, b => miss_on,   y => n_miss_i);
    u_n_ms   : entity work.and2_gate port map (a => n_miss_i, b => rst_n, y => n_miss);
    u_n_out_i: entity work.or2_gate  port map (a => rd_hit,  b => miss_done, y => n_out_i);
    u_n_out  : entity work.and2_gate port map (a => n_out_i, b => rst_n, y => n_out);
    -- IDLE: reset, nothing to do in IDLE / DATA_OUT, or the end of a write
    u_keep   : entity work.nor2_gate port map (a => accept, b => reset, y => idle_keep);
    u_kept   : entity work.and2_gate port map (a => waiting, b => idle_keep, y => idle_set);
    u_n_idle : entity work.or3_gate  port map (a => reset, b => idle_set, c => s_write, y => n_idle);

    -- ---------------- State flip-flops ----------------
    u_s_idle  : entity work.dff port map (d => n_idle,  clk => clk, q => s_idle,  qbar => q_n(0));
    u_s_check : entity work.dff port map (d => n_check, clk => clk, q => s_check, qbar => q_n(1));
    u_s_write : entity work.dff port map (d => n_write, clk => clk, q => s_write, qbar => q_n(2));
    u_s_miss  : entity work.dff port map (d => n_miss,  clk => clk, q => s_miss,  qbar => q_n(3));
    u_s_out   : entity work.dff port map (d => n_out,   clk => clk, q => s_out,   qbar => q_n(4));

    -- ---------------- Registered outputs (change on the negative edge) ----------------
    -- busy: on in CHECK, WRITE and READ_MISS
    u_busy_a : entity work.or2_gate  port map (a => n_check, b => n_write, y => b1);
    u_busy_b : entity work.or2_gate  port map (a => b1,      b => n_miss,  y => busy_d);
    u_busy   : entity work.dff port map (d => busy_d, clk => clk, q => busy, qbar => q_n(5));
    -- memory enable: one cycle, on the 2nd negative edge of a read miss
```

```
    u_en_d   : entity work.and2_gate port map (a => rd_miss, b => rst_n, y => enable_d);
    u_en     : entity work.dff port map (d => enable_d, clk => clk, q => enable, qbar => q_n(6));
    -- output enable: one cycle in DATA_OUT
    u_oe     : entity work.dff port map (d => n_out, clk => clk, q => output_enable, qbar => q_n(7));
    -- hit of the CHECK cycle, kept for the write and shown as hit_result until the next CHECK;
    -- reset clears it
    u_hit_d  : entity work.and2_gate port map (a => hit,     b => rst_n, y => hit_d);
    u_hit_ld : entity work.or2_gate  port map (a => s_check, b => reset, y => hit_load);
    u_hit_r  : entity work.dff_en port map (d => hit_d, load => hit_load, clk => clk, q => hit_r);

    -- ---------------- Write signal ----------------
    -- write hit: in WRITE if the CHECK cycle hit;  read miss: c = 10, 12, 14 (01010, 01100, 01110:
    -- bit 3 and bit 2 or 1) or 16 (10000: bit 4 without bit 1), always with bit 0 = 0
    u_wr_hit : entity work.and2_gate port map (a => s_write, b => hit_r, y => wr_hit);
    u_c21    : entity work.or2_gate  port map (a => count(2), b => count(1), y => c21);
    u_c321   : entity work.and2_gate port map (a => count(3), b => c21,      y => c321);
    u_c4     : entity work.and2_gate port map (a => count(4), b => c1_n,     y => c16);
    u_byte_c : entity work.or2_gate  port map (a => c321,     b => c16,      y => byte_c);
    u_bv     : entity work.and3_gate port map (a => s_miss, b => byte_c, c => c0_n, y => byte_valid);
    u_wph_a  : entity work.or2_gate  port map (a => wr_hit, b => byte_valid, y => wphase_a);
    u_wphase : entity work.and2_gate port map (a => wphase_a, b => rst_n, y => wphase);
    -- the write signal is the same phase delayed to the positive edge: on for exactly one clock
    -- cycle, from the positive edge after the data is valid to the next positive edge
    u_write  : entity work.pos_dff port map (d => wphase, clk => clk, q => write);

    -- ---------------- Control outputs ----------------
    -- the counter runs in CHECK, WRITE and READ_MISS and restarts at 1 when a request is accepted
    u_bst    : entity work.or3_gate  port map (a => s_check, b => s_write, c => s_miss, y => busy_st);
    u_count  : entity work.and2_gate port map (a => busy_st, b => rst_n, y => count_enable);
    count_restart <= n_check;
    -- byte of the block written by a read miss: c = 10,11 -> 0, 12,13 -> 1, 14,15 -> 2, 16,17 -> 3
    -- (count(2 downto 1) - 1:  bit 1 = count(2) xnor count(1),  bit 0 = not count(1))
    u_fb1    : entity work.xnor2_gate port map (a => count(2), b => count(1), y => fill_byte(1));
    fill_byte(0) <= c1_n;
    hit_result   <= hit_r;

    -- ---------------- Replacement control of the two-way cache ----------------
    -- sampled by the cache on the negative edge that ends CHECK: the victim way is kept for a
    -- read miss, and the way used (read hit, write hit, or the way a read miss fills) becomes
    -- the most recently used. A write miss uses no way (no write allocate).
    u_used   : entity work.or2_gate  port map (a => hit, b => read_write, y => used);
    u_lru    : entity work.and2_gate port map (a => s_check, b => used, y => lru_update);
    latch_way <= s_check;

    latch_input  <= accept;
    latch_output <= n_out;
    fill         <= miss_on;
    state        <= s_out & s_miss & s_write & s_check & s_idle;
end architecture structural;
```

6.5 Testbench and simulation results

The testbench drives the state machine with the real counter and the same clock as the course testbench and checks busy, enable, output enable and the state after every negative edge, and the write signal after every positive edge (and in the middle of each write pulse), for all four scenarios.

Listing: two_way/testbench/control/tb_state_machine.vhd

```
-- Testbench: state_machine (with the clock cycle counter) - every scenario of the specification,
-- checked on every clock edge:
--   read hit (2 clocks), write hit and write miss (3 clocks), read miss (19 clocks)
-- After each negative edge k (k = 1 is the first negative edge after start) the testbench checks
-- busy, enable, output enable, the state, the counter (k while the request runs, then 0) and the
-- kept hit/miss; after the following positive edge, the write signal and the byte being written.
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use work.tb_pkg.all;

entity tb_state_machine is
end entity tb_state_machine;
```

```
architecture sim of tb_state_machine is
    constant S_IDLE  : std_logic_vector(4 downto 0) := "00001";
    constant S_CHECK : std_logic_vector(4 downto 0) := "00010";
    constant S_WRITE : std_logic_vector(4 downto 0) := "00100";
    constant S_MISS  : std_logic_vector(4 downto 0) := "01000";
    constant S_OUT   : std_logic_vector(4 downto 0) := "10000";

    signal clk                          : std_logic := '1';
    signal reset, start, read_write, hit : std_logic := '0';
    signal count                        : std_logic_vector(4 downto 0);
    signal busy, enable, output_enable, latch_input, latch_output : std_logic;
    signal count_enable, fill, write    : std_logic;
    signal count_restart, hit_result    : std_logic;
    signal fill_byte                    : std_logic_vector(1 downto 0);
    signal latch_way, lru_update        : std_logic;
    signal state                        : std_logic_vector(4 downto 0);
    signal done                         : boolean := false;
begin
    dut : entity work.state_machine
        port map (clk => clk, reset => reset, start => start, read_write => read_write, hit => hit,
                  count => count, busy => busy, enable => enable, output_enable => output_enable,
                  latch_input => latch_input, latch_output => latch_output, count_enable => count_enable,
                  count_restart => count_restart, fill_byte => fill_byte, hit_result => hit_result,
                  fill => fill, write => write, latch_way => latch_way, lru_update => lru_update,
                  state => state);
    u_counter : entity work.counter
        port map (count => count_enable, restart => count_restart, clk => clk, q => count);

    clk_gen : process   -- as in chip_test: high at 10k ns, low at 10k + 5 ns
    begin
        while not done loop
            clk <= '1'; wait for 5 ns;
            clk <= '0'; wait for 5 ns;
        end loop;
        wait;
    end process clk_gen;

    stimulus : process
        variable errors : natural := 0;

        function bool_sl (b : boolean) return std_logic is
        begin
            if b then return '1'; else return '0'; end if;
        end function;

        procedure check (cond : boolean; msg : string) is
        begin
            if not cond then
                report "FAIL: " & msg severity error;
                errors := errors + 1;
            end if;
        end procedure;

        -- one request; scenario: 0 read hit, 1 write hit, 2 write miss, 3 read miss
        procedure run (scenario : natural; name : string) is
            variable n          : natural;
            variable e_busy, e_en, e_oe, e_wr : std_logic;
            variable e_state    : std_logic_vector(4 downto 0);
            variable e_count    : natural;
            variable e_hit      : std_logic;
        begin
            -- the CPU raises start with the request on a positive edge, for one clock
            wait until rising_edge(clk);
            start <= '1';
            if scenario = 0 or scenario = 3 then read_write <= '1'; else read_write <= '0'; end if;
            if scenario = 0 or scenario = 1 then hit <= '1'; else hit <= '0'; end if;
            if scenario = 3 then n := 20; else n := 3; end if;
            for k in 1 to n loop
                wait until falling_edge(clk);                   -- negative edge k
                wait for 2 ns;
                e_busy := '0'; e_en := '0'; e_oe := '0'; e_state := S_IDLE;
                case scenario is
                    when 0 =>                                   -- read hit
                        if k = 1 then e_busy := '1'; e_state := S_CHECK; end if;
                        if k = 2 then e_oe := '1'; e_state := S_OUT; end if;
```

```
                    when 1 | 2 =>                               -- write hit / write miss
                        if k = 1 then e_busy := '1'; e_state := S_CHECK; end if;
                        if k = 2 then e_busy := '1'; e_state := S_WRITE; end if;
                    when others =>                              -- read miss
                        if k = 1 then e_busy := '1'; e_state := S_CHECK; end if;
                        if k >= 2 and k <= 18 then e_busy := '1'; e_state := S_MISS; end if;
                        if k = 2 then e_en := '1'; end if;
                        if k = 19 then e_oe := '1'; e_state := S_OUT; end if;
                end case;
                check(busy = e_busy, name & " edge " & str(k) & ": busy=" & str(busy));
                check(enable = e_en, name & " edge " & str(k) & ": enable=" & str(enable));
                check(output_enable = e_oe, name & " edge " & str(k) & ": output enable=" &
str(output_enable));
                check(state = e_state, name & " edge " & str(k) & ": state=" & str(state) & " expected " &
str(e_state));
                -- the counter: clock cycle k of the request, 0 once it is over
                e_count := k;
                if (scenario = 0 and k > 2) or ((scenario = 1 or scenario = 2) and k > 3) or k > 19 then
                    e_count := 0;
                end if;
                check(count = conv_std_logic_vector(e_count, 5), name & " edge " & str(k) & ": count=" &
str(count));
                -- the kept hit/miss: this request's from the end of CHECK (edge 2)
                if k >= 2 then
                    e_hit := bool_sl(scenario = 0 or scenario = 1);
                    check(hit_result = e_hit, name & " edge " & str(k) & ": hit_result=" &
str(hit_result));
                end if;
                -- two-way replacement control: only in CHECK; lru_update unless it is a write miss
                if k = 1 then
                    check(latch_way = '1', name & ": latch_way off in CHECK");
                    check(lru_update = bool_sl(scenario /= 2), name & ": lru_update=" & str(lru_update) &
" in CHECK");
                else
                    check(latch_way = '0' and lru_update = '0', name & " edge " & str(k) & ": replacement
control on outside CHECK");
                end if;
                -- write signal at this negative edge: still on if it started at the previous positive
edge
                e_wr := '0';
                if scenario = 1 and k = 3 then e_wr := '1'; end if;
                if scenario = 3 and (k = 11 or k = 13 or k = 15 or k = 17) then e_wr := '1'; end if;
                check(write = e_wr, name & " edge " & str(k) & ": write=" & str(write) & " (mid-pulse)");
                -- the write signal after the following positive edge
                wait until rising_edge(clk);
                if k = 1 then start <= '0'; end if;             -- CPU removes the request (busy seen)
                wait for 2 ns;
                e_wr := '0';
                if scenario = 1 and k = 2 then e_wr := '1'; end if;
                if scenario = 3 and (k = 10 or k = 12 or k = 14 or k = 16) then e_wr := '1'; end if;
                check(write = e_wr, name & " after edge " & str(k) & ": write=" & str(write));
                if scenario = 3 and e_wr = '1' then             -- bytes 0,1,2,3 of the block
                    check(fill_byte = conv_std_logic_vector((k - 10) / 2, 2),
                          name & " after edge " & str(k) & ": fill_byte=" & str(fill_byte));
                end if;
            end loop;
        end procedure;
    begin
        reset <= '1';
        wait until falling_edge(clk); wait until falling_edge(clk);
        wait until rising_edge(clk);
        reset <= '0';
        wait until falling_edge(clk); wait for 2 ns;
        check(state = S_IDLE and busy = '0', "reset -> IDLE, busy off");
        check(hit_result = '0', "reset -> hit_result 0");

        run(0, "read hit");
        run(1, "write hit");
        run(2, "write miss");
        run(3, "read miss");

        if errors = 0 then
            report "tb_state_machine: ALL TESTS PASSED";
        else
            report "tb_state_machine: " & str(errors) & " TEST(S) FAILED" severity error;
```

```
        end if;
        done <= true;
        wait;
    end process stimulus;
end architecture sim;
```

Simulation result: PASS — `tb_state_machine`: all checks passed, simulation ended at 352 ns.

state machine: read hit (2 clocks)

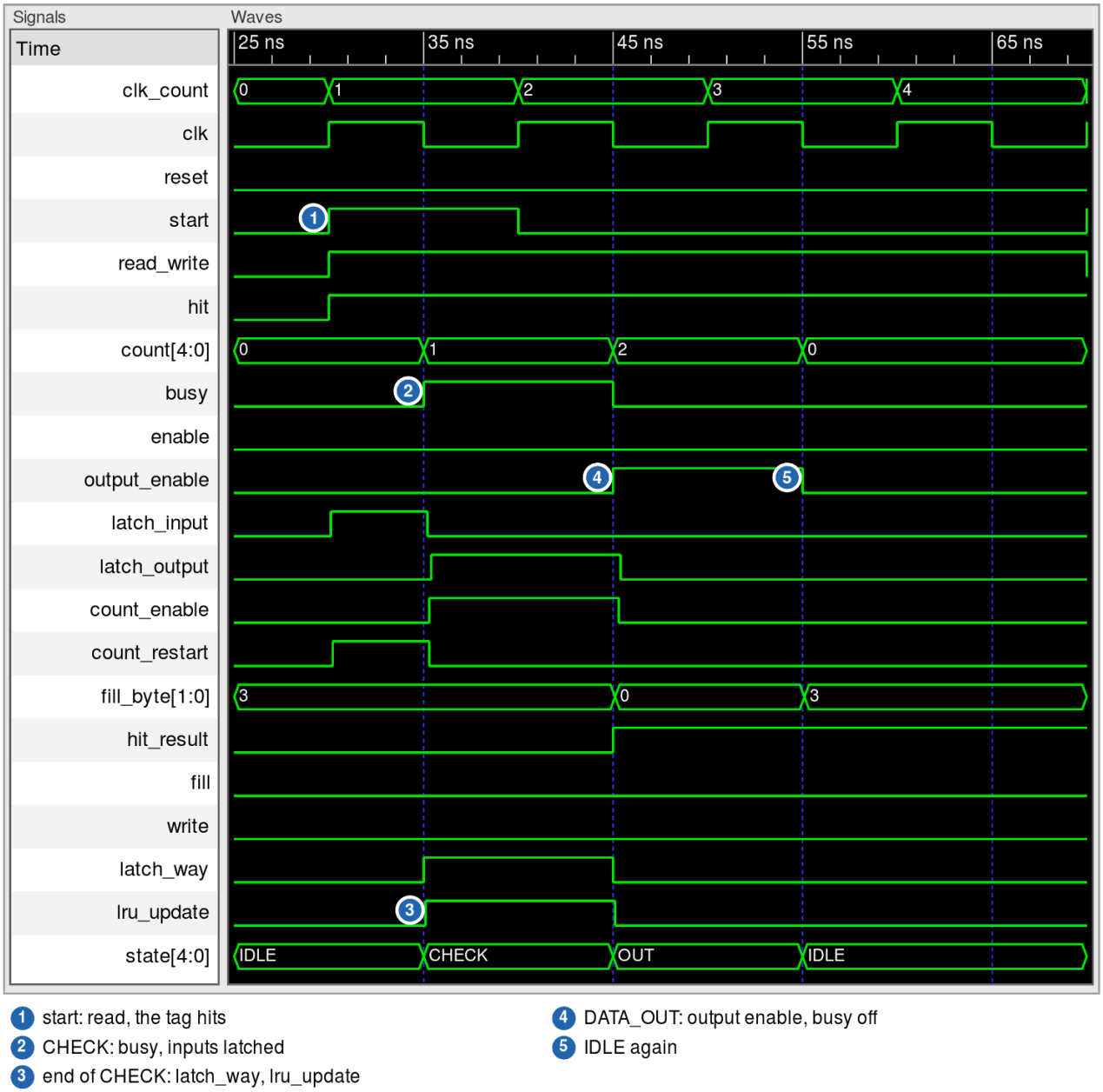


Figure 67: State machine: read hit

busy is high for one clock (CHECK); on the 2nd negative edge the state is DATA_OUT, output enable is on for one clock and busy is off: 2 clocks.

state machine: write hit (3 clocks)

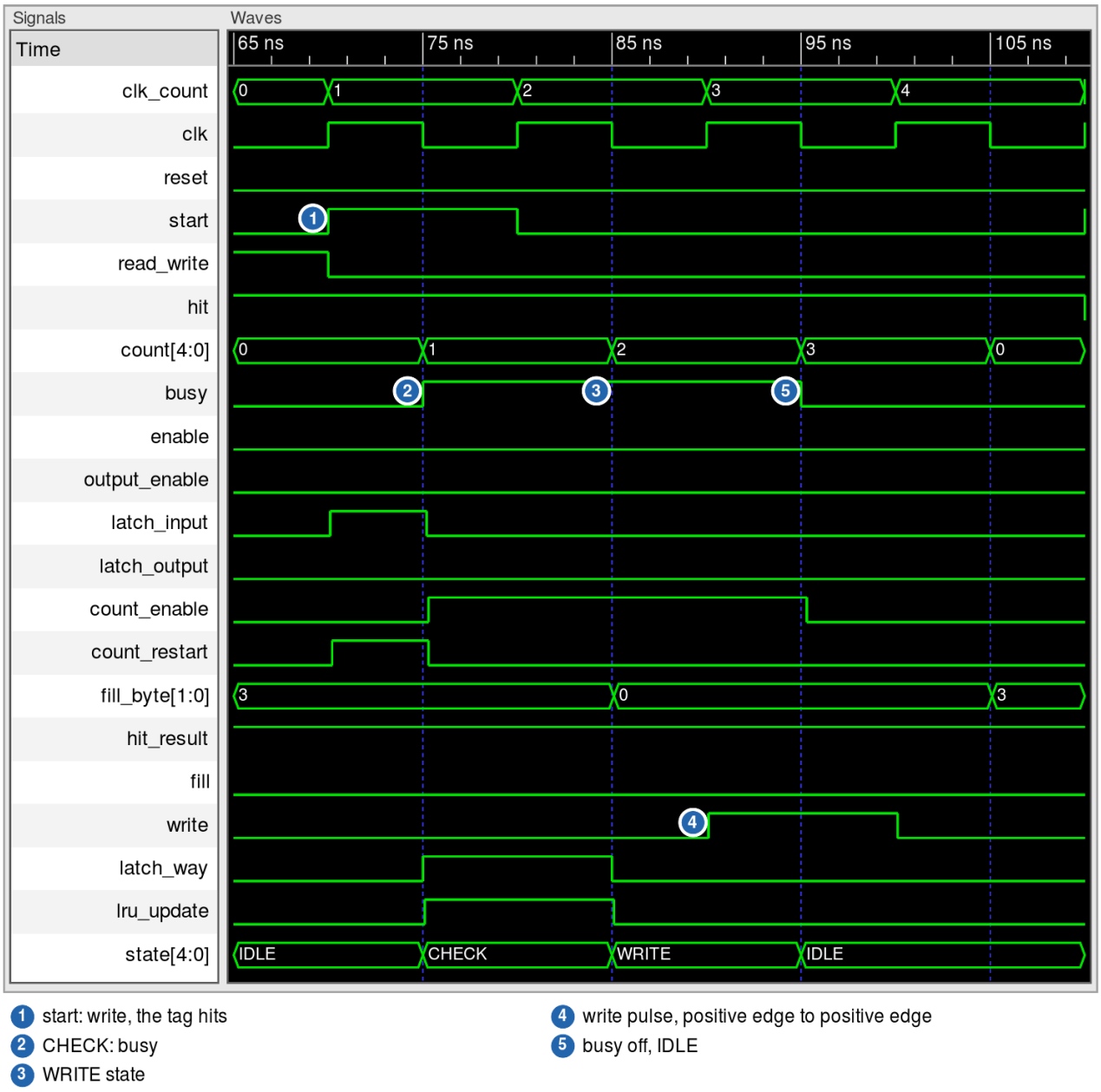


Figure 68: State machine: write hit

CHECK, then WRITE; the write signal is on from the positive edge after the 2nd negative edge to the next positive edge; busy goes low on the 3rd negative edge: 3 clocks.

state machine: write miss (3 clocks, nothing written)

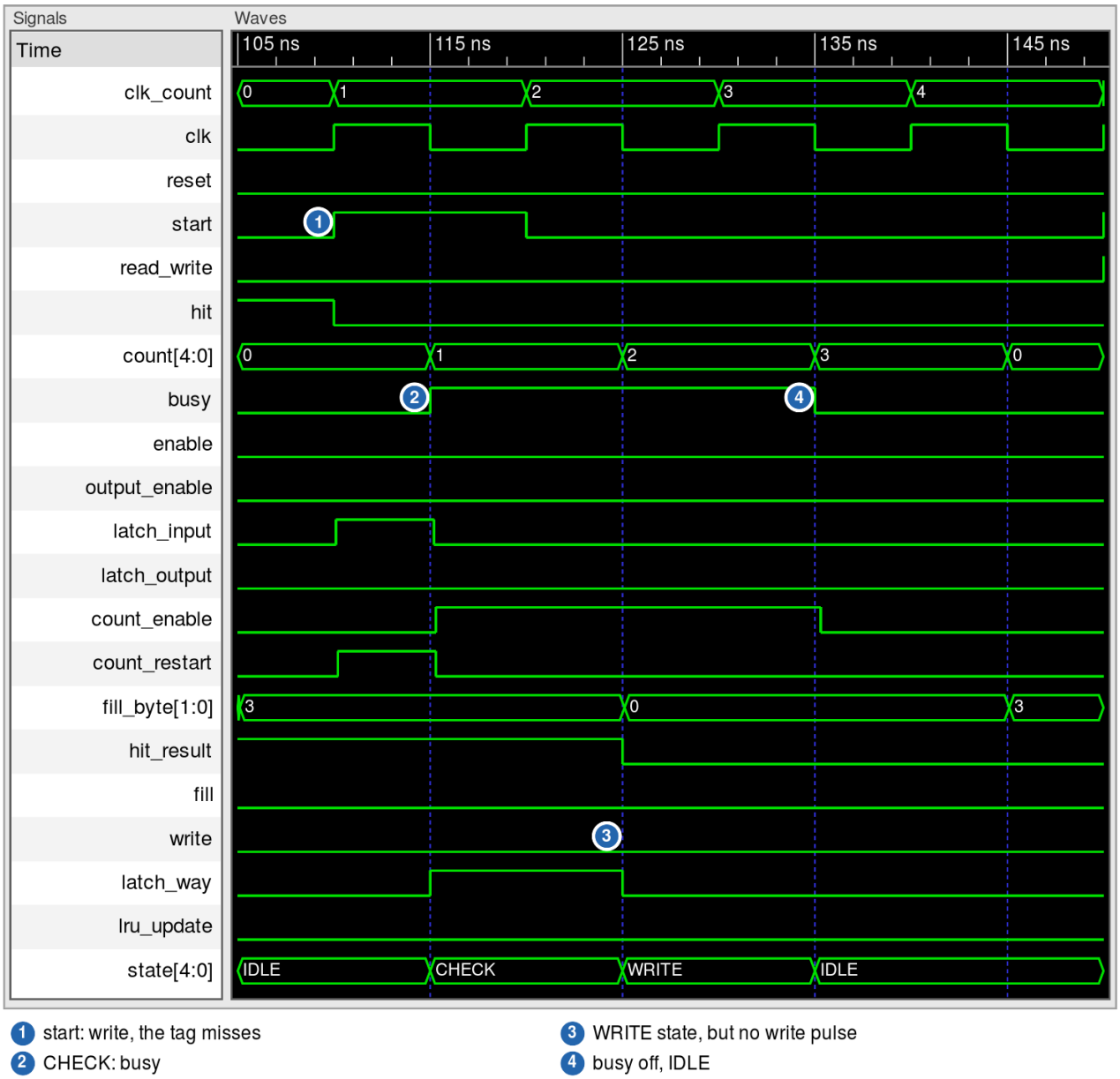


Figure 69: State machine: write miss

The same sequence as the write hit, but hit = 0 so the write signal stays off (no write allocate): 3 clocks.

state machine: read miss (19 clocks)

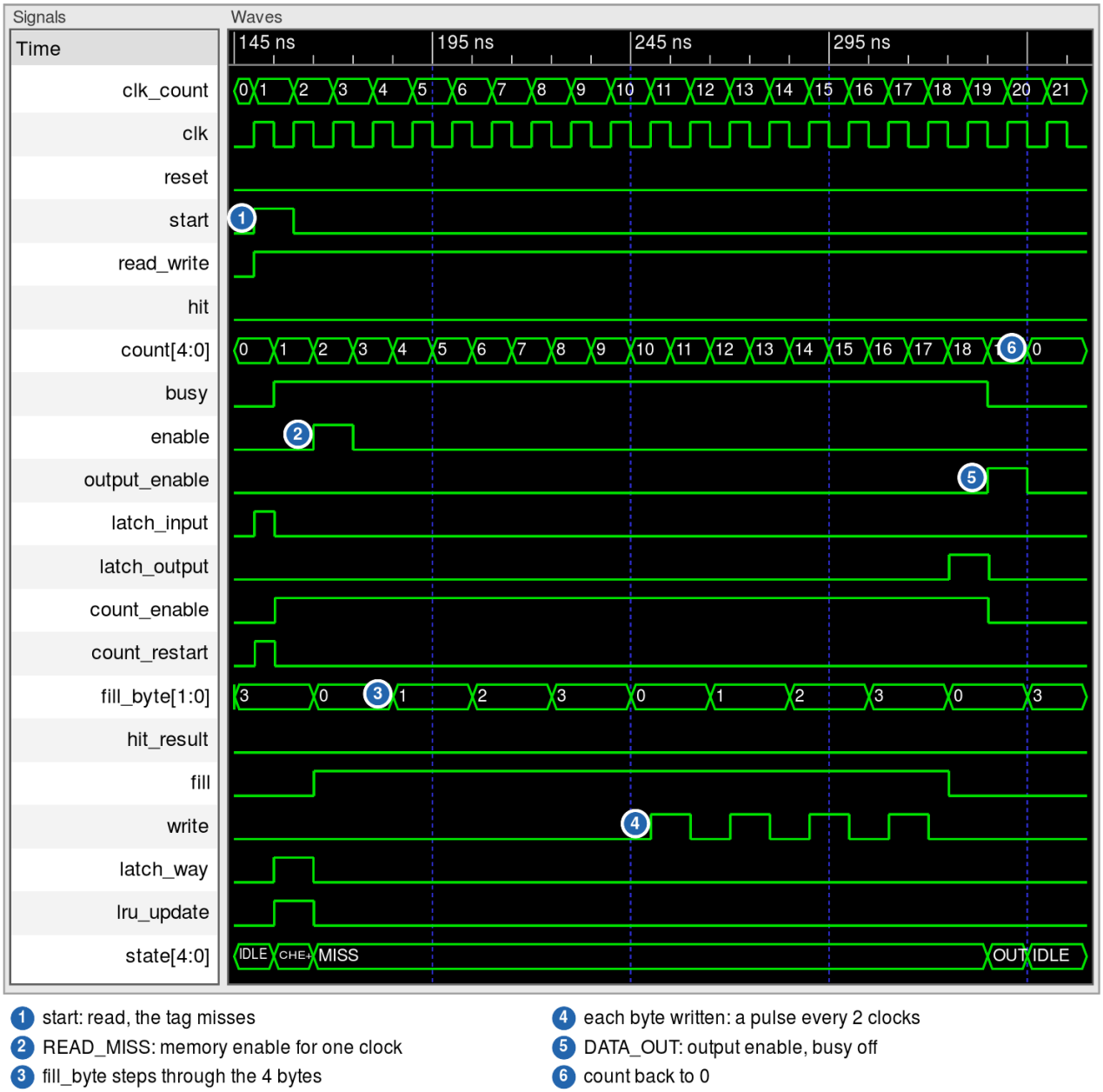


Figure 70: State machine: read miss

enable for one clock on the 2nd negative edge (c = 2); the counter runs; the write signal pulses after c = 10, 12, 14, 16 while fill = 1; at c = 18 the requested byte is read (latch_output), then DATA_OUT with output enable and busy low on the 19th negative edge: 19 clocks.

7 Chip (top level)

The chip connects the blocks: the CA, CD and RD/WR registers (loaded by latch_input), the state machine and counter, the multiplexer that chooses the cache's write data (CPU or memory), the two-way cache (which takes the whole CPU address, reports hit/miss and keeps the LRU state), the output register, and the transmission gates that drive CD (output enable) and MA (enable). MA is tag, block offset and two zeros taken from Gnd. Its ports are exactly those of the component in chip_test.vhd. The hit/miss of a request is decided once, at the end of CHECK: the cache's hit (tag_hit) and the state machine's hit_result both keep it until the next request.

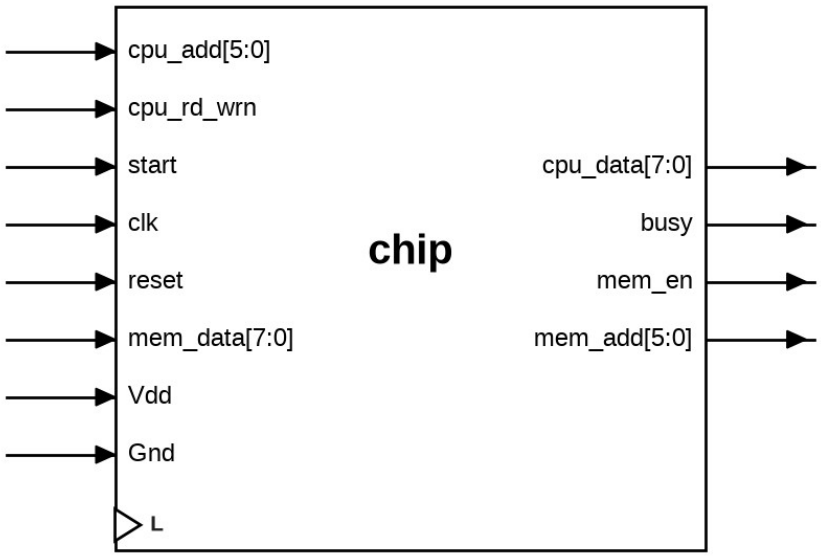


Figure 71: `chip`: inputs and outputs

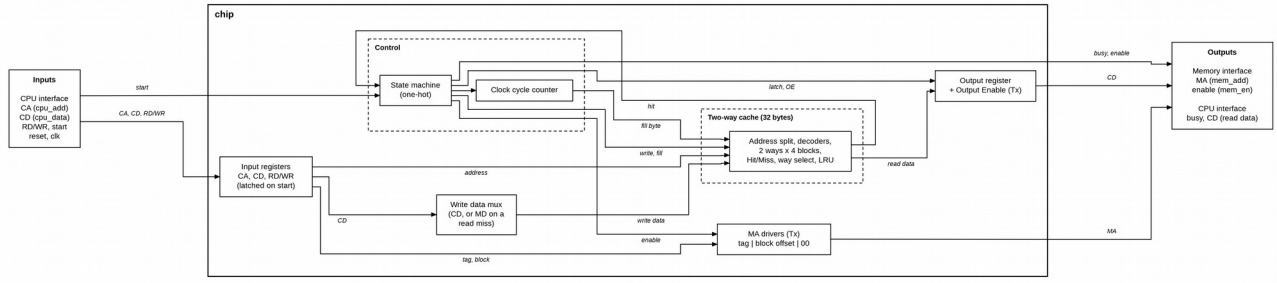


Figure 72: `chip`: block diagram

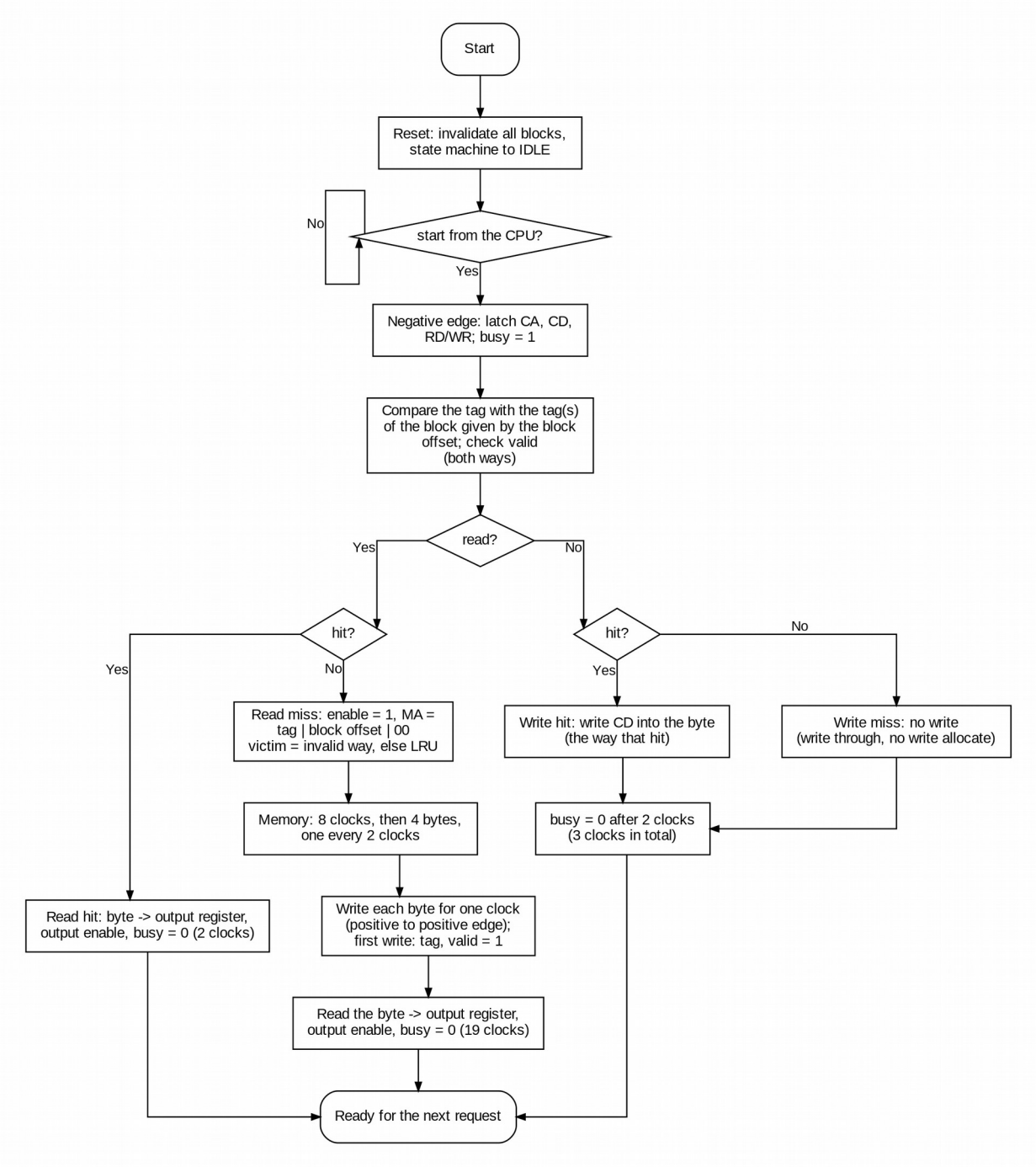


Figure 73: `chip`: operation flowchart

7.1 Code

Listing: two_way/chip.vhd

```
-- Cache chip (CMPE 413/640 project, extra credit): two-way set associative with LRU replacement,
-- byte addressable, 32 bytes (2 ways of 4 blocks x 4 bytes), write through with no write allocate.
-- Read hit 2 clocks, write hit/miss 3, read miss 19 (the same timing as the direct mapped chip).
--
-- Ports match the component in the course testbench chip_test.vhd:
--   CPU Address (CA)   cpu_add    6-bit address from the CPU
--   CPU Data (CD)      cpu_data   8-bit bidirectional: write data in, read data out
--   Read_Write         cpu_rd_wrn 1 = read, 0 = write
--   Start / Busy       start, busy
```

```
--   Reset / Clock      reset, clk
--   Memory Address(MA) mem_add    6-bit address to the memory, last two bits 00
--   Memory Data (MD)   mem_data   8-bit data from the memory
--   Enable             mem_en     memory read request
-- The chip works on the negative edge of clk. busy and enable change on negative edges;
-- CD and MA are high impedance when the chip is not driving them.
library ieee;
use ieee.std_logic_1164.all;
use work.cache_pkg.all;

entity chip is
    port (
        cpu_add    : in    std_logic_vector(5 downto 0);
        cpu_data   : inout std_logic_vector(7 downto 0);
        cpu_rd_wrn : in    std_logic;
        start      : in    std_logic;
        clk        : in    std_logic;
        reset      : in    std_logic;
        mem_data   : in    std_logic_vector(7 downto 0);
        Vdd        : in    std_logic;
        Gnd        : in    std_logic;
        busy       : out   std_logic;
        mem_en     : out   std_logic;
        mem_add    : out   std_logic_vector(5 downto 0)
    );
end entity chip;

architecture structural of chip is
    -- registered CPU request (latched on the negative edge after start)
    signal ca_r                       : std_logic_vector(ADDR_W-1 downto 0);
    signal cd_r                       : std_logic_vector(DATA_W-1 downto 0);
    signal rw_r                       : std_logic;

    -- cache
    signal write_data, read_data, out_r : std_logic_vector(DATA_W-1 downto 0);
    signal tag_hit, tag_miss            : std_logic;   -- tag/valid compare of the cache (held after
CHECK)
    signal hit, miss                    : std_logic;   -- hit/miss of the request (from the end of CHECK)

    -- control
    signal count                                    : std_logic_vector(COUNT_W-1 downto 0);
    signal enable, output_enable, latch_input, latch_output : std_logic;
    signal count_enable, count_restart, fill, write : std_logic;
    signal fill_byte                                : std_logic_vector(1 downto 0);
    signal latch_way, lru_update, way               : std_logic;
    signal state                                    : std_logic_vector(4 downto 0);
    signal oe_n, en_n                               : std_logic;
    signal mem_add_i                                : std_logic_vector(ADDR_W-1 downto 0);
begin
    -- ---------------- Registers for the CPU address, data and read_write ----------------
    u_ca_reg : entity work.register_n generic map (N => ADDR_W)
        port map (d => cpu_add, load => latch_input, clk => clk, q => ca_r);
    u_cd_reg : entity work.register_n generic map (N => DATA_W)
        port map (d => cpu_data, load => latch_input, clk => clk, q => cd_r);
    u_rw_reg : entity work.dff_en
        port map (d => cpu_rd_wrn, load => latch_input, clk => clk, q => rw_r);

    -- ---------------- State machine and clock cycle counter ----------------
    u_sm : entity work.state_machine
        port map (clk => clk, reset => reset, start => start, read_write => rw_r, hit => tag_hit,
                  count => count, busy => busy, enable => enable, output_enable => output_enable,
                  latch_input => latch_input, latch_output => latch_output,
                  count_enable => count_enable, count_restart => count_restart,
                  fill_byte => fill_byte, hit_result => hit, fill => fill, write => write,
                  latch_way => latch_way, lru_update => lru_update, state => state);
    u_counter : entity work.counter
        port map (count => count_enable, restart => count_restart, clk => clk, q => count);
    u_miss    : entity work.not_gate port map (a => hit, y => miss);

    -- ---------------- Cache write data ----------------
    -- read miss: the bytes come from the memory (MD); write hit: from the CPU data register
    u_data_mux : entity work.mux2_n generic map (N => DATA_W)
        port map (a => cd_r, b => mem_data, s => fill, y => write_data);

    -- ---------------- Two-way cache (with its Hit/Miss logic and LRU replacement) ----------------
```

```
    -- the cache takes the whole CPU address and splits it into tag, block offset and byte
    -- offset itself; during a read miss the state machine gives the byte of the block being written
    u_cache : entity work.cache
        port map (clk => clk, address => ca_r, fill => fill, fill_byte => fill_byte, write => write,
                  latch_way => latch_way, lru_update => lru_update, reset => reset, Vdd => Vdd,
                  write_data => write_data, read_data => read_data, hit => tag_hit, miss => tag_miss, way
=> way);

    -- ---------------- Output register and Output Enable onto CPU Data ----------------
    u_out_reg : entity work.register_n generic map (N => DATA_W)
        port map (d => read_data, load => latch_output, clk => clk, q => out_r);
    u_oe_n : entity work.not_gate port map (a => output_enable, y => oe_n);
    g_cd : for i in 0 to DATA_W-1 generate
        u_cd_tx : entity work.tx port map (sel => output_enable, selnot => oe_n, input => out_r(i), output
=> cpu_data(i));
    end generate g_cd;

    -- ---------------- Memory Address and Enable ----------------
    -- MA = | tag | block offset | 00 | (the block's address), driven only while enable is on
    mem_add_i <= ca_r(5 downto 2) & Gnd & Gnd;
    u_en_n : entity work.not_gate port map (a => enable, y => en_n);
    g_ma : for i in 0 to ADDR_W-1 generate
        u_ma_tx : entity work.tx port map (sel => enable, selnot => en_n, input => mem_add_i(i), output =>
mem_add(i));
    end generate g_ma;
    mem_en <= enable;
end architecture structural;
```

7.2 Extended chip testbench

- CPU model: drives start, RD/WR, CA (and CD) on a positive edge, removes them on the positive edge after busy, reads CD on the positive edge after busy goes low.
- Memory model: latches MA when enable is high on a positive edge and drives byte i of the block on the (8 + 2i)th negative edge after enable, stable for 2 cycles (byte at address A = 5A + 17).
- Reference model of the two-way cache (valid, tag and data of both ways, one LRU bit per block offset, victim = invalid way first, else LRU) predicts hit or miss and the data of every read; because the expected busy time depends on hit or miss, every hit/miss decision is checked.
- For every request it checks the data read, busy high on the 1st negative edge, busy low on the 2nd (read hit), 3rd (write) or 19th (read miss) negative edge, enable for exactly one cycle with the right MA on a read miss, MA and CD high impedance when not driven, and CD driven for exactly one cycle.
- Directed tests: the course sequence; two tags in block offset 0 kept in the two ways (0x00 and 0x10 both hit afterwards — in the direct mapped chip 0x10 would evict 0x00); a third tag (0x20) replacing the LRU way; the evicted block missing again; a write hit in way 1; write miss with no allocate; back-to-back requests; reset invalidating every block; then 150 random requests.

Listing: two_way/testbench/chip/tb_chip.vhd

```
-- Testbench: chip - CPU model + memory model following the specification's protocol and timing,
-- checked against a reference model of the two-way set associative cache with LRU replacement.
--   CPU    : drives start, RD/WR, CA (and CD for a write) on a positive edge; removes them on the
--            positive edge after it sees busy; reads CD on the positive edge after busy goes low.
--   Memory : latches MA on the positive edge when enable is high; drives byte i of the block on
--            the (8 + 2i)th negative edge after enable went high, stable for 2 clock cycles.
-- Checked for every request: the data read, busy high on the 1st negative edge, busy low on the
-- 2nd (read hit), 3rd (write hit/miss) or 19th (read miss) negative edge, enable for exactly one
-- cycle with MA = tag & block offset & 00 on a read miss, and CD driven for exactly one cycle.
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use work.tb_pkg.all;
```

```
entity tb_chip is
    generic (N_RANDOM : natural := 150);
end entity tb_chip;

architecture sim of tb_chip is
    signal clk                    : std_logic := '1';
    signal cpu_add, mem_add       : std_logic_vector(5 downto 0);
    signal cpu_data, mem_data     : std_logic_vector(7 downto 0);
    signal cpu_rd_wrn, start, reset, busy, mem_en : std_logic;
    signal Vdd                    : std_logic := '1';
    signal Gnd                    : std_logic := '0';
    signal done                   : boolean := false;
    signal neg_edges              : natural := 0;     -- negative edge counter
    signal mem_block_addr         : std_logic_vector(5 downto 0) := (others => '0');

    -- the memory: byte at address A holds A * 5 + 17 (mod 256)
    function mem_value (a : natural) return std_logic_vector is
    begin
        return conv_std_logic_vector((a * 5 + 17) mod 256, 8);
    end function;
begin
    dut : entity work.chip
        port map (cpu_add => cpu_add, cpu_data => cpu_data, cpu_rd_wrn => cpu_rd_wrn, start => start,
                  clk => clk, reset => reset, mem_data => mem_data, Vdd => Vdd, Gnd => Gnd,
                  busy => busy, mem_en => mem_en, mem_add => mem_add);

    clk_gen : process   -- as in chip_test: high at 10k ns, low at 10k + 5 ns
    begin
        while not done loop
            clk <= '1'; wait for 5 ns;
            clk <= '0'; wait for 5 ns;
        end loop;
        wait;
    end process clk_gen;

    edge_count : process (clk)
    begin
        if falling_edge(clk) then
            neg_edges <= neg_edges + 1;
        end if;
    end process edge_count;

    -- ---------------- memory model ----------------
    memory : process
        variable base : natural;
        variable e    : natural;   -- negative edges since enable went high
    begin
        mem_data <= (others => 'Z');
        loop
            wait until rising_edge(clk);
            if mem_en = '1' then                      -- memory latches MA on a positive edge
                mem_block_addr <= mem_add;
                base := conv_integer(unsigned(mem_add));
                e := 0;
                for i in 0 to 3 loop
                    -- byte i on the (8 + 2i)th negative edge after enable went high
                    while e < 8 + 2 * i loop
                        wait until falling_edge(clk);
                        e := e + 1;
                    end loop;
                    mem_data <= mem_value(base + i);
                end loop;
                wait until falling_edge(clk);
                wait until falling_edge(clk);          -- last byte stable for 2 cycles
                mem_data <= (others => 'Z');
            end if;
        end loop;
    end process memory;

    -- ---------------- CPU model and checker ----------------
    cpu : process
        -- reference model of the two-way cache: index way * 16 + block * 4 + byte (data),
        -- way * 4 + block (tag, valid); one LRU bit per block offset = the way to replace next
        type bytes_t is array (0 to 31) of std_logic_vector(7 downto 0);
```

```
        type tags_t  is array (0 to 7) of natural;
        variable c_data   : bytes_t;
        variable c_tag    : tags_t;
        variable c_valid  : std_logic_vector(7 downto 0) := (others => '0');
        variable c_lru    : std_logic_vector(3 downto 0) := "0000";
        variable errors   : natural := 0;
        variable n_req    : natural := 0;
        variable n_rd_hit, n_rd_miss, n_wr_hit, n_wr_miss : natural := 0;
        variable seed     : natural := 12345;

        function other (w : natural) return std_logic is   -- the other way, as an LRU bit
        begin
            if w = 0 then return '1'; else return '0'; end if;
        end function;

        procedure check (cond : boolean; msg : string) is
        begin
            if not cond then
                report "FAIL: " & msg severity error;
                errors := errors + 1;
            end if;
        end procedure;

        procedure do_reset is
        begin
            wait until rising_edge(clk);
            reset <= '1'; start <= '0';
            wait until rising_edge(clk);
            wait until rising_edge(clk);
            reset <= '0';
            c_valid := (others => '0'); c_lru := "0000";
        end procedure;

        -- one CPU request; gap = idle clock cycles before it (0 = back to back)
        procedure request (rd : std_logic; a : natural; d : natural; gap : natural) is
            variable tag, blk, byt : natural;
            variable hit           : boolean;
            variable expected_end  : natural;
            variable start_edge    : natural;
            variable k             : natural;
            variable got           : std_logic_vector(7 downto 0);
            variable expected      : std_logic_vector(7 downto 0);
            variable what          : string(1 to 10);
            variable hw, victim    : natural;           -- way that hits, way a read miss fills
        begin
            tag := a / 16; blk := (a / 4) mod 4; byt := a mod 4;
            hit := false; hw := 0;
            for w in 0 to 1 loop
                if c_valid(w * 4 + blk) = '1' and c_tag(w * 4 + blk) = tag then
                    hit := true; hw := w;
                end if;
            end loop;
            if c_valid(blk) = '0' then victim := 0;              -- an invalid way first,
            elsif c_valid(4 + blk) = '0' then victim := 1;
            elsif c_lru(blk) = '1' then victim := 1;            -- else the LRU way
            else victim := 0;
            end if;
            for i in 1 to gap loop
                wait until rising_edge(clk);
            end loop;
            -- positive edge: start, RD/WR, CA (and CD for a write)
            cpu_add <= conv_std_logic_vector(a, 6); cpu_rd_wrn <= rd; start <= '1';
            if rd = '0' then cpu_data <= conv_std_logic_vector(d, 8); else cpu_data <= (others => 'Z');
end if;
            start_edge := neg_edges;                    -- the next negative edge is number start_edge + 1
            if rd = '1' and hit then expected_end := 2; what := "read hit  "; n_rd_hit := n_rd_hit + 1;
            elsif rd = '1' then expected_end := 19; what := "read miss "; n_rd_miss := n_rd_miss + 1;
            elsif hit then expected_end := 3; what := "write hit "; n_wr_hit := n_wr_hit + 1;
            else expected_end := 3; what := "write miss"; n_wr_miss := n_wr_miss + 1;
            end if;
            n_req := n_req + 1;

            -- negative edge 1: busy goes high; the CPU removes its signals on the next positive edge
            wait until falling_edge(clk); wait for 1 ns;
            check(busy = '1', what & " @" & str(a) & ": busy not high on the 1st negative edge");
```

```
            wait until rising_edge(clk);
            cpu_add <= (others => 'Z'); cpu_data <= (others => 'Z'); cpu_rd_wrn <= 'Z'; start <= '0';

            -- follow the request edge by edge until busy goes low
            k := 1;
            loop
                wait until falling_edge(clk); wait for 1 ns;
                k := k + 1;
                if rd = '1' and not hit then          -- read miss: enable for exactly one cycle
                    if k = 2 then
                        check(mem_en = '1', what & " @" & str(a) & ": enable not on at the 2nd negative
edge");
                        check(mem_add = conv_std_logic_vector(tag * 16 + blk * 4, 6),
                              what & " @" & str(a) & ": MA = " & str(mem_add));
                    else
                        check(mem_en = '0', what & " @" & str(a) & ": enable on at edge " & str(k));
                        check(mem_add = "ZZZZZZ", what & " @" & str(a) & ": MA driven while enable is
off");
                    end if;
                end if;
                exit when busy = '0' or k > 25;
                check(cpu_data = "ZZZZZZZZ", what & " @" & str(a) & ": CD driven while busy");
            end loop;
            check(k = expected_end, what & " @" & str(a) & ": busy low on negative edge " & str(k)
                                    & ", expected " & str(expected_end));

            -- read: the CPU reads CD on the next positive edge; CD is driven for one cycle only
            wait until rising_edge(clk);
            got := cpu_data;
            if rd = '1' then
                if hit then
                    expected := c_data(hw * 16 + blk * 4 + byt);
                    c_lru(blk) := other(hw);            -- the way that hit is now the most recent
                else
                    for i in 0 to 3 loop                -- read miss: the whole block from memory
                        c_data(victim * 16 + blk * 4 + i) := mem_value(tag * 16 + blk * 4 + i);
                    end loop;
                    c_tag(victim * 4 + blk) := tag; c_valid(victim * 4 + blk) := '1';
                    c_lru(blk) := other(victim);
                    expected := c_data(victim * 16 + blk * 4 + byt);
                end if;
                check(got = expected, what & " @" & str(a) & ": CD = " & str(got) & ", expected " &
str(expected));
                wait until falling_edge(clk); wait for 1 ns;
                check(cpu_data = "ZZZZZZZZ", what & " @" & str(a) & ": CD still driven after one cycle");
                wait until rising_edge(clk);
            else
                check(got = "ZZZZZZZZ", what & " @" & str(a) & ": CD driven after a write");
                if hit then                                 -- write hit updates the way that hit;
                    c_data(hw * 16 + blk * 4 + byt) := conv_std_logic_vector(d, 8);   -- write miss:
                    c_lru(blk) := other(hw);                                          -- no allocate
                end if;
            end if;
        end procedure;

        impure function rnd (n : natural) return natural is   -- small LCG (fits 32-bit integers)
        begin
            seed := (seed * 75 + 74) mod 65537;
            return (seed / 7) mod n;
        end function;
        variable a, d : natural;
    begin
        cpu_add <= (others => 'Z'); cpu_data <= (others => 'Z'); cpu_rd_wrn <= 'Z';
        start <= '0'; reset <= '0';
        do_reset;

        -- ---------------- directed tests ----------------
        request('1', 16#00#, 0, 1);          -- read miss (the course example)
        request('1', 16#01#, 0, 1);          -- read hits in the same block
        request('1', 16#02#, 0, 1);
        request('0', 16#03#, 16#FF#, 1);     -- write hit
        request('1', 16#03#, 0, 1);          -- read back FF
        request('0', 16#3F#, 16#AA#, 1);     -- write miss: no write allocate
        request('1', 16#3F#, 0, 1);          -- so this is a read miss (memory value, not AA)
        request('1', 16#3C#, 0, 0);          -- read hit, back to back
```

```
        request('1', 16#10#, 0, 0);          -- tag 01, block 0: read miss into way 1 (way 0 kept)
        request('1', 16#03#, 0, 1);          -- tag 00 is still in way 0: read hit (FF)
        request('1', 16#11#, 0, 0);          -- tag 01 in way 1: read hit
        request('1', 16#20#, 0, 0);          -- tag 10, block 0: replaces the LRU way (way 0, tag 00)
        request('1', 16#12#, 0, 0);          -- tag 01 still in way 1: hit
        request('1', 16#00#, 0, 0);          -- tag 00 was evicted: read miss, replaces LRU (tag 10)
        request('0', 16#13#, 16#C3#, 0);     -- write hit in way 1
        request('1', 16#13#, 0, 0);          -- read C3 from way 1
        request('1', 16#03#, 0, 0);          -- way 0 holds tag 00 from memory again (not FF)
        request('0', 16#25#, 16#5A#, 0);     -- write miss on an invalid block (block 1)
        request('1', 16#25#, 0, 0);          -- read miss
        request('0', 16#25#, 16#5A#, 0);     -- now a write hit
        request('1', 16#25#, 0, 0);          -- read 5A
        do_reset;                            -- reset invalidates every block
        request('1', 16#25#, 0, 1);          -- miss after reset
        report "Directed tests done, errors = " & str(errors);

        -- ---------------- random requests ----------------
        for i in 1 to N_RANDOM loop
            a := rnd(64); d := rnd(256);
            if rnd(10) < 4 then
                request('0', a, d, rnd(3));
            else
                request('1', a, 0, rnd(3));
            end if;
        end loop;

        report "STATS: requests=" & str(n_req) & " read hits=" & str(n_rd_hit) & " read misses=" &
str(n_rd_miss)
             & " write hits=" & str(n_wr_hit) & " write misses=" & str(n_wr_miss);
        if errors = 0 then
            report "tb_chip: ALL TESTS PASSED";
        else
            report "tb_chip: " & str(errors) & " TEST(S) FAILED" severity error;
        end if;
        done <= true;
        wait;
    end process cpu;
end architecture sim;
```

Simulation result: PASS — `tb_chip`: all checks passed, simulation ended at 15.22 µs.

Statistics: `requests=172 read hits=66 read misses=50 write hits=25 write misses=31`

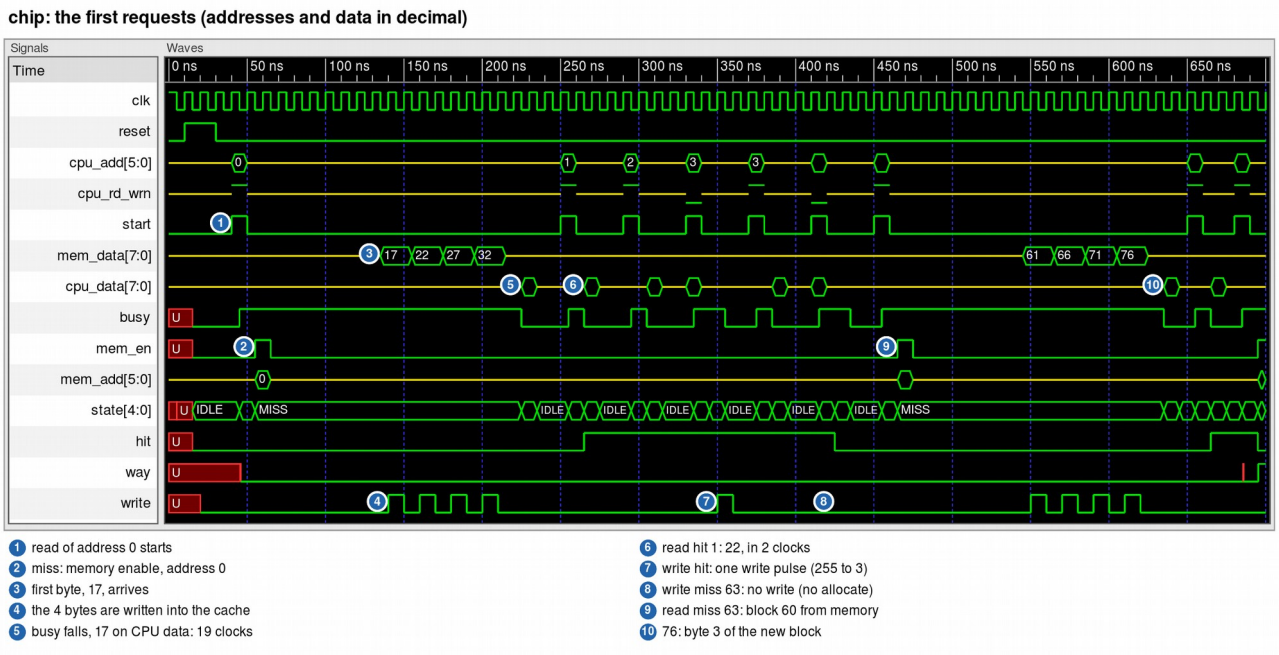


Figure 74: `tb_chip`: the first requests (read miss, read hits, write hit, read, write miss)

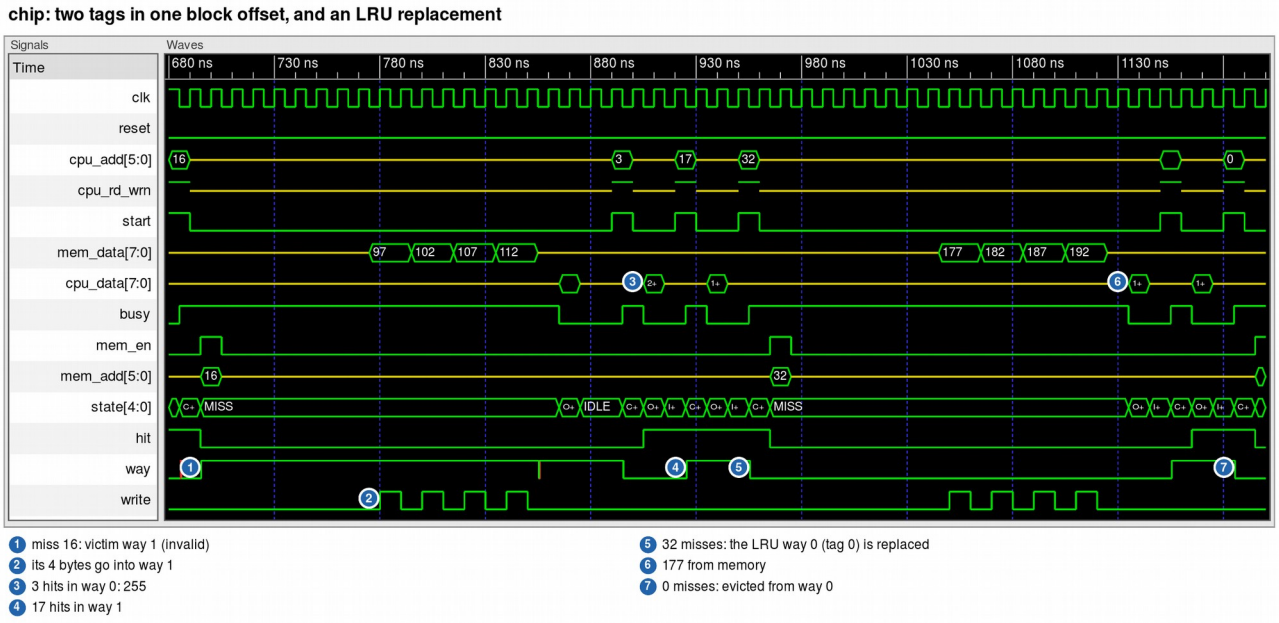


Figure 75: `tb_chip`: two tags in one block offset, and an LRU replacement

Read 0x10 (tag 01, block offset 0) misses while tag 00 is in way 0: way 1 is invalid, so it is the victim (`way` = 1 during the fill). Read 0x03 then hits in way 0 (FF, written earlier) and 0x11 hits in way 1 — both blocks of block offset 0 stay in the cache. Read 0x20 (tag 10) misses; both ways are valid and way 0 is the least recently used, so way 0 is filled.

8 Course testbench (chip_test)

The course testbench `chip_test.vhd` was run without any change: its chip component already has the same port names, types and order as this design's `chip` entity, and it binds `work.chip(structural)`. It reads `chip_in.txt` from the run directory and prints the chip's inputs and outputs on every positive and negative clock edge. It was run for 360 ns, as in `run.ncsim`.

8.1 Sample input

`chip_in.txt` holds six lines per clock: CA, CD, RD/WR, start and reset (applied on the positive edge) and MD (applied on the negative edge). One row per clock:

Table 12: chip_in.txt (inputs of the course testbench)

Clock	CA	CD	RD/WR	Start	Reset	MD
1	ZZZZZZ	ZZZZZZZZ	Z	0	1	ZZZZZZZZ
2	ZZZZZZ	ZZZZZZZZ	Z	0	1	ZZZZZZZZ
3	000000	ZZZZZZZZ	1	1	0	ZZZZZZZZ
4	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
5	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
6	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
7	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
8	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
9	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
10	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
11	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
12	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000000
13	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000000
14	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000001
15	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000001
16	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000010
17	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000010
18	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000011
19	ZZZZZZ	ZZZZZZZZ	Z	0	0	00000011
20	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
21	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
22	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
23	000011	11111111	0	1	0	ZZZZZZZZ
24	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
25	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
26	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
27	000011	ZZZZZZZZ	1	1	0	ZZZZZZZZ
28	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
29	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
30	111111	10101010	0	1	0	ZZZZZZZZ
31	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
32	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
33	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
34	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ
35	ZZZZZZ	ZZZZZZZZ	Z	0	0	ZZZZZZZZ

Requests: reset for 2 clocks; read 0x00 (read miss; memory returns 00, 01, 02, 03); write 0xFF to 0x03 (write hit); read 0x03 (read hit, returns FF); write 0xAA to 0x3F (write miss).

8.2 Output and comparison with the reference

The output of this design (two_way/testbench/chip_out.log, complete in Appendix A) was compared field by field with the course's reference ncsim.log. The reference was produced with 8-bit addresses (as its README notes), so only the low 6 address bits are compared. Result: MATCH in all 70 prints (differences only before the first reset edge: X in the reference, U in GHDL). The course sequence never puts two tags in one block offset, so the two-way chip produces exactly the same output as the direct mapped chip, with the same timing.

Listing: comparison with the reference (logs/chip_test.log)

```
prints compared: 70 (reference 70, this design 70)
  print   0  clock  0  Busy             reference=X          this design=U   (uninitialised before reset:
X vs U)
  print   0  clock  0  Memory  Enable   reference=X          this design=U   (uninitialised before reset:
X vs U)
  print   1  clock  0  Busy             reference=X          this design=U   (uninitialised before reset:
X vs U)
  print   1  clock  0  Memory  Enable   reference=X          this design=U   (uninitialised before reset:
X vs U)
RESULT: MATCH
```

Output for the read miss (the first and last clocks of the request):

```
 Clock: 2 Start: 0 Cpu Read/Write: Z Reset: 1
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 0 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 ------------------------------------------------
 Clock: 2 Start: 1 Cpu Read/Write: 1 Reset: 0
 CPU address: 000000 CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 0 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 ------------------------------------------------
 Clock: 3 Start: 1 Cpu Read/Write: 1 Reset: 0
 CPU address: 000000 CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 1 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 ------------------------------------------------
 Clock: 3 Start: 0 Cpu Read/Write: Z Reset: 0
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 1 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 ------------------------------------------------
 Clock: 4 Start: 0 Cpu Read/Write: Z Reset: 0
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 1 Memory  Enable: 1
 Memory  Address: 000000
 ------------------------------------------------
 Clock: 4 Start: 0 Cpu Read/Write: Z Reset: 0
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 1 Memory  Enable: 1
 Memory  Address: 000000
 ------------------------------------------------
 Clock: 5 Start: 0 Cpu Read/Write: Z Reset: 0
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ
```

```
Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 5 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
  ...
Clock: 19 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000011

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 19 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000011

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 20 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 20 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 21 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: 00000000
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 21 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: 00000000
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 22 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 22 Start: 1 Cpu Read/Write: 0 Reset: 0
CPU address: 000011 CPU data: 11111111
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
```

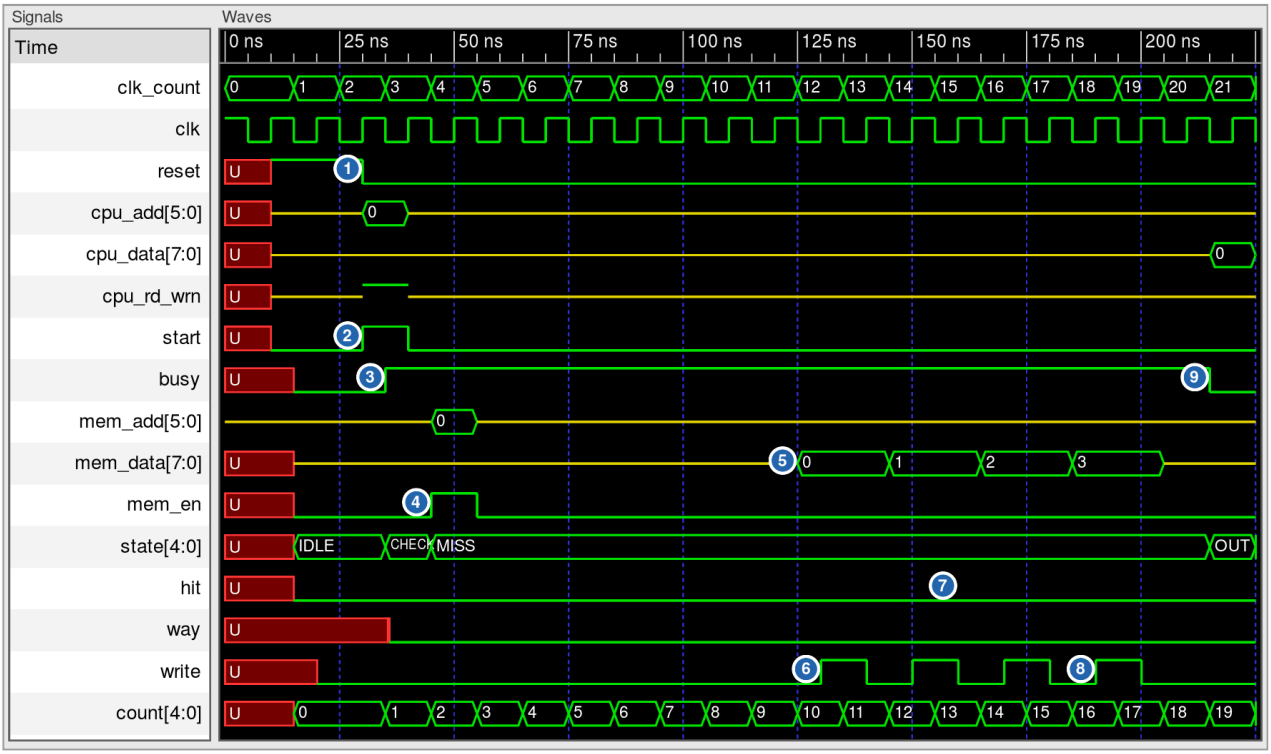
8.3 Waveforms

The course run is shown one operation per figure (read miss, write hit, read hit, write miss), followed by the clock cycle counter over the four requests. Each figure is drawn

from the simulation of chip_test.vhd with the current design, with numbered markers on the events, explained with their clock number (clk_count) in the key under each figure.

All waveform figures in this report are drawn from the GHDL simulation results (VCD files), in the layout of a waveform viewer: a Signals panel with the names and a black Waves panel with a time ruler. Green is a valid 0/1 or bus value (buses in decimal), a red box is U or X, and a yellow line in the middle is Z (bus released); a value too long for its box is cut with a +. The one-hot state is shown with its state name. The chip_test figures use the signal names of chip_test.vhd: clk_count = CLOCK_COUNT, cpu_add = CPU_ADDRESS (CA), cpu_data = CPU_DATA (CD), cpu_rd_wrn = READ_WRITE, mem_add = MEMORY_ADDRESS (MA), mem_data = MEMORY_DATA (MD), mem_en = MEMORY_ENABLE; state, hit, way, write and count are inside the chip.

Read miss: reset, then a read of address 0 (19 clocks)



1. clock 2: reset released: every valid bit is 0, state IDLE
2. clock 2: start with CA = 0, RD/WR = 1 (read)
3. clock 3: busy rises; CHECK compares the tag: miss
4. clock 4: READ_MISS: enable for one clock, MA = 0
5. clock 12: memory byte 0 (value 0), 8th negative edge after enable
6. a write pulse stores it; bytes 1-3 follow every 2 clocks
7. hit stays 0 for the whole read miss (held from CHECK)
8. clock 18: byte 3 written: the block is valid with its tag
9. clock 21: busy falls, 0 on CD; count = 19: 19 clocks

Figure 76: chip_test: read miss (read of 0x00 after reset, 19 clocks)

Reset for two clocks; the read of 0x00 starts on clock 2. busy goes high on the next negative edge (clock 3); on the miss, MEMORY_ENABLE is on for one clock with MEMORY_ADDRESS = 0 (clock 4). The memory returns bytes 0, 1, 2, 3 from clock 12, each for 2 clocks; each is written into block 0 by a one-cycle write pulse (positive edge

to positive edge) while it is stable. On clock 21 busy goes low and the data 0 is on CPU_DATA for one clock: 19 clocks (count = 19). hit, the request's hit/miss, stays 0 from the CHECK cycle to the end of the request: the tag and valid cells being written during the fill do not reach it. The four operation figures are the GHDL simulation of the course testbench chip_test.vhd with the course input file. Compared with the course's desired waveforms (cmpe413_project_desired_waveforms.pdf), every event falls on the same clock: busy high on clock 3, memory enable and MA = 00 on clock 4, the memory bytes 00, 01, 02, 03 from clock 12 every 2 clocks, busy low with 00 on CPU data on clock 21 (19 clocks); write hit 3 clocks, read hit 2 clocks with FF, write miss 3 clocks with no memory access (addresses are 6 bits in this design).

Write hit: write 255 to address 3 (3 clocks)

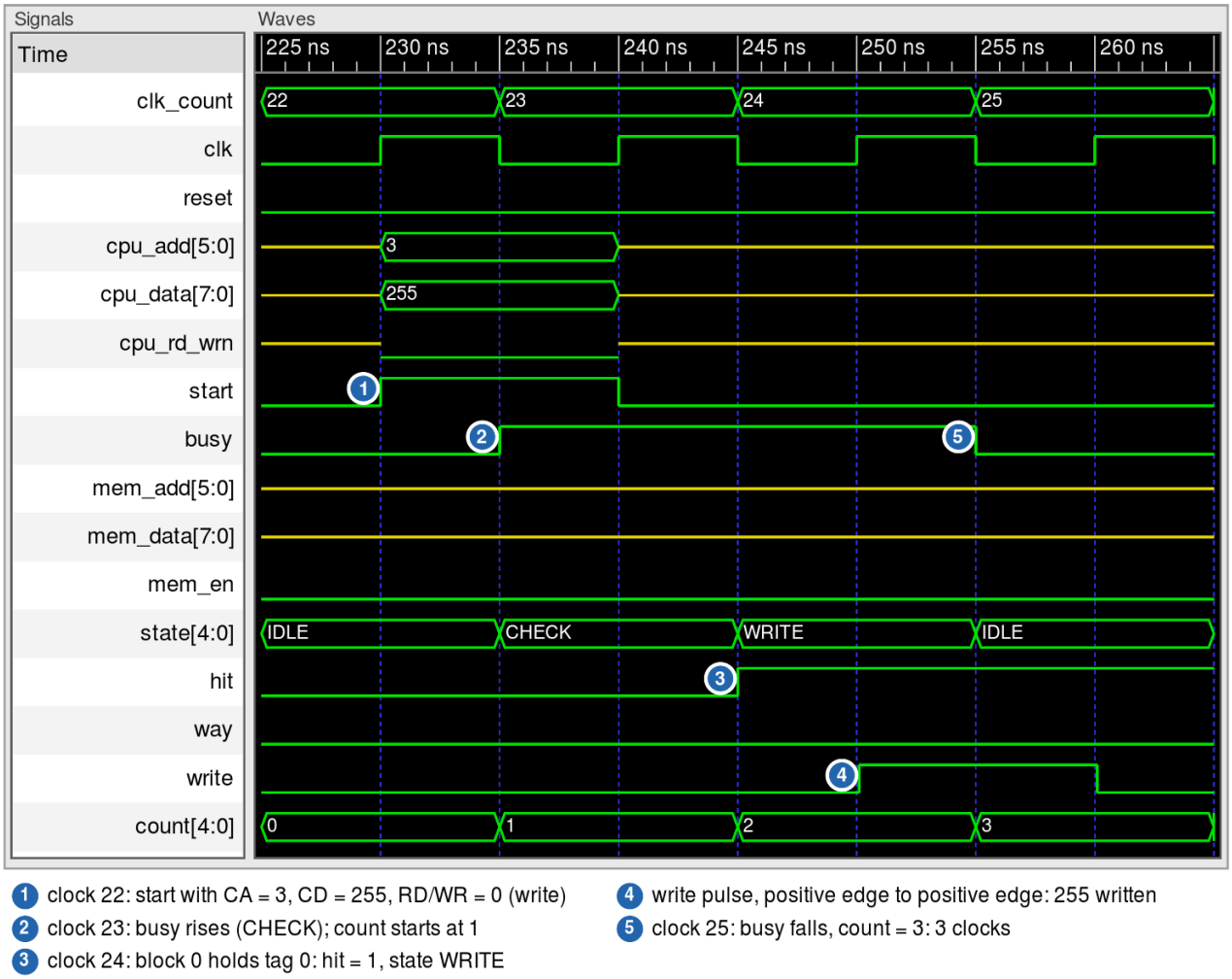


Figure 77: chip_test: write hit (0xFF to 0x03, 3 clocks)

Write 0xFF (255) to 0x03: the block is already in the cache, so CHECK finds hit = 1 and the state machine goes to WRITE; the write signal is on for one clock, from the positive edge after the 2nd negative edge to the next positive edge, and busy goes low on the 3rd negative edge: 3 clocks (count = 3). The counter started again at 1 with this request.

Read hit: read address 3 (2 clocks)

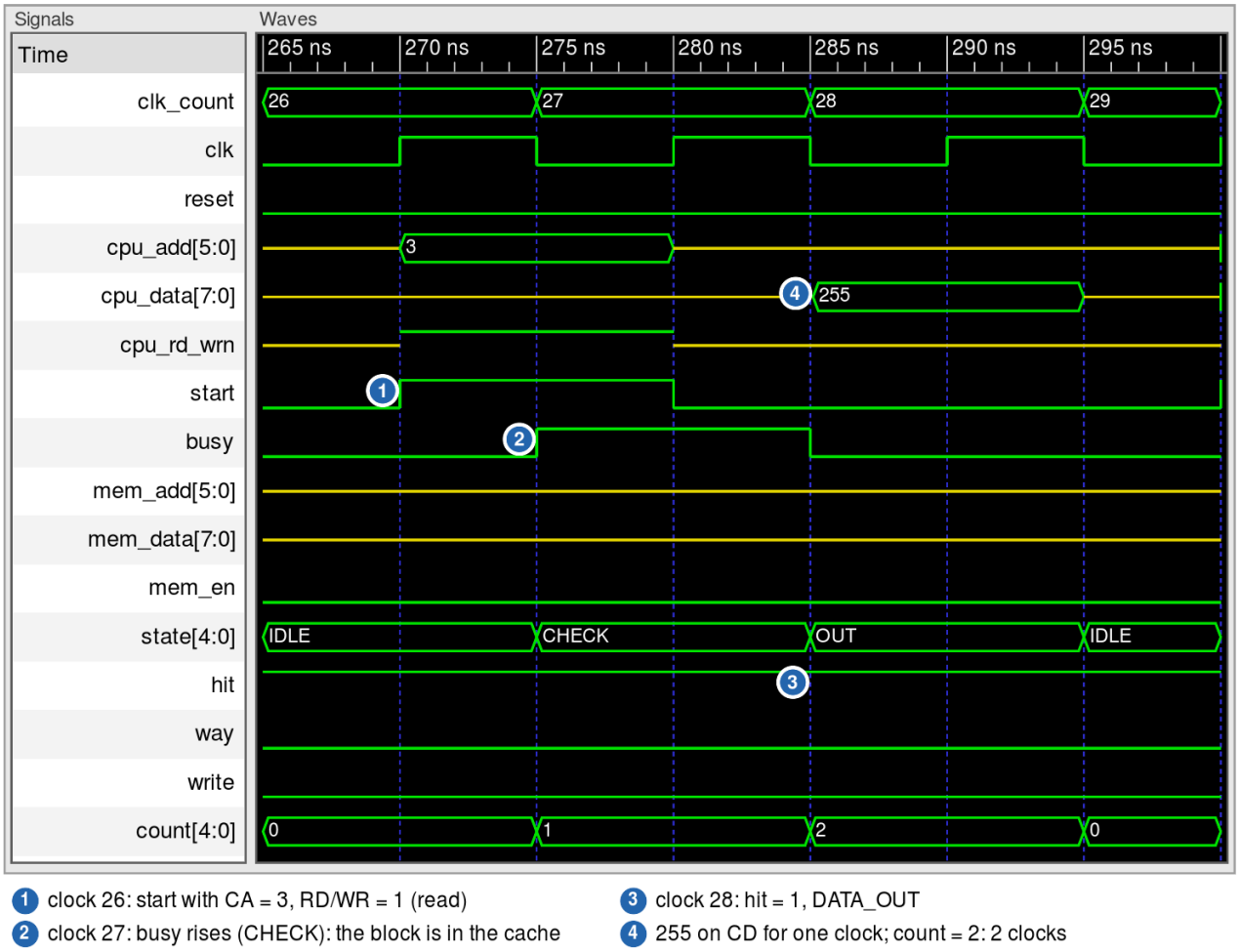


Figure 78: chip_test: read hit (0x03, 2 clocks)

Read 0x03: busy for one clock (CHECK, hit = 1), then DATA_OUT puts 255 (0xFF, written by the write hit) on CPU_DATA for one clock while busy is low: 2 clocks (count = 2).

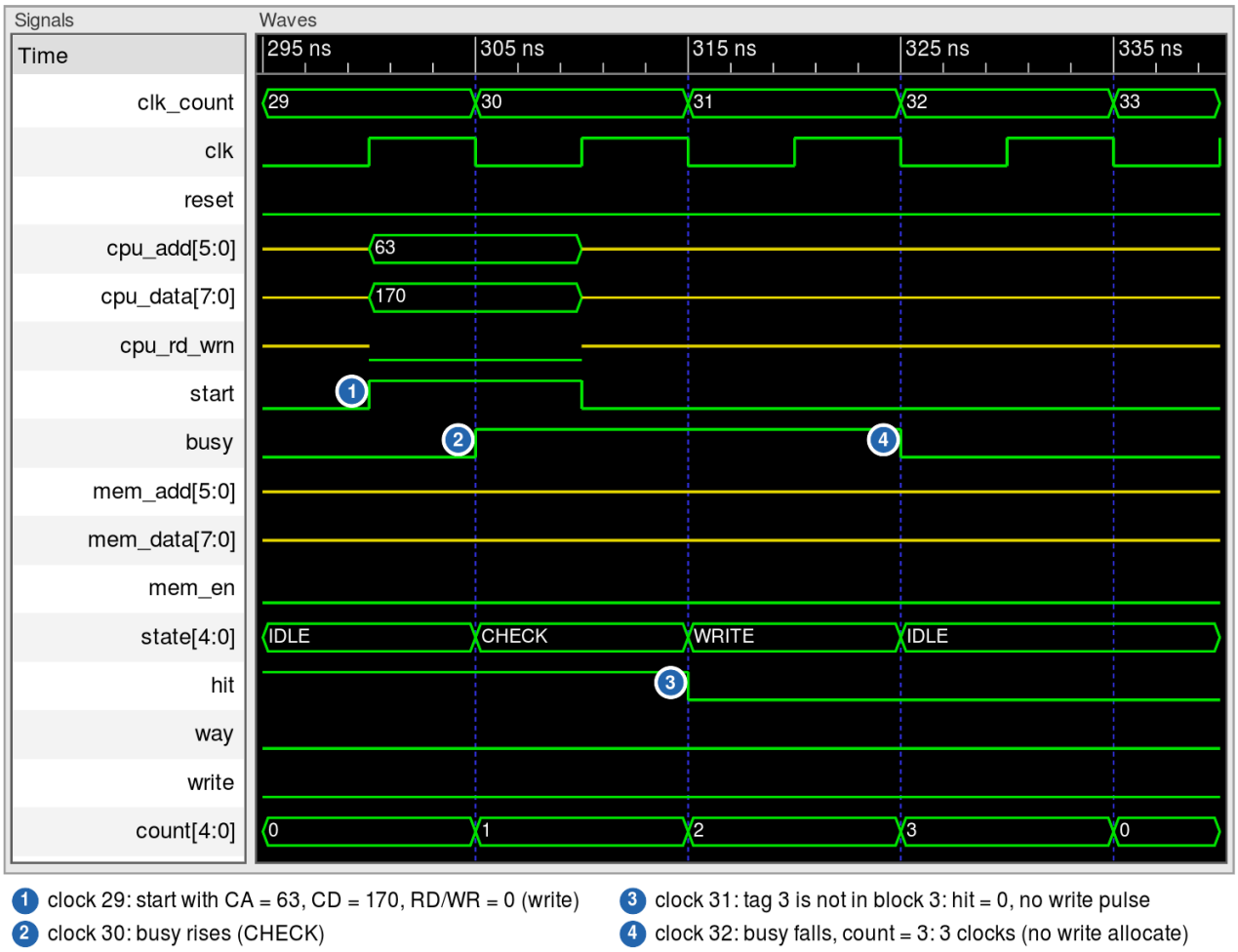


Figure 79: chip_test: write miss (0xAA to 0x3F, 3 clocks)

Write 0xAA (170) to 0x3F (63): tag 11 is not in block 3 of either way, so hit = 0 and no write pulse is made (write through with no write allocate: the cache is not changed); busy for 2 clocks, low on the 3rd negative edge: 3 clocks (count = 3).

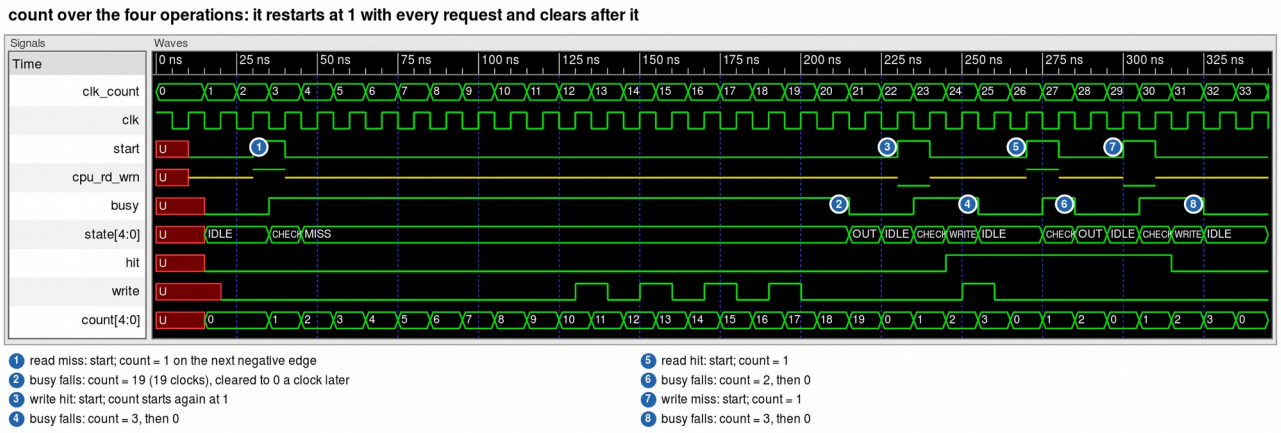


Figure 80: chip_test: the clock cycle counter over the four operations

The same run as the four figures above (chip_test with the course input file). count is started at 1 on the negative edge that accepts a request (busy goes high), counts while busy is high, and after the edge where busy goes low holds the request's length: 19 (read miss), 3 (write hit), 2 (read hit), 3 (write miss). One clock later it is cleared to 0; a request that follows at once starts it at 1 again.

9 Notes for the TA: running the simulations

The design is in the folder two_way/ of the submission: the VHDL in primitives/, gates/, blocks/, cache/, control/, cache_pkg.vhd and chip.vhd; the testbenches, the course testbench chip_test.vhd, its input files and its output chip_out.log in testbench/. Everything is compiled as VHDL-93 with relaxed rules, because chip_test.vhd uses VHDL-87 file declarations and the Synopsys std_logic_arith package. The code, this report and a README with these instructions are also on GitHub: https://github.com/aali12-eng/CMPE413-Cache-Chip-2Way . In the course lab (Cadence Xcelium and SimVision), from two_way/:

```
./run_two_way.sh            compile, run chip_test, open SimVision with the waveforms
./run_two_way.sh text       compile, run chip_test for 360 ns in text mode (testbench/xrun.log)
./run_two_way.sh verify     compile, run chip_test and check every request's clocks
./run_two_way.sh tests      compile, run every testbench tb_*.vhd (logs in testbench/logs/)
./run_two_way.sh tb_NAME    one testbench in SimVision, e.g. ./run_two_way.sh tb_counter

With GHDL (the figures of this report), in the order of Section 3.3:
ghdl -a --std=93c --ieee=synopsys -frelaxed-rules cache_pkg.vhd primitives/*.vhd ... chip.vhd
cd testbench
ghdl -a --std=93c --ieee=synopsys -frelaxed-rules chip_test.vhd
ghdl --elab-run --std=93c --ieee=synopsys -frelaxed-rules chip_test --stop-time=360ns > chip_out.log
```

Testbench	Simulates	Input	Output
chip_test (course)	chip	two_way/testbench/ chip_in.txt (format: Section 8.1)	printed on every clock edge → chip_out.log
tb_chip	chip	built in (CPU, memory and reference models)	PASS/FAIL report + statistics
tb_state_machine	state_machine + counter	built in	PASS/FAIL report
tb_cache, tb_cache_block, tb_hit_miss	cache blocks	built in	PASS/FAIL report
tb_selector, tb_cache_cell, tb_register_n, tb_counter	building blocks	built in	PASS/FAIL report

9.1 Assumptions

- Write through: the chip interface has no memory write port (MA is driven only with enable, MD is an input), so writes update the cache on a hit and the memory side of write through is outside this chip.
- Gate delay 100 ps (Section 3.2); the course primitives have zero delay, as provided.
- The CPU follows the protocol of the specification: it removes its signals on the positive edge after busy, and starts a new request no earlier than the positive edge after busy goes low.

10 Breakup of work

Table 13: Breakup of work

Task	Asif Ali (RE16458)
Cache cell, cache block, cache	Done
State machine and counter	Done
Chip top level	Done
Testbenches and simulation	Done
Report	Done

Appendix A Course testbench files and output

`chip_test.vhd` (unchanged from the course), and the complete output of this design.

Listing: two_way/testbench/chip_test.vhd

```
-- Entity: chip_test
-- Architecture : test
-- Author: cpatel2
-- Created On: 11/01/05
--
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_textio.all;
use IEEE.std_logic_arith.all;
use STD.textio.all;

entity chip_test is

end chip_test;

architecture test of chip_test is

  component chip
    port (
      cpu_add    : in  std_logic_vector(5 downto 0);
      cpu_data   : inout  std_logic_vector(7 downto 0);
      cpu_rd_wrn : in  std_logic;
      start      : in  std_logic;
      clk        : in  std_logic;
      reset      : in  std_logic;
      mem_data   : in  std_logic_vector(7 downto 0);
      Vdd     : in  std_logic;
      Gnd        : in  std_logic;
      busy       : out std_logic;
      mem_en     : out std_logic;
      mem_add    : out std_logic_vector(5 downto 0));
  end component;



  for c1 : chip use entity work.chip(structural);

  signal Vdd, Gnd: std_logic;
  signal cpu_data, mem_data: std_logic_vector(7 downto 0);
  signal cpu_add, mem_add: std_logic_vector(5 downto 0);
  signal cpu_rd_wrn, reset, clk, start, clock, busy, mem_en: std_logic;

  signal clk_count: integer:=0;

procedure print_output is
   variable out_line: line;

   begin
   write (out_line, string' (" Clock: "));
   write (out_line, clk_count);
   write (out_line, string'(" Start: "));
   write (out_line, start);
   write (out_line, string'(" Cpu Read/Write: "));
   write (out_line, cpu_rd_wrn);
   write (out_line, string'(" Reset: "));
   write (out_line, reset);
   writeline(output, out_line);

   write (out_line, string' (" CPU address: "));
   write (out_line, cpu_add);
   write (out_line, string'(" CPU data: "));
   write (out_line, cpu_data);
   writeline(output, out_line);

   write (out_line, string'(" Memory data: "));
   write (out_line, mem_data);
```

```
   writeline(output, out_line);
   writeline(output, out_line);

   write (out_line, string'(" Busy: "));
   write (out_line, busy);
   write (out_line, string'(" Memory  Enable: "));
   write (out_line, mem_en);
   writeline(output, out_line);

   write (out_line, string'(" Memory  Address: "));
   write (out_line, mem_add);
   writeline(output, out_line);

   write (out_line, string'(" ------------------------------------------------"));
   writeline(output, out_line);


end print_output;



begin

  Vdd <= '1';
  Gnd <= '0';
  clk <= clock;

  c1 : chip port map (cpu_add, cpu_data, cpu_rd_wrn, start, clk, reset, mem_data, Vdd, Gnd, busy, mem_en,
mem_add);

  clking : process
  begin
    clock<= '1', '0' after 5 ns;
    wait for 10 ns;
  end process clking;

  io_process: process

    file infile  : text is in "./chip_in.txt";
    variable out_line: line;
    variable buf: line;
    variable value: std_logic_vector(7 downto 0);
    variable value0: std_logic_vector(5 downto 0);
    variable value1: std_logic;

  begin

    while not (endfile(infile)) loop

      wait until rising_edge(clock);
      print_output;

      readline(infile, buf);
      read(buf, value0);
      cpu_add <= value0;

      readline(infile, buf);
      read(buf, value);
      cpu_data <= value;

      readline(infile, buf);
      read(buf, value1);
      cpu_rd_wrn <= value1;

      readline(infile, buf);
      read(buf, value1);
      start <= value1;

      readline(infile, buf);
      read(buf, value1);
      reset <= value1;

      wait until falling_edge(clock);

      readline(infile, buf);
```

```
      read(buf, value);
      mem_data <= value;

      clk_count <= clk_count+1;

      print_output;

    end loop;
    wait;

  end process io_process;


end test;
```

Listing: two_way/testbench/chip_out.log (complete output, 360 ns)

```
 Clock: 0 Start: U Cpu Read/Write: U Reset: U
 CPU address: UUUUUU CPU data: UUUUUUUU
 Memory data: UUUUUUUU

 Busy: U Memory  Enable: U
 Memory  Address: ZZZZZZ
 ---------------------------------------------
 Clock: 0 Start: 0 Cpu Read/Write: Z Reset: 1
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: UUUUUUUU

 Busy: U Memory  Enable: U
 Memory  Address: ZZZZZZ
 ---------------------------------------------
 Clock: 1 Start: 0 Cpu Read/Write: Z Reset: 1
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 0 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 ---------------------------------------------
 Clock: 1 Start: 0 Cpu Read/Write: Z Reset: 1
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 0 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 ---------------------------------------------
 Clock: 2 Start: 0 Cpu Read/Write: Z Reset: 1
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 0 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 ---------------------------------------------
 Clock: 2 Start: 1 Cpu Read/Write: 1 Reset: 0
 CPU address: 000000 CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 0 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 ---------------------------------------------
 Clock: 3 Start: 1 Cpu Read/Write: 1 Reset: 0
 CPU address: 000000 CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 1 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 ---------------------------------------------
 Clock: 3 Start: 0 Cpu Read/Write: Z Reset: 0
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 1 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 ---------------------------------------------
 Clock: 4 Start: 0 Cpu Read/Write: Z Reset: 0
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ
```

```
Busy: 1 Memory  Enable: 1
Memory  Address: 000000
-----------------------------------------------
Clock: 4 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 1
Memory  Address: 000000
-----------------------------------------------
Clock: 5 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 5 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 6 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 6 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 7 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 7 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 8 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 8 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 9 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 9 Start: 0 Cpu Read/Write: Z Reset: 0
```

```
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 10 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 10 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 11 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 11 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 12 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000000

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 12 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000000

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 13 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000000

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 13 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000000

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 14 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000001

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 14 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000001

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
```

```
-------------------------------------------
Clock: 15 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000001

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-------------------------------------------
Clock: 15 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000001

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-------------------------------------------
Clock: 16 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000010

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-------------------------------------------
Clock: 16 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000010

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-------------------------------------------
Clock: 17 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000010

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-------------------------------------------
Clock: 17 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000010

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-------------------------------------------
Clock: 18 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000011

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-------------------------------------------
Clock: 18 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000011

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-------------------------------------------
Clock: 19 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000011

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-------------------------------------------
Clock: 19 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: 00000011

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-------------------------------------------
Clock: 20 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ
```

```
Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 20 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 21 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: 00000000
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 21 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: 00000000
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 22 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 22 Start: 1 Cpu Read/Write: 0 Reset: 0
CPU address: 000011 CPU data: 11111111
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 23 Start: 1 Cpu Read/Write: 0 Reset: 0
CPU address: 000011 CPU data: 11111111
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 23 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 24 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 24 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 25 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 25 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
```

```
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 26 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 26 Start: 1 Cpu Read/Write: 1 Reset: 0
CPU address: 000011 CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 27 Start: 1 Cpu Read/Write: 1 Reset: 0
CPU address: 000011 CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 27 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 28 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: 11111111
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 28 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: 11111111
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 29 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 29 Start: 1 Cpu Read/Write: 0 Reset: 0
CPU address: 111111 CPU data: 10101010
Memory data: ZZZZZZZZ

Busy: 0 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 30 Start: 1 Cpu Read/Write: 0 Reset: 0
CPU address: 111111 CPU data: 10101010
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
Clock: 30 Start: 0 Cpu Read/Write: Z Reset: 0
CPU address: ZZZZZZ CPU data: ZZZZZZZZ
Memory data: ZZZZZZZZ

Busy: 1 Memory  Enable: 0
Memory  Address: ZZZZZZ
-----------------------------------------------
```

```
 Clock: 31 Start: 0 Cpu Read/Write: Z Reset: 0
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 1 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 --------------------------------------------
 Clock: 31 Start: 0 Cpu Read/Write: Z Reset: 0
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 1 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 --------------------------------------------
 Clock: 32 Start: 0 Cpu Read/Write: Z Reset: 0
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 0 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 --------------------------------------------
 Clock: 32 Start: 0 Cpu Read/Write: Z Reset: 0
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 0 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 --------------------------------------------
 Clock: 33 Start: 0 Cpu Read/Write: Z Reset: 0
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 0 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 --------------------------------------------
 Clock: 33 Start: 0 Cpu Read/Write: Z Reset: 0
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 0 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 --------------------------------------------
 Clock: 34 Start: 0 Cpu Read/Write: Z Reset: 0
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 0 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 --------------------------------------------
 Clock: 34 Start: 0 Cpu Read/Write: Z Reset: 0
 CPU address: ZZZZZZ CPU data: ZZZZZZZZ
 Memory data: ZZZZZZZZ

 Busy: 0 Memory  Enable: 0
 Memory  Address: ZZZZZZ
 --------------------------------------------
ghdl:info: simulation stopped by --stop-time @360ns
```

Appendix B Testbench helper package

Listing: two_way/testbench/tb_pkg.vhd

```
-- Helpers for the self-checking testbenches (VHDL-93: no to_string in the standard library)
library ieee;
use ieee.std_logic_1164.all;

package tb_pkg is
    function str (v : std_logic_vector) return string;
    function str (b : std_logic) return string;
    function str (n : integer) return string;
end package tb_pkg;

package body tb_pkg is
```

```
    function str (b : std_logic) return string is
        constant img : string(1 to 3) := std_logic'image(b);   -- e.g. '1'
    begin
        return img(2 to 2);
    end function;

    function str (v : std_logic_vector) return string is
        variable s : string(1 to v'length);
        variable k : positive := 1;
    begin
        for i in v'range loop
            s(k to k) := str(v(i));
            k := k + 1;
        end loop;
        return s;
    end function;

    function str (n : integer) return string is
    begin
        return integer'image(n);
    end function;
end package body tb_pkg;
```